

Fragmentación semántica y ensamblaje estocástico, versión 2

Un protocolo de inferencia descentralizada de modelos de lenguaje sobre nodos voluntarios no confiables

Sebastián A. Espinoza-Ulloa, Ph.D. Investigador independiente ORCID: [0000-0003-1497-356X](https://orcid.org/0000-0003-1497-356X) · GitHub: [@Sebastardito](https://github.com/Sebastardito)

Nota sobre afiliación e independencia. Este trabajo se realiza íntegramente a título personal como investigador independiente. El autor mantiene afiliaciones académicas — Pontificia Universidad Católica del Ecuador y University of Saskatchewan— ajenas al objeto de este trabajo. **Ninguna ha aportado financiación, materiales, recursos de cómputo, personal ni apoyo institucional a este proyecto, y no se reclama ni se implica respaldo institucional alguno.**

Antecedente relevante. El autor es Ph.D. en Biología (University of Saskatchewan) en genómica de poblaciones de aves, y trabaja en secuenciación de genoma completo, llamado de variantes y ensamblaje *de novo*. El marco de ensamblaje genómico que recorre este documento procede de ese antecedente; la sección 3 establece el criterio por el cual una analogía de ese origen se acepta o se descarta.

Versión 2.0 — 29 de septiembre de 2026

Estado de este documento: PUBLICADO

Versión 2.0, del 29 de septiembre de 2026. Sustituye a la versión 1.4, del 14 de agosto de 2026, que se conserva como WHITEPAPER_ES.md.

Los modelos nuevos se sometieron a su propia prueba. Los cinco se midieron sobre **473 corridas reales** con cinco familias de modelos locales (§15.9). **M4 queda falsado** en dos instrumentos independientes, con un experimento controlado a p constante. **M2 sobrevive corregido**, y la corrección retira una frase de este mismo documento. M3 sobrevive. M1 sigue sin el régimen que necesita y M5 sin medir. El documento se publica **con** esos veredictos dentro.

Arte previo. Los elementos E19-E24 que declara la sección 17 pasan a ser arte previo **con la publicación de este documento.**

Lo que este documento no afirma está en L21 y en §15.9: con presupuesto de salida igualado fragmentar cuesta (T09R), pero el agregado sigue confundido con la longitud de salida, así que el criterio de abandono **no se ha cumplido ni incumplido — no se ha medido.** **Lo que sí afirma, con su alcance declarado**, está en §15.4: extraer con el modelo y agregar con código supera al modelo solo (T08R3), y partir aporta por sí mismo cuando la tarea supera lo que un nodo pequeño extrae bien (T08R4).

Índice

0. Qué es esta versión y qué cambia respecto de la 1.4
1. Introducción
2. Antecedentes y trabajo relacionado
3. El método: qué hace que una homología sirva
4. Principios de diseño
5. La unidad semántica y el fragmento
6. El presupuesto de contexto
7. Inteligencia de enjambre con modelos pequeños
8. Arquitectura
9. Especificación del protocolo (v0.3)
10. Algoritmos
11. Restricciones de cardinalidad
12. Redundancia: para qué sirve k
13. Privacidad, verificación y nodos adversarios
14. Economía, gobernanza y sostenibilidad
15. Evaluación
16. Limitaciones y resultados negativos
17. Declaración de arte previo
18. Conclusión
19. Referencias

0. Qué es esta versión y qué cambia respecto de la 1.4

La versión 1.4 describe una arquitectura y reporta lo que se midió de ella, incluyendo dos retiradas de calado: la curva del impuesto de coherencia decreciente en ρ , y la afirmación de fiabilidad del mapa de confianza. Esta versión conserva íntegras ambas retiradas —no se rescatan por la puerta de atrás— y añade lo que la 1.4 no tenía: **una teoría de la fragmentación misma**.

La 1.4 daba por sentado que un fragmento es «una parte de tamaño parecido a las demás» y que el solapamiento es «un porcentaje». Ninguna de las dos cosas estaba derivada. Esta versión las deriva, y al derivarlas cambia decisiones concretas de planificación, empaquetado, ensamblaje y despacho.

Tabla de cambios.

en la v1.4	en esta versión
El codón como unidad mínima de significado	Retirado. Tres de tres propiedades no transfieren (§5.1). La oración es homóloga del aminoácido —unidad de alineamiento—; el dominio proteico es homólogo del fragmento —unidad de trabajo— (§5.2, §5.3)
Fragmentar en N partes de tamaño parecido	Reemplazado por corte en interfaces de acoplamiento débil, con L como objetivo y N derivado: $N = \lceil \text{material}/L \rceil$ (§5.4)
Solapamiento como porcentaje del fragmento	Reemplazado por $F = \text{máximo entre la repetición más larga y la dependencia más larga que cruza un límite. Ahorro medido: 29 \% del cómputo de la red}$ (§6.5)
ρ como parámetro que se barre	Derivada de L , F y el costo de cabecera H ; y formalizada como la tasa de un problema de tasa-distorsión indirecto, con un segundo eje δ (§6.6-6.8)
Mapa de confianza por conteo de coincidencias	Reemplazado como mecanismo por alineamiento múltiple con costo de sustitución aprendido y extensión por consistencia ($M1$, §10.5). La afirmación de fiabilidad sigue retirada (§15.3, L13)

en la v1.4	en esta versión
term_once impuesto en el ensamblador	Complementado por asignación de unicidad en el plan, al modo U-unitig (M2, §11)
DAG sin control entre niveles	Extendido con compuerta de triage por nivel, por analogía con chaperonas (M3, §8.2, §10.7)
k réplicas como un parámetro	Separado en disponibilidad, verificación y redundancia epistémica (M5, §12)
Analogía genómica como vocabulario más una advertencia	Formalizada como método explícito: prueba del instrumento y prueba del modo de fallo (§3)
Elementos declarados E1-E18	E1-E18 sin cambios , más E19-E24 (§17), publicados con esta versión
Los cinco modelos M1-M5, enunciados y sin medir	Medidos contra 473 corridas reales sobre cinco familias de modelos locales (§15.9). Uno queda falsado (M4, en dos instrumentos independientes), uno sobrevive corregido (M2), uno sobrevive (M3) y dos siguen sin medir (M1, M5)

Dos hipótesis entraron a la revisión bibliográfica y **no sobrevivieron**: los *mate pairs* como homólogo de las restricciones globales (§11.1) y los *fountain codes* como mecanismo de redundancia (§12.1). Las dos se documentan con más espacio que sus reemplazos, porque la caída es el resultado.

Esta versión llega con su propia campaña de medición. La 1.4 se publicó con un experimento; ésta se acompaña de una implementación de referencia (`swarmbly_ref/`), un arnés de falsación (`swarmbly_validation/`) y un corpus de **473 corridas** sobre cinco familias de modelos locales, los tres publicados junto al documento. §15.9 reporta qué mató esa campaña. Conviene decirlo aquí porque cambia el carácter del documento: las afirmaciones de §15 ya no descansan en una corrida que el lector deba creer, sino en un artefacto que puede volver a ejecutar.

1. Introducción

1.1 La concentración está en el capital, no en el conocimiento

La capacidad de construir modelos de lenguaje ya no es escasa. Los pesos, las recetas de entrenamiento y los motores de inferencia se publican abiertamente y mejoran cada mes. Lo que sigue siendo escaso —y lo que concentra poder— es el capital necesario para *operarlos* a escala: los aceleradores, los edificios, los contratos de energía y la interconexión.

Esa concentración tiene forma medible. Los centros de datos consumieron 415 TWh en 2024, cerca del 1.5 % de la electricidad mundial, con proyecciones a 945 TWh en 2030 [83]. Las instalaciones a hiperescala en Estados Unidos se abastecen de redes medidas en 545 gCO₂/kWh frente a una media nacional de 370 g [84]. Son las cifras de una industria cuya vía de crecimiento pasa por la construcción, y construir sólo está al alcance de quien puede financiarlo.

La consecuencia es estructural y no conspirativa: una tecnología cuyo *conocimiento* es público se vuelve, en la práctica, controlable por quien puede pagar el *hardware*. La apertura de los pesos no democratiza una capacidad cuya operación cuesta cientos de millones.

Y sin embargo el hardware ya existe, distribuido y ocioso. La plataforma de computación voluntaria de referencia agrega aproximadamente 700.000 dispositivos activos, 4 millones de núcleos de CPU y 560.000 GPU a una media de 93 PetaFLOPS [47] — y lo hace desde una base de participantes que ha encogido de cerca de un millón a unos doscientos mil en dos

décadas. Ese número es un *piso*, extraído de un nicho en declive, no una proyección de lo que un protocolo convincente podría movilizar. A nivel de nodo individual, GPU de consumo ociosas sirven inferencia a \$0.111–0.149 por millón de tokens en una RTX 4090, al 62–78 % del rendimiento de una H100 por aproximadamente la mitad del costo [49].

La capacidad de inferencia sobrante del mundo no es una hipótesis. Lo que falta es un protocolo bajo el cual pueda usarse — y la razón de que no exista es una restricción física que la sección siguiente enuncia con exactitud.

1.2 La restricción física y el replanteamiento

Cualquier arquitectura de inferencia distribuida sobre hardware de consumo queda decidida, antes de elegir ningún algoritmo, por una medición. Una NVIDIA H100 SXM mueve 900 GB/s por GPU sobre NVLink; Quantum-2 InfiniBand ofrece 400 Gb/s por puerto y 51.2 Tb/s agregados por conmutador. El ancho de banda de subida típico de consumo es del orden de 60 Mbps. La razón es aproximadamente $120.000\times$ contra NVLink y $6.700\times$ contra InfiniBand. La latencia intra-nodo de NVLink es submicrosegundo y la de InfiniBand de microsegundos de un dígito, frente a 30–170 ms de ida y vuelta en área amplia: **cuatro a cinco órdenes de magnitud** [22, 23, 24].

Ese único hecho parte el espacio de diseño en dos. Las arquitecturas que requieren comunicación *por token* chocan contra la brecha en cada paso de generación; las que la cruzan *una vez por unidad de trabajo* no. Todo lo demás en este documento se sigue de elegir la segunda clase.

El comportamiento medido de la primera clase concuerda con la predicción. Petals —la implementación de referencia de inferencia con paralelismo de tubería sobre internet— sirve Llama-2-70B en tres T4 a 2.29 pasos/s sobre un enlace de 1 Gbit/s con RTT menor a 5 ms, y cae a 1.57 pasos/s a 100 Mbit/s y 100 ms: un 31 % de pérdida atribuible sólo a la red. Un enjambre geodistribuido real de catorce servidores heterogéneos alcanza 0.83 pasos/s [1, 2]. Los análisis de esquemas de paralelismo de modelo a latencia de internet público encuentran que el paralelismo de tubería es la disposición más adecuada —es la que menos comunica— y que el micro-batching asíncrono no ayuda, porque la decodificación está limitada por el movimiento de la caché KV y no por el cómputo [25].

La conclusión que extraigo no es que el paralelismo de tubería se haya implementado mal. Es que es la respuesta correcta a la pregunta equivocada.

Swarmbly hace otra pregunta: en lugar de *cómo correr un modelo grande a través de muchas máquinas*, **cómo correr muchos modelos pequeños completos sobre un problema grande**.

No son variantes de la misma idea. Partir un modelo crea una cadena en la que el nodo k no puede empezar hasta que el $k-1$ termina, y en la que cada token recorre la cadena entera. Partir un problema crea un conjunto —más precisamente un orden parcial— en el que subtarefas independientes avanzan en paralelo y cada una cruza la red una vez. La primera está acotada por $\sum_i (t_{\text{cómputo},i} + t_{\text{red},i})$; la segunda por $\max_i (t_{\text{cómputo},i} + t_{\text{red},i})$ más el ensamblaje local. La afirmación estructural es que la segunda es el régimen en el que el hardware voluntario puede participar siquiera.

El vocabulario de diseño se toma deliberadamente del ensamblaje shotgun de genomas. Una petición se fragmenta en *lecturas*; cada respuesta devuelta es un *contig*; los contigs adyacentes se unen por *solapamiento* y, donde discrepan, por *consenso*; el plan que los ordena es un *andamio*. La sección 3 establece exactamente qué autoriza y qué no autoriza esa analogía, y lo hace con un criterio explícito en lugar de con una advertencia general — que es el cambio metodológico central de esta versión.

1.3 Qué hace posible esto

La primera medición del criterio de abandono está hecha y **el criterio no se cumplió**: a $\rho = 3.5$, $N = 2$, $k = 1$ sobre 16 prompts reservados, el impuesto de coherencia es +2.30 % con un IC del 95 % de $[-2.05 \%, +7.49 \%,]$, y el criterio está escrito contra la cota superior, que queda corta por 2.49 puntos (§15.3). Lo que sobrevive es más estrecho y está mejor fundado: **el prompt mediano no pierde exactamente nada (0.00 %), 11 de 16 prompts quedan en cero o por debajo**, y el control a $N = 8$ falla como se le exigía. La media está fabricada por dos prompts que entre los dos son el 140 % de ella; sin ellos, los otros catorce promedian -1.06% .

Cuatro cosas siguen siendo posibles, y ahora están motivadas por un costo bimodal medido en lugar de por una tendencia confirmada.

1. Servir capacidad sin poseerla. Un participante aporta una máquina que ya existe y que ya consume energía estando ociosa. El requisito de entrada es un modelo pequeño completo, no un fragmento de uno grande, lo que sitúa el parque de hardware direccionable órdenes de magnitud por encima de lo que alcanzan los esquemas de paralelismo de tubería. La capacidad escala entonces con la *participación* y no con el gasto de capital.

2. Un mapa de divergencia que la centralización no puede producir — con su afirmación de fiabilidad retirada. Porque una micro-tarea la responden k nodos de familias de modelo *distintas*, las respuestas pueden alinearse entre sí y el acuerdo puntuarse por unidad semántica. **Un proveedor con un solo modelo no tiene nada que alinear.** Que esa señal informe sobre corrección es una pregunta separada, y fue respondida: no, con el instrumento que se usó (§15.3, L13). Esta versión propone un instrumento distinto (M1, §10.5) y enuncia por adelantado qué lo mataría. La retirada no se levanta por proponer un reemplazo.

3. Contexto acotado por la máquina del usuario y no por la decisión de producto de un proveedor. La fragmentación traslada el límite de contexto desde una ventana fija a una función del tiempo y la memoria de ensamblaje del cliente. Con ensamblaje jerárquico, la memoria de trabajo crece logarítmicamente con el volumen total.

4. Un sustrato que se puede auditar en lugar de tener que confiar en él. El protocolo, el cliente, el software de nodo y la licencia son públicos. La fracción de tráfico servida por nodos ancla de la fundación se publica (§14.4). La contabilidad energética se publica contra un estándar público (§14.3). La degradación de coherencia se devuelve con cada respuesta (§10.8). Ninguna es una cortesía: cada una es un requisito de conformidad.

1.4 Alcance de las afirmaciones

Cuatro afirmaciones que un lector podría esperar están deliberadamente ausentes, y la sección 16 desarrolla cada una.

No afirmo **paridad de latencia**: la decodificación especulativa de un solo nodo ya entrega 2–3× con una prueba de que la distribución de salida se preserva [13], y ningún esquema de fragmentación compite con eso en velocidad. La comparación que importa es otra — para un usuario sin hardware para correr un modelo capaz, el eje relevante no es *más rápido o más lento* sino *posible o imposible*.

No afirmo **contexto ilimitado**, sólo un límite reubicado y mucho más alto.

No afirmo **confidencialidad criptográfica**. La fragmentación no es cifrado, la sección 13 da los ataques que lo zanján, y el protocolo enruta el trabajo sensible a ejecución local o hardware atestiguado.

No afirmo un **beneficio ambiental demostrado**. El argumento de carbono incorporado es fuerte y el operacional es condicional; §14.3 enuncia ambos.

Y esta versión añade una quinta ausencia. No afirmo **una cota computable para ρ** . La sección 6.7 establece que el problema de Swarmbly es un problema de tasa-distorsión indirecto, de lo que se hereda la *existencia* de una cota inferior y la monotonía cualitativa de la curva. No se hereda un número, y decir lo contrario sería exactamente el tipo de sobrelectura que la sección 3 existe para impedir.

1.5 Aportaciones y estructura

La sección 2 sitúa el trabajo frente al arte previo, incluida la evidencia adversa. La sección 3 establece el método por el cual una homología biológica se acepta o se descarta: es la aportación metodológica de esta versión y lo que organiza todo lo que sigue. La sección 4 enuncia los principios de diseño. La sección 5 desarrolla **la unidad semántica y el fragmento** — qué tamaño tiene un fragmento y dónde se corta. La sección 6 desarrolla el **presupuesto de contexto**, ahora derivado y formalizado. La sección 7 formaliza la tesis del **enjambre de modelos pequeños**. Las secciones 8 a 10 dan arquitectura, protocolo y algoritmos. Las secciones 11 y 12 tratan las restricciones de cardinalidad y el papel de la redundancia. La sección 13 cubre privacidad y verificación; la 14, economía y gobernanza. La sección 15 reporta lo medido y especifica lo que falta medir, incluido el criterio bajo el cual abandonaría el diseño. La 16 lista limitaciones y resultados negativos. La 17 declara los elementos divulgados.

2. Antecedentes y trabajo relacionado

2.1 Inferencia descentralizada por partición del modelo

Petals [1, 2] distribuye bloques contiguos de capas de transformador entre voluntarios; los clientes guardan los *embeddings* localmente y enrutan activaciones por una cadena de servidores. Lo trato como pionero del campo y no como competidor: demostró que la inferencia entre pares sobre internet público es posible, que es la precondition de este trabajo. Swarmbly no mejora el paralelismo de tubería; renuncia a usarlo, y esa divergencia es de estrategia y no de calidad. Hivemind y SWARM parallelism [4, 5] abordan entrenamiento tolerante a fallos sobre dispositivos heterogéneos poco fiables con la misma premisa de fondo: el modelo es la unidad de distribución.

Bittensor [6, 7] añade una capa de incentivos, con un mecanismo de consenso cuya regularización basada en conectividad se describe como resistente a colusión de hasta el 50 % del peso de la red — una formulación que, leída con cuidado, presupone un ancla de confianza.

2.2 Entrenamiento descentralizado sobre enlaces lentos

El subcampo que más ha avanzado es el entrenamiento, y avanzó atacando el volumen de comunicación en lugar de la topología. DiLoCo iguala la optimización completamente síncrona comunicando 500× menos [8]. OpenDiLoCo entrenó a través de dos continentes con 90–95 % de utilización de cómputo [9]. INTELLECT-1 entrenó un modelo de 10B sobre 1T tokens con hasta 14 nodos concurrentes en tres continentes y 400× de reducción de ancho de banda [10].

La lección que Swarmbly toma es metodológica: el problema de ancho de banda cede ante la compresión y la asincronía.

2.3 Paralelismo a nivel de tarea

Skeleton-of-Thought (SoT) [12] es el precedente directo de descomponer un *prompt*: un *prompt* de esqueleto produce una lista de puntos, cada uno expandido de forma independiente y en paralelo. Reporta hasta 2.39× de aceleración y —esto importa más— reporta su propio daño: la calidad mejora en conocimiento, genérico, sentido común, *roleplay* y contrafácticos, y se degrada en matemáticas, programación, escritura y estimación de Fermi. Los autores enuncian la causa estructural sin rodeos: «SoT currently ignores the dependencies between points».

Su respuesta no fue defender el método sino condicionarlo. SoT-R [12] añade un router que decide por pregunta si descomponer; un router RoBERTa de 120M basta, y se entrena con pérdida de Tversky precisamente para penalizar falsos positivos.

Los descendientes refinan la idea. APAR [16] hace que el modelo planifique sus propias ramas. PASTA [17] aprende un lenguaje de anotación para tramos semánticamente independientes y reporta aceleraciones de 1.21–1.93× con un delta de tasa de victoria de +2.2 % a −7.1 %. Plato/ASGD [18] reemplaza la lista plana por un **grafo de dependencias** y reporta 68 % de ganancia de rendimiento. Hogwild! Inference [19] toma el camino opuesto: trabajadores concurrentes compartiendo una caché KV viva.

ParallelBench [20] aporta la teoría: el supuesto de independencia condicional que subyace a la generación paralela «inevitably degrad[es] generation quality when dependencies are strong». Tran y Kiela [21] dan la versión teórico-informacional vía la desigualdad de procesamiento de datos.

Lo que falta en esa literatura es mi objeto: nadie ha combinado descomposición a nivel de prompt con despacho a nodos voluntarios no confiables. Esa intersección, y no cualquiera de sus mitades, es lo que este documento divulga.

2.4 El vecino más cercano, y la evidencia adversa

Dos cosas han cambiado desde la v1.4 y las dos hay que enfrentarlas de frente.

El vecino más cercano ya existe. SWARM-LLM (Dahshan, Mamun & Debnath, VTC 2026) [134] enruta una consulta a un SLM local, a varios SLM pares en el borde con **consenso ponderado donde los nodos de menor incertidumbre pesan más**, o escala a la nube. Reporta ~28 % de uso de nube y exactitud en preguntas difíciles de 0 % a 15 % sobre una carga de **50 consultas**. Es evidencia empírica débil —50 consultas— pero reclama el territorio, y hay que citarlo y diferenciarse de él. La diferencia es que su consenso es una ponderación escalar por nodo, y el de Swarmbly es una correspondencia posición a posición (§10.5): la suya dice *cuánto* confiar en cada nodo, la nuestra dice *dónde* divergieron.

El acuerdo semántico entre modelos distintos ya se usa como señal. Soiffer, Kolawole & Smith (2025) [101] usan el acuerdo entre modelos pequeños como señal de derivación hacia uno mayor: «when independently generated outputs are semantically consistent — even if lexically distinct — their agreement suggests the underlying meaning is reliable». Arquitectónicamente es lo más cercano al mapa de confianza. También agrupa en vez de alinear, que es la apertura que §10.5 ocupa.

Y la crítica seria a toda la familia. «Why Do Multi-Agent LLM Systems Fail?» (Cemri et al., 2025) [90] construye la taxonomía MAST: **14 modos de fallo en 3 categorías** —problemas de diseño del sistema, desalineación entre agentes, verificación de tareas— a partir de más de 1.600 trazas anotadas sobre **7 frameworks**, con 150 trazas validadas por anotadores expertos a $\kappa = 0.88$. «If Multi-Agent Debate is the Answer, What is the Question?» (Zhang et al., 2025) [135] encuentra que los métodos de debate multi-agente «fail to consistently

outperform single-agent baselines such as Chain-of-Thought and Self-Consistency, even when consuming additional inference-time compute», sobre 9 benchmarks, 4 modelos y 5 métodos.

La respuesta honesta de Swarmbly no es que su arquitectura sea inmune. Es que **las tres categorías de MAST son exactamente las tres que esta versión aborda**:

categoría MAST	dónde se aborda aquí
Problemas de diseño del sistema	§5.4 regla de corte por interfaz; §11.3 unicidad asignada en el plan
Desalineación entre agentes	§10.5 alineamiento con sustitución semántica; §9.2 contrato global
Verificación de tareas	§10.7 compuerta de triage por nivel

Que una taxonomía construida independientemente sobre 1.600 trazas coincida con las tres áreas donde las homologías biológicas apuntaron es, si no una validación, al menos una convergencia que vale la pena declarar.

La crítica de Zhang et al. es además más específica de lo que parece: apunta al **debate** —agentes que discuten para converger— y no a la **descomposición** —agentes que resuelven partes disjuntas. Swarmbly no debate. Pero la lección transfiere igual, y esta versión la adopta como requisito: **el cómputo adicional debe justificarse contra una línea base de un solo agente con self-consistency**, no contra el modelo monolítico ingenuo. La sección 15.6 lo incorpora al protocolo de evaluación.

2.5 Compresión de prompts como problema de tasa-distorsión

Nagle et al. (NeurIPS 2024) [123] formalizan la **compresión de prompts** como un problema de tasa-distorsión para modelos de lenguaje de caja negra. Su tasa es $E[\text{len}(M)/\text{len}(X)]$ —longitud esperada del prompt comprimido sobre la del original— que es **estructuralmente la misma cantidad que ρ** . Su distorsión es la degradación de rendimiento bajo pérdida logarítmica o 0/1. Derivan la función distorsión-tasa mediante el dual de un programa lineal y reportan una brecha grande entre los métodos actuales y la estrategia óptima.

La consecuencia para este documento es directa y limitante: **la aportación no puede ser « ρ es un problema de tasa-distorsión», porque eso está publicado**. La aportación tiene que ser ρ **bajo fragmentación con reensamblado distribuido**, que es un problema distinto y más duro, y que Nagle et al. no cubren. La sección 6.7 lo desarrolla así, y la 6.8 añade el segundo eje que su encuadre no necesita y el nuestro sí.

2.6 Ensamblaje de genomas

La estadística de cobertura de Lander-Waterman [26] da, para un genoma de longitud G muestreado por N clones de longitud L con fracción mínima detectable de solapamiento θ :

$c = L \cdot N / G$	(redundancia de cobertura)
$P(\text{base no cubierta})$	$= e^{(-c)}$
$E[\# \text{ islas aparentes}]$	$= N \cdot e^{(-c \cdot \theta)}$
$E[\# \text{ clones por isla}]$	$= e^{(c \cdot \theta)}$

Este modelo transfiere, pero sólo tras una corrección que versiones anteriores de este trabajo no hacían bien. La diferencia relevante entre ensamblaje de genomas y ensamblaje de texto **no** es que la secuencia objetivo sea *conocida*: en ensamblaje *de novo* no existe referencia. La diferencia real es más estrecha y más útil: en genómica existe **una sola molécula física de la que toda lectura es una muestra**. Esa unicidad es lo que garantiza que dos solapamientos verdaderos sean reconciliables. En generación de texto libre no hay tal objeto garante.

Pero el objeto garante puede fabricarse. Si el plan $\mathfrak{D}$ y el contrato global Γ se fijan *antes* de toda generación y se tratan como la referencia, entonces vuelve a existir un objeto común y la estadística de cobertura vuelve a ser aplicable. La sección 7.4 lo desarrolla, y es el punto en que la analogía deja de ser una convención de nombres y se vuelve una derivación.

Queda una transferencia genuina de la literatura de ensamblaje, y es una advertencia. En ensamblaje de de Bruijn, una repetición más larga que k colapsa en un solo nodo del grafo; el camino euleriano deja de ser único y el número de reconstrucciones válidas crece combinatoriamente [29]. Veinte años de práctica establecieron la consecuencia: **las repeticiones, no la cobertura, son la restricción vinculante** [30, 31]. La sección 6.4 convierte esa advertencia en una desigualdad con dos formas, y en un dato medido del propio proyecto.

3. El método: qué hace que una homología sirva

Esta sección es nueva y es la aportación metodológica de la versión 2. Sin ella, las secciones 5, 6, 10, 11 y 12 serían un catálogo de analogías atractivas.

3.1 El problema con las analogías

Swarmbly nació de una analogía: una petición demasiado grande para un modelo pequeño se parece a un genoma demasiado largo para un secuenciador. Se rompe, se lee por partes, se reconstruye.

Las analogías de ese tipo son productivas al principio y peligrosas después. Son productivas porque importan vocabulario maduro para un problema nuevo. Son peligrosas porque el vocabulario viene con connotaciones que no se transfieren, y porque una analogía que suena bien resiste el escrutinio precisamente por eso.

La versión 1.4 enunciaba la cautela —ningún algoritmo de ensamblaje genómico corre dentro de Swarmbly— pero no ofrecía un criterio para decidir cuándo una transferencia es real. Esta sección propone uno, y el resto del documento lo aplica caso por caso.

3.2 La prueba del instrumento

Una homología sirve cuando trae un instrumento: un procedimiento, una desigualdad o un número que se puede aplicar al problema nuevo. No sirve cuando trae sólo una manera de hablar.

El caso claro es la ecuación de cobertura. Lander & Waterman (1988) derivan que el número esperado de islas es $N \cdot e^{-c\theta}$ con $c = \ln G$. Esa derivación entrega un número —cuántas réplicas hacen falta— que antes era una conjetura. Es un instrumento, y la sección 7.4 lo usa como tal.

El caso contrario, y hay que decirlo porque es del propio proyecto: llamar *contig* a un fragmento de respuesta no hace nada. Es vocabulario. Se conserva porque es cómodo, no porque derive nada.

3.3 La prueba del modo de fallo

Al aplicar la prueba del instrumento a las homologías que este proyecto usa, emergió un patrón que no se había buscado:

Las homologías que transfieren vienen acompañadas de su modo de fallo. Las que no transfieren traen un mecanismo sin su patología asociada.

El criterio de solapamiento mínimo viene con el *mis-assembly*: si el solapamiento no excede la repetición, el ensamblador pega los tramos equivocados. El dominio proteico viene con su condición de interfaz: la independencia se pierde cuando la interfaz es grande y empaquetada. El triage de chaperonas viene con la degradación: lo irrecuperable se destruye, no se acopla.

En cambio el codón —que §5.1 descarta— se propuso como «la unidad mínima de significado» sin ningún enunciado sobre qué ocurre cuando se la viola. Los *fountain codes* —§12.1— se propusieron por su propiedad de recuperación sin ninguna condición sobre cuándo esa propiedad deja de existir.

La razón es que un campo maduro no descubre un mecanismo aislado: descubre un mecanismo y los casos donde falla, y suele publicar los segundos con más cuidado que los primeros. Una transferencia que sólo trae la parte buena está importando la conclusión sin el trabajo.

3.4 La prueba operativa

pregunta	si la respuesta es no
¿Trae un procedimiento, una desigualdad o un número?	Es vocabulario. Úsese como vocabulario y no se derive nada de ella.
¿Trae el enunciado de qué ocurre cuando la condición se viola?	Está importada a medias. Búsquese el modo de fallo antes de construir sobre ella.
¿Las condiciones del campo de origen se cumplen aquí?	Transferencia inválida. Dígase por qué, que suele ser informativo.

Las cinco homologías que este documento usa pasan las tres preguntas. Las dos que se descartan fallan la segunda, la tercera, o ambas — y en los dos casos el diagnóstico de *por qué* falla resultó más útil que la homología habría sido.

4. Principios de diseño

P1 — Cruzar la red una vez por unidad de trabajo. El único argumento de rendimiento defendible disponible para una red voluntaria.

P2 — El orquestador puede negarse. Un sistema que fragmenta toda petición es estrictamente peor que SoT en 2023, porque SoT venía con router. Fragmentar es una decisión con función de costo asimétrica, no un valor por defecto.

P3 — Modelar las dependencias explícitamente. Los planes son grafos acíclicos dirigidos. El paralelismo es el ancho de un nivel, no el tamaño del conjunto de tareas.

P4 — Seleccionar antes que sintetizar. Donde existen varios fragmentos candidatos, elegir uno y empalmarlo. Reescribir sólo donde una costura falla de verdad.

P5 — Calibrar todo umbral. Ningún corte de coseno fijo, ninguna razón de redundancia supuesta. Los umbrales se derivan de datos etiquetados por modelo y por dominio, con objetivos asimétricos.

P6 — Reportar el impuesto. Todo ensamblaje devuelve una auditoría de coherencia. Un protocolo que oculta su propia degradación no puede evaluarse.

P7 — Verificar barato o no verificar. La verificación que cuesta una fracción significativa de la inferencia destruye la economía.

P8 — Enrutar por sensibilidad, no fingir que se cifra. La confidencialidad es una decisión de enrutamiento con tres carriles, no una propiedad reclamada para la fragmentación.

Esta versión añade tres, y los tres salen de las secciones nuevas.

P9 — Cortar donde el acoplamiento es débil, no donde cae la cuenta. El límite de fragmento es una decisión sobre la interfaz, no sobre el tamaño. L es un objetivo; la interfaz es la regla. (§5.4)

P10 — Resolver las restricciones globales en la representación, no en la salida. Una restricción que ningún nodo puede cumplir aisladamente se asigna en el plan, donde sí puede cumplirse por construcción. (§11.3)

P11 — Un parámetro, un propósito. k sirve hoy a tres fines distintos —disponibilidad, verificación y redundancia epistémica— y paga por los tres al precio del más caro. Se separan. (§12.4)

5. La unidad semántica y el fragmento

Esta sección no existe en la v1.4. Es la que responde qué tamaño tiene un fragmento y dónde se corta, y de ella dependen el planificador (§10.2), el presupuesto de contexto (§6) y el ensamblador (§10.4).

5.1 El codón no es el homólogo

La propuesta original tomó el **codón** —tres nucleótidos que se traducen a un aminoácido— como homólogo de la unidad mínima de significado del texto, y de ahí derivó que existe un tamaño mínimo de fragmento por debajo del cual se destruye significado.

La conclusión es correcta. La derivación no. El codón tiene tres propiedades estructurales, y la oración no tiene ninguna:

propiedad del codón	la oración
Ancho fijo — exactamente 3 nucleótidos	Variable. Medido sobre el corpus del proyecto: $p_{25} = 9$ tokens, $p_{75} = 25$. Casi $3\times$ de rango.
Marco de lectura — no se solapa; un corrimiento arruina todo río abajo	No hay marco. Un corte desplazado no destruye el resto del texto.
Mapa determinista — GGA es siempre glicina	La misma idea admite infinitas superficies textuales.

Tres de tres. La transferencia no es parcial: es nula en las tres propiedades que definen al codón. Aplicando §3.4, falla además la segunda pregunta — nunca vino acompañada de un enunciado sobre qué ocurre al violarla.

5.2 El aminoácido es el homólogo de la unidad de alineamiento

La tercera fila señala dónde sí encaja. El aminoácido es una unidad de **ancho variable**, **auto-delimitada**, con **propiedades propias**, y es **la unidad de la estructura plegada**. La oración comparte las cuatro.

Y la distinción no es terminológica, porque la biología computacional tiene **dos tecnologías de alineamiento** y la elección del homólogo determina cuál se hereda:

- **Alineamiento de nucleótidos** — identidad o no. A frente a G es un desacuerdo y punto.

- **Alineamiento de proteínas** — matrices de sustitución. Henikoff & Henikoff (1992) [92] construyen BLOSUM contando pares de aminoácidos alineados en bloques sin huecos y calculando, para cada par, $s_{ij} = \log_2(q_{ij} / e_{ij})$, el logaritmo de la razón entre frecuencia observada y esperada, en unidades de medio bit.

Esa fórmula es el instrumento. Dice que la intercambiabilidad de dos unidades no se postula: **se cuenta**, sobre casos que ya se sabe que son homólogos.

Para Swarmbly la consecuencia es directa. Dos nodos que responden

«The tide window closes at noon.» «The channel shuts at midday.»

no están en desacuerdo. Bajo comparación tipo nucleótido son un desacuerdo total; bajo una matriz de sustitución son una **sustitución conservativa**. El ensamblador de la v1.4 —que compara por similitud de coseno contra un umbral— está operando en el régimen nucleótido sobre material que es de proteína.

5.3 El dominio es el homólogo del fragmento

Ésta es la corrección más importante de esta versión.

El aminoácido no es el homólogo del **fragmento**. Ningún aminoácido se pliega solo ni tiene función solo. La unidad que sí lo hace es el **dominio**.

Porter & Rose (2012) [110] dan la definición rigurosa: un dominio es «un segmento contiguo de la proteína plegada cuyo valor-m queda mayormente inalterado cuando ese segmento se extrae de su estructura madre», y equivalen dominio a «unidad cooperativa de plegado; es decir, su cooperatividad depende principalmente de interacciones intra-segmento, no inter-segmento».

Léase la segunda cláusula con cuidado, porque **es la especificación de un buen fragmento de tarea**: una pieza cuya resolución depende de lo que tiene dentro y no de lo que tienen sus vecinas.

El trabajo de referencia sobre proteínas multidominio (Han et al., 2007) [111] confirma que más del 70 % de las proteínas eucariotas son multidominio y que el plegado de los dominios es independiente — **pero con una condición explícita**: «donde la interfaz es pequeña y poco empaquetada, o no estructurada, el plegado de los dominios es independiente».

5.4 La condición de interfaz es la regla de corte

Esa condición no es una salvedad a pie de página. Es el instrumento, y es P9:

Fragméntese donde el acoplamiento entre fragmentos es débil. El límite de fragmento es una decisión sobre la interfaz, no sobre el tamaño.

Esto reordena el diseño. El planificador no debe cortar cada L oraciones y esperar que salga bien: debe **buscar los puntos de acoplamiento débil** y cortar ahí, con L como objetivo y no como regla. Un corte que parte una dependencia fuerte produce dos fragmentos que no son dominios, y la garantía de independencia no aplica a ninguno de los dos.

El proyecto ya tiene un incidente registrado que es exactamente este fallo: el segmentador separó una pregunta del material que la respondía. Eso no fue un error de implementación, fue un corte a través de una interfaz fuerte. Con la formulación de la v1.4 —cortar en N partes de tamaño parecido— ese fallo es estructural y recurrente. Con la regla de interfaz es detectable antes de despachar, porque el planificador puede medir el acoplamiento que está a punto de cortar.

Y hay un dato del propio proyecto que apunta en la misma dirección. En la medición de §15.3, los dos únicos prompts donde $N = 2$ es *peor* que $N = 8$ son los dos que fabrican la media. Que más fragmentos *ayuden* no es lo que predice una historia de impuesto de coherencia; sí es lo que predice un fallo de **calidad de partición** — un corte en dos que pone una costura en un sitio caro, que cortar en ocho evita por casualidad. Bajo P9 eso deja de ser casualidad.

5.5 La autonomía funcional está cuantificada, y no es total

Bashton & Chothia (2007) [112] comparan dominios homólogos que aparecen tanto en proteínas de un dominio como en multidominio, sobre 70 pares únicos de dominios en 45 conjuntos de proteínas: aproximadamente **tres cuartas partes conservan su función** al cambiar de contexto; **poco menos de un sexto cambia sustancialmente**.

El número acota la expectativa. «Un fragmento resuelto aisladamente vale igual que en contexto» es cierto la mayoría de las veces y falso una de cada seis o siete. Un diseño que asuma autonomía total está asumiendo algo que la naturaleza no cumple ni en el sistema del que se tomó prestada la idea.

5.6 El tamaño: hay una banda, no hay un número

¿Existe un tamaño natural de dominio que sugiera un L natural para el texto? La respuesta honesta es que **existe una banda característica y no existe un número**, y la razón por la que no existe es instructiva.

Zhang et al. (2005) [113] mapean dominios definidos por secuencia contra dominios definidos por estructura, sobre las mismas proteínas: **Pfam promedia 96 residuos; SCOP promedia 174**. Casi el doble, sobre el mismo material, en el mismo trabajo. La diferencia no es ruido: Pfam corta por familia de secuencia y SCOP por estructura. **La definición determina el número**.

Schaeffer et al. (2023) [114], clasificando dominios sobre estructuras predichas, reportan **99.8 ± 64.8 residuos** — una desviación estándar que es el 65 % de la media.

El piso, en cambio, sí es nítido. Porter & Rose fijan **25 residuos** como mínimo, «aproximando el tamaño de una unidad de estructura supersecundaria»; UniDoc (Zhu et al., 2023) [115] usa 30 como restricción de parseo.

Las dos lecciones transfieren limpiamente:

1. **El piso es duro y principiado.** Por debajo de cierto tamaño nada pliega cooperativamente. Para el texto, esto predice que existe un $L_{\min}$ por debajo del cual un fragmento no es independientemente resoluble, y que ese piso es más nítido que el óptimo. La sección 6.5 da una razón independiente para el mismo piso, por el lado del costo de cabecera.
2. **El óptimo es una banda ancha y dependiente de la definición.** Cualquier afirmación de que $L^* = 50$ es un número es una sobre-lectura. Lo que se puede esperar es una banda de factor 3 a 6, y la definición de «unidad» que se elija moverá su centro.

5.7 Los fundamentos de la unidad semántica

1. La unidad mínima de alineamiento es la **oración**, homóloga del aminoácido: ancho variable, auto-delimitada, con significado propio, y la unidad sobre la que se calcula sustitución.
2. La unidad de fragmentación es el **dominio de tarea**, homólogo del dominio proteico: el segmento cuya resolución depende de interacciones internas y no de sus vecinos.

3. El límite de fragmento se elige **donde el acoplamiento es débil**, con L como objetivo en oraciones y no como regla de división.
4. L es una propiedad de la **clase de nodo**, medida y revisable. N se deriva: $N = \lceil \text{material} / L \rceil$. Esto invierte la formulación de la v1.4, donde N era el parámetro y el tamaño la consecuencia.
5. Existe un $L_{\min}$ duro; el óptimo es una banda.

6. El presupuesto de contexto

Ésta es la restricción central del proyecto. La v1.4 la enunciaba y la barría; esta versión la deriva y la formaliza.

6.1 Definición

Sea una petición P descompuesta en micro-tareas $T = \{t_1 \dots t_N\}$ con DAG de dependencias $D = (V, E)$. Cada paquete despachado es

$$K_i = (\Gamma, \sigma(R_j : (t_j \rightarrow t_i) \in E), t_i)$$

donde Γ es el *contrato global* —objetivo, audiencia, registro, formato de salida, longitud objetivo, vocabulario prohibido, identificador de sesión— y $\sigma(\cdot)$ resume los resultados de las predecesoras de t_i .

Se define el **presupuesto de contexto**

$$S = |\Gamma| + E[|\sigma(\cdot)|] \quad (\text{tokens de contexto compartido por paquete})$$

y la **razón de redundancia contextual**

$$\rho = (\sum_i |K_i|) / |P|$$

ρ es lo que el operador paga. En la v1.4, S era lo que el operador elegía. En esta versión no lo es: §6.5 deriva ρ de L , F y el costo de cabecera, y lo que el operador elige son esos tres.

6.2 La tensión a cuatro bandas

Cuatro deseos son funciones de S , y no coinciden:

Deseo	Comportamiento en S	Mecanismo
Coherencia de ensamblaje	crece	Los trabajadores comparten las decisiones que hacen compatibles los fragmentos. Sin contrato, uno renderiza la escena en un registro y otro en otro, y el cliente hereda partes incompatibles [33]
Verificabilidad del fragmento	crece	Un verificador no puede juzgar si un fragmento es fiel a una especificación que no recibió
Privacidad por descontextualización	decrece	Γ es el objetivo, la audiencia y las restricciones de la sesión. Un nodo que tiene Γ tiene la forma de la petición
Capacidad requerida del trabajador	plausiblemente decrece	El contexto suministrado en el prompt sustituye al conocimiento en parámetros — enunciado como hipótesis en §7.3, no como resultado

Y ρ , y por tanto el costo, crece aproximadamente de forma lineal en $N \cdot S$.

6.3 El núcleo falsable

Proposición (Presupuesto de contexto). Swarmbly es viable si y sólo si existe un presupuesto S^* que satisfaga simultáneamente: un impuesto de coherencia por debajo de la tolerancia de la aplicación; una cota de fuga por debajo de la tolerancia del usuario para el carril de sensibilidad aplicable; una exactitud de verificación por encima del requisito de seguridad del protocolo; y un requisito de capacidad de trabajador que cumplan modelos pequeños de mercado — todo ello a un ρ cuyo costo quede por debajo del valor de la capacidad agregada.

Esto es el proyecto entero enunciado como una afirmación comprobable, y es por lo que la implementación de referencia mide una curva en vez de demostrar un sistema.

También predice algo útil. Como S es compartido por los cuatro, **toda mejora que eleve la coherencia por token de contexto vale más que una que eleve la coherencia por token de salida** — compra progreso en privacidad y en costo a la vez. Esto hace de la compresión del contrato, y no de la calidad del fragmento, la dirección de investigación de mayor palanca en el diseño.

6.4 Flancos, repeticiones y el piso informacional

La v1.4 trataba el solapamiento como un porcentaje heredado por analogía. No lo es, y la literatura informacional dice exactamente qué es.

Aquí el solapamiento resuelve un problema distinto. En ensamblaje *de novo* el solapamiento existe sobre todo para **descubrir el orden**: nadie sabe de qué parte del genoma vino cada lectura. Swarmbly no tiene ese problema — el orquestador crea los fragmentos y conoce su orden. Quedan dos razones, y sólo una es genómica: **continuidad de transición** (que el final de un fragmento encaje con el principio del siguiente) y **detección de repeticiones**.

El criterio de solapamiento mínimo tiene dos formas. El enunciado folclórico —«el solapamiento debe exceder la repetición más larga»— es correcto pero débil. Bresler, Bresler & Tse (2013) [102] lo precisan en dos condiciones distintas:

Condición suficiente para un algoritmo voraz:

$$L > \ell_{\text{repeat}} + 1$$

«GREEDY reconstruye la secuencia original si toda repetición está puenteada.» Es un enunciado sobre **un algoritmo particular**.

Condición necesaria, informacional:

$$L > \max\{\ell_{\text{interleaved}}, \ell_{\text{triple}}\} + 1$$

donde una repetición *entrelazada* es un par de repeticiones cuyas posiciones se intercalan, y una *triple* es una subsecuencia que aparece tres veces. Por debajo de ese umbral **ningún algoritmo** puede reconstruir, porque dos secuencias distintas producen conjuntos de lecturas idénticos.

La distinción importa más de lo que parece. La primera forma dice «hazlo mejor». La segunda dice **que existe una clase de entradas donde los fragmentos simplemente no contienen la información necesaria para reensamblar correctamente, por listo que sea el ensamblador**.

Motahari, Bresler & Tse (2013) [103] formalizan el umbral: para secuencias i.i.d., hay una transición neta según si la longitud de lectura normalizada supera la entropía de Rényi de orden 2 de la fuente. Por encima, «la condición obvia de cobertura es también suficiente para la reconstrucción»; por debajo, no hay cobertura que alcance.

El paralelo que esto habilita, y que esta versión adopta: la cobertura —cuántas réplicas— y la resolubilidad —si los fragmentos contienen la información— son **dos preguntas distintas**, y la primera no implica la segunda. La v1.4 razonaba sobre k y guardaba silencio sobre resolubilidad. El modelo de cobertura de §7.4 sigue siendo correcto y sigue siendo sólo la mitad del asunto.

La repetición es una propiedad de un par, y el proyecto ya la midió. El homólogo semántico de una repetición genómica es una frase o plantilla recurrente. Dos fragmentos que no se ven repiten la misma estructura, y el ensamblador no puede detectarlo porque **la repetición es una propiedad de un par, no de un fragmento**. La definición formal de Bresler et al. es inherentemente relacional: una repetición «es una subsecuencia que aparece dos veces» en posiciones t_1 y t_2 . Ninguna lectura individual puede inspeccionarse para saber si está en una repetición.

Y el proyecto **ya midió el mismo fenómeno**. La restricción `no_repeated_ngram` es la clase irreducible de sus mediciones de composición: **4 de 12 frente a 11 de 12 del brazo monolítico**, y ninguna política de asignación de contexto la alcanza. Era el defecto que no cedía. La razón, ahora nombrable, es que ninguna asignación *puede* alcanzarla: un fragmento no puede evitar repetir lo que no puede ver. La sección 11 da el mecanismo que sí puede.

6.5 El flanco F y la derivación de ρ

De lo anterior sale el criterio operativo:

F es el máximo entre dos cantidades medidas sobre el corpus: la unidad repetible más larga que el ensamblador deba poder detectar, y la dependencia más larga que cruza un límite de fragmento.

Estimación de referencia para el corpus del proyecto: **$F \approx 5\text{--}10$ oraciones.**

Con F definido por medición y L como propiedad de la clase de nodo, el presupuesto de contexto **se deriva** en lugar de barrerse:

$$\rho \approx (L + 2F) / L + H / (L \cdot s)$$

donde H es el costo fijo por paquete —contrato, glosario, instrucciones de formato— medido en **≈ 39 tokens** sobre el corpus del proyecto, y $s \approx 15$ tokens por oración.

Con $L = 50$, $F = 10$: **$\rho \approx 1.45$** . Con un flanco heredado por analogía, $F = 25$: **$\rho \approx 2.05$** .

Un 29 % del cómputo de toda la red, por la única decisión de medir el flanco en vez de copiar un porcentaje. Y como §6.4 dice exactamente qué medir, el ahorro no relaja ninguna garantía: F se fija por la repetición y la dependencia observadas, no por comodidad.

El segundo término explica algo que el proyecto observó sin poder nombrarlo. La cabecera se paga **entera por paquete**, así que su peso relativo es inversamente proporcional a L . Con fragmentos de 35 tokens —el tamaño real de los fragmentos en las mediciones de composición del proyecto— la cabecera de 39 tokens **pesa más que el material**. Ése es el régimen en el que se midió durante meses, y explica por qué ρ parecía tener un piso alto: no era una propiedad del protocolo, era una propiedad de fragmentar demasiado fino.

Esto tiene una consecuencia directa sobre la métrica de la v1.4. Aquel documento fijaba « ρ operativo < 2.0 » como objetivo y luego reportaba que no era alcanzable en su corpus, porque el piso de empaquetado a $N = 2$ era 1.42–1.68. Con la derivación de arriba, ese piso deja de ser un hecho bruto y se vuelve una predicción: era el término $H/(L \cdot s)$ dominando porque L era demasiado pequeño. El objetivo correcto no es un número absoluto de ρ sino **la distancia a $(L+2F)/L$** , que es lo que se puede reducir sin perder garantías.

6.6 Por qué formalizar ρ

ρ ha funcionado como cantidad empírica: se mide, se compara, se reporta. La derivación anterior ya es un avance sobre tratarla como dial. Pero derivarla no dice **si existe un piso**, ni de qué depende. La teoría de tasa-distorsión es el marco donde esa pregunta tiene respuesta.

Concepto base. La teoría de tasa-distorsión (Shannon, 1959) [121] responde: dado que voy a transmitir una versión imperfecta de algo, ¿cuál es la tasa mínima de información necesaria para que la imperfección no exceda un nivel dado?

Shannon define una matriz de distorsión donde « d_{ij} mide el "costo" o "distorsión" si la letra i se reproduce en el receptor como la letra j », y $R(d^*)$ como la mínima tasa sujeta a que la distorsión promedio no exceda d^* . Cover & Thomas [122] lo escriben como

$$R(D) = \min I(X; \hat{X})$$

minimizado sobre distribuciones condicionales $p(\hat{x}|x)$ que cumplen la restricción de distorsión esperada. Su Teorema 13.2.1 establece que la función de tasa-distorsión de una fuente i.i.d. con distorsión acotada **es igual** a la función informacional asociada.

La forma de la curva es lo esencial: $R(D)$ decrece con D . Menos fidelidad exigida, menos información necesaria. Y hay un piso: por debajo de $R(D)$ la distorsión D es inalcanzable, haga uno lo que haga.

6.7 ρ como tasa de un problema indirecto

La objeción natural es que la tasa-distorsión clásica supone una fuente que se reconstruye, y Swarmbly no reconstruye la petición: produce una respuesta.

La teoría tiene un nombre para esa situación: el problema de tasa-distorsión **indirecto** o **remoto**, donde «el codificador no puede observar la fuente directamente sino que obtiene sólo observaciones ruidosas» — formulación que se remonta a Wolf & Ziv.

El mapeo es exacto, y es la aportación formal de esta sección:

elemento de la teoría	en Swarmbly
Fuente Y^*	La respuesta ideal a la petición. Nunca se observa , ni por el orquestador ni por ningún nodo.
Observación X	La petición P . Es lo que el codificador <i>sí</i> ve: una descripción de Y^* , no Y^* .
Codificador	La política de fragmentación: router, planificador, empaquetador. Produce $\{K_1 \dots K_N\}$.
Canal	La red de nodos. Cada nodo aplica una transformación estocástica $K_i \rightarrow R_i$.
Decodificador	El ensamblador local, que produce $\hat{Y}$ a partir de $\{R_i\}$.
Tasa	$\rho = \Sigma$
Distorsión	$d(Y^*, \hat{Y})$ — pérdida de tarea, no fidelidad de reconstrucción. Es el movimiento de Nagle et al., y es el correcto.

Con esto, la afirmación defendible queda enunciada:

M4 — ρ es la tasa de un problema de tasa-distorsión indirecto cuya distorsión es pérdida de tarea. De ahí se hereda la existencia de una cota inferior y la monotonía cualitativa de la curva. No se hereda una cota computable.

Lo que no se puede afirmar, y por qué. Tres límites, enunciados antes de que un revisor los enuncie.

La cota existe pero no es computable aquí. Nagle et al. obtienen un número porque restringen a borrado de tokens sobre un prompt conocido con un LLM de caja negra, y resuelven un programa lineal finito. El codificador de Swarmby es una política de descomposición y su decodificador es un modelo de lenguaje; no hay $p(x)$ tratable sobre el espacio de tareas ni una distorsión de letra única. Se puede afirmar que la cota **existe**; no que se ha calculado.

El teorema es asintótico en bloques i.i.d. Una petición de usuario es una realización, no una secuencia larga. El recíproco no da garantía por petición.

Y el más importante: la distorsión no está causada sólo por la tasa.

6.8 El segundo eje: densidad de dependencias

Éste es el aporte propio de la sección, y sale de un dato del arte previo.

Skeleton-of-Thought [12] genera un esqueleto y expande los puntos en paralelo. Logra aceleraciones de hasta **2.39×**. Y su degradación de calidad **no es uniforme**: mejora «diversidad y relevancia mientras perjudica la inmersión y la coherencia», y **falla en matemáticas y programación**.

Ésas son precisamente las tareas donde las sub-respuestas son interdependientes. La degradación no sigue a la tasa: sigue a **cuánto se cortó a través de dependencias**.

Una curva $D(\rho)$ indexada sólo por ρ no puede capturar eso. Se necesita al menos un segundo eje:

$$D = D(\rho, \delta)$$

donde **δ es la densidad de dependencias de la descomposición**: alguna medida de cuántas relaciones necesarias cruzan un límite de fragmento, normalizada por el número de fragmentos.

Esto cierra el argumento del documento por tres lados a la vez. **La regla de corte por interfaz débil (P9, §5.4) es, en este lenguaje, la minimización de δ** : cortar donde el acoplamiento es débil es exactamente elegir la descomposición de menor densidad de dependencias para un ρ dado. Y el piso informacional de Bresler et al. (§6.4) es el enunciado, en el lenguaje del ensamblaje, de que **existe una región del plano (ρ, δ) donde ninguna distorsión aceptable es alcanzable**, porque los fragmentos no contienen la información.

Las tres formulaciones —interfaz débil, piso informacional, densidad de dependencias— son la misma afirmación en tres vocabularios. Que converjan desde tres literaturas independientes es la razón para creerla.

Y es falsable sin teoría adicional. Fijar δ variando sólo ρ (mismo corte, distinto flanco) debería producir una curva monótona decreciente en distorsión. Fijar ρ variando δ (mismo presupuesto, cortes de distinto acoplamiento) debería producir variación de distorsión **a tasa constante** — que es el resultado que mataría la idea de que ρ basta como eje. Y la región de inalcanzabilidad debería manifestarse como un suelo de distorsión que ningún ρ mejora.

6.9 Por qué la formulación anterior era inadecuada

Una versión anterior de este diseño especificaba un objetivo fijo de redundancia ($C_{\text{sem}} > 1.2$, «20 % de redundancia intencional») derivado por analogía de Lander-Waterman, y un umbral de costura fijo ($\tau_{\text{sem}} = 0.85$) sobre similitud de coseno de *embeddings*.

Ambos están retirados. El primero por las razones de §2.6 y porque el paso de una razón a un porcentaje de redundancia sólo se sostiene si todo el exceso de longitud es flanco, lo que deja de ser cierto en cuanto se introduce un contrato global. El segundo porque el espacio de *embeddings* contextuales es anisótropo —palabras elegidas al azar ya exhiben alta similitud media de coseno [34]—, porque la similitud de coseno en modelos regularizados puede ser «arbitraria y por tanto sin significado» [35], porque ningún modelo de *embedding* domina en todos los tipos de tarea [36], y porque la biblioteca de referencia para esta operación recomienda deliberadamente **no usar umbral** y advierte que la similitud es asimétrica [37].

Esta versión añade una tercera razón, y es la más de fondo: **el umbral de coseno es un instrumento de tipo nucleótido aplicado a material de tipo proteína** (§5.2). No es que el umbral esté mal calibrado; es que la operación correcta es una matriz de sustitución aprendida, y §10.5 la especifica.

7. Inteligencia de enjambre con modelos pequeños

7.1 La tesis

Swarmbly prescinde del modelo monolítico por completo. Ningún participante tiene un fragmento de una red de 70B o 400B. Los nodos trabajadores corren modelos de lenguaje pequeños *completos e independientes* —típicamente 1-8B parámetros, cuantizados, sobre GPU de consumo, portátiles de memoria unificada o CPU multinúcleo— y cada uno recibe una micro-tarea descontextualizada y la responde aislado. El cliente también corre un modelo pequeño, pero su competencia es otra: no conocimiento del mundo, sino *lógica y sintaxis*, en el sentido de entender la petición, planificar su descomposición y suturar los fragmentos devueltos.

La afirmación es que la capacidad avanzada no necesita residir en una sola red grande, sino que puede ser el resultado aritmético de coordinar muchas pequeñas — la respuesta emergiendo, como un genoma, sólo en el ensamblaje.

Es una afirmación fuerte. También está en parte respaldada, en parte sin respaldo y en parte es falsa tal como suele enunciarse. Esta sección separa las tres.

7.2 Cobertura y conversión

Propongo descomponer el rendimiento del enjambre en dos factores independientes.

Cobertura c es la probabilidad de que *al menos una* respuesta de trabajador a una micro-tarea dada sea aceptable, y **conversión** v es la probabilidad de que el orquestador, dado que existen fragmentos aceptables, los seleccione y ensamble en un todo aceptable.

Para un plan de N micro-tareas con cobertura por tarea c_i y factor global de conversión v :

$$Q_{\text{sistema}} \approx v \cdot \prod_i c_i$$

El producto sobre i es el término incómodo —es por lo que N no puede crecer libremente— pero el factor que decide la arquitectura es v .

La cobertura escala con el enjambre. La conversión no.

La evidencia de la primera mitad es fuerte. El muestreo repetido eleva la cobertura log-linealmente sobre cuatro órdenes de magnitud del número de muestras: en SWE-bench Lite con DeepSeek-Coder-V2-Instruct, 15.9 % con una muestra sube a 56 % con 250, superando un estado del arte de 43 % con una sola muestra [38].

La evidencia de la segunda mitad es igual de fuerte y suele pasarse por alto. El mismo trabajo afirma que «majority voting and reward models plateau beyond several hundred samples» — la cobertura sigue subiendo y la *capacidad de cobrarla* se satura [38]. La selección por juez sobre equipos diversos alcanza 81 % de tasa de victoria contra una línea base de un solo modelo, mientras que los equipos homogéneos alcanzan 51.2 % —azar— y producen 100 % de empates sobre 756 veredictos [39]. Y en el estudio más cercano a la arquitectura de Swarmbly, un sistema multi-agente de 8B empatiza con un agente único de 32B con herramientas en GAIA (23.0 frente a 23.0) y lo supera en AIME (55.0 frente a 45.0), corriendo 4.2× más rápido — pero el rendimiento está «primarily driven by orchestrator capacity rather than sub-agent capacity» [40].

7.3 Tres consecuencias, y una hipótesis sin respaldo

(a) El cliente es el techo, no la red. El valor marginal del nodo ($N+1$) está acotado superiormente por la capacidad del orquestador de seleccionar entre lo que ya llega. Un presupuesto de ingeniería que compra nodos antes que un mejor selector en el cliente está gastando en el orden equivocado.

(b) Seleccionar, no sintetizar. La selección por juez supera a la agregación de tipo síntesis por 63.1 puntos porcentuales, y la síntesis estilo Mixture-of-Agents pierde contra la línea base de un solo modelo en 42 de 42 tareas [39]. Esto está en tensión directa con los resultados que la propia MoA reporta —65.1 % en AlpacaEval 2.0 frente a 57.5 % de GPT-4 Omni usando sólo modelos abiertos [41]— y señalo el desacuerdo en lugar de elegir el lado conveniente. El diseño lo resuelve conservadoramente: la selección es el camino por defecto, la síntesis la excepción invocada sólo en una costura fallida.

(c) La heterogeneidad es un activo, no un defecto. Los equipos diversos llegan a 81 % de victoria donde los homogéneos llegan al azar, y las salidas homogéneas empatan el 100 % de las veces — un selector al que se le dan candidatos idénticos no tiene nada que seleccionar [39]. El protocolo debe preservar la diversidad deliberadamente: despachar fragmentos críticos a trabajadores de familias *distintas*, no meramente a máquinas distintas.

La hipótesis de atomicidad sigue sin respaldo. La forma fuerte de la afirmación —que un modelo de 3B respondiendo una subtarea atómica iguala a un modelo frontera— no está establecida. Lo que está respaldado es más estrecho, y hay un mecanismo específico por el que la afirmación puede fallar: **un trabajador más pequeño necesita más contexto para hacer el mismo trabajo**. El conocimiento que el modelo no tiene en parámetros hay que suministrarlo en el prompt. Encoger al trabajador no es gratis; se paga en S , y S se paga en privacidad y en costo.

H2 (Sustitución de capacidad). Para micro-tareas atómicas, bien especificadas y verificables, existe un presupuesto S al que la calidad de fragmento de un trabajador de 3–8B es estadísticamente indistinguible de la de un modelo frontera en la misma micro-tarea; y el S requerido decrece al aumentar la capacidad del trabajador.

H3 (Conversión). Un selector en el cliente sobre $k > 1$ trabajadores heterogéneos recupera una fracción especificada de la calidad de selección oráculo. Una fracción de

recuperación que no mejore con el tamaño del orquestador falsaría la premisa de la consecuencia (a).

Un orquestador de 8B no es obviamente adecuado para ese papel, y la literatura desanima: en una tarea de planificación con estado, Llama-3.1-8B-Instruct puntuó cerca de 0-2 %, y hasta los modelos frontera entraron en bucle en 92-100 % de los intentos bajo un validador externo [44]. Los modelos pequeños son buenos *routers* —routers baratos recortan costos en más de 85 % en MT-Bench reteniendo 95 % del rendimiento de GPT-4 [46]— pero enrutar es clasificación y planificar no lo es.

7.4 Un modelo de cobertura para ensamblaje semántico

Con el plan como secuencia de referencia, la maquinaria de Lander-Waterman aplica — y la fuente de aleatoriedad resulta estar exactamente donde los supuestos del modelo se cumplen.

Montaje. El plan $\mathcal{D}$ define un conjunto ordenado de unidades semánticas $U = \{u_1 \dots u_M\}$, fijado antes de toda generación. Cada paquete K_i apunta a un subconjunto $S_i \subseteq U$. La cobertura nominal es $c = (\sum_i |S_i|) / M$.

Dónde vive la aleatoriedad. Ésta es la diferencia sustantiva respecto de la genómica, y es lo que hace legítima la transferencia:

En secuenciación, el elemento estocástico es **dónde caen las lecturas**. En Swarmbly la colocación es determinista —la elige el orquestador—. El elemento estocástico es **qué paquetes vuelven**.

Los nodos voluntarios fallan, expiran, se desconectan y devuelven salida inservible, de forma independiente y a una tasa que la red puede medir. Sea p esa probabilidad de pérdida por paquete. La cobertura efectiva es $c_{\text{eff}} = c \cdot (1-p)$, y los resultados clásicos se mantienen con c_{eff} en lugar de c :

```
P(la unidad u queda sin cubrir) = e^( -c_eff )
E[ unidades sin cubrir ]         = M · e^( -c_eff )
E[ islas de ensamblaje ]         = N_p · e^( -c_eff · θ )
```

La ecuación de diseño. Invirtiendo el primer resultado se obtiene un requisito de redundancia derivado de una tolerancia declarada:

$$c \geq \ln(1/\epsilon) / (1 - p)$$

Tasa de pérdida p	$\epsilon = 5 \%$	$\epsilon = 1 \%$	$\epsilon = 0.1 \%$
0.05	$c \geq 3.2$	$c \geq 4.8$	$c \geq 7.3$
0.10	$c \geq 3.3$	$c \geq 5.1$	$c \geq 7.7$
0.20	$c \geq 3.7$	$c \geq 5.8$	$c \geq 8.6$

Y los números de campo con que alimentarla. Anderson & Fedak (2006) [131], sobre más de 330.000 hosts de SETI@home: fracción media de encendido **0.81**, fracción de conexión **0.83**, fracción activa media **0.84**, **vida media del host 91 días**. La versión de 2018 [132] reporta ~700.000 dispositivos y disponibilidad de **~60 %** para equipos de escritorio y **~40 %** para móviles. Una tasa de pérdida del 16-40 % es exactamente el régimen para el que la redundancia por umbral está diseñada (§12.3).

Alcance, y una afirmación. El modelo acota la *disponibilidad*: la probabilidad de que una unidad semántica quede sin responder. No modela la corrección semántica — una unidad puede estar cubierta por cinco réplicas que coincidan y estén todas mal. Dentro de ese

alcance, creo que es **el primer modelo de cobertura publicado para ensamblaje semántico**, y es el punto en que el marco genómico deja de ser vocabulario y se vuelve derivación.

Lo que §6.4 le añade, y que la v1.4 no tenía. Este modelo responde «¿llegará respuesta?». No responde «¿contiene la respuesta la información necesaria?». Ésa es la condición informacional de Bresler et al., y es independiente: **cobertura alta con fragmentos irresolubles produce un ensamblaje confiadamente equivocado.**

7.5 El enunciado honesto de la tesis del enjambre

El enjambre suministra **cobertura**; el cliente suministra **conversión**. Los nodos adicionales elevan la cobertura con rendimientos logarítmicos y no hacen nada por la conversión. La heterogeneidad entre nodos es lo que hace posible la selección, y debe preservarse en lugar de eliminarse por ingeniería. El techo de calidad del sistema lo fija el selector del cliente, y el tamaño de trabajador que basta es función del presupuesto de contexto y no una constante. Y la cobertura sólo responde por la disponibilidad: la resolubilidad de los fragmentos es una condición aparte que ninguna cantidad de réplicas repara.

Y ahora hay una medición. Sobre 80 celdas operativas de resumen de tablas (16 tareas × 5 familias, presupuesto de salida igualado), el impuesto medio por **clase de nodo** va de **+8.4 %** a **+26.5 %**, con desviación típica intra-clase de **16** a **39** puntos (T0RR, §15.9). Con el presupuesto igualado todas las clases salen caras —la división anterior entre caras y baratas era el confundido de longitud (T09R)—, y la varianza *dentro* de una clase sigue superando la distancia *entre* clases. Eso último es exactamente por qué la predicción por celda falla (§10.1) y por qué el enjambre responde por cobertura y no por el resultado de una celda concreta.

8. Arquitectura

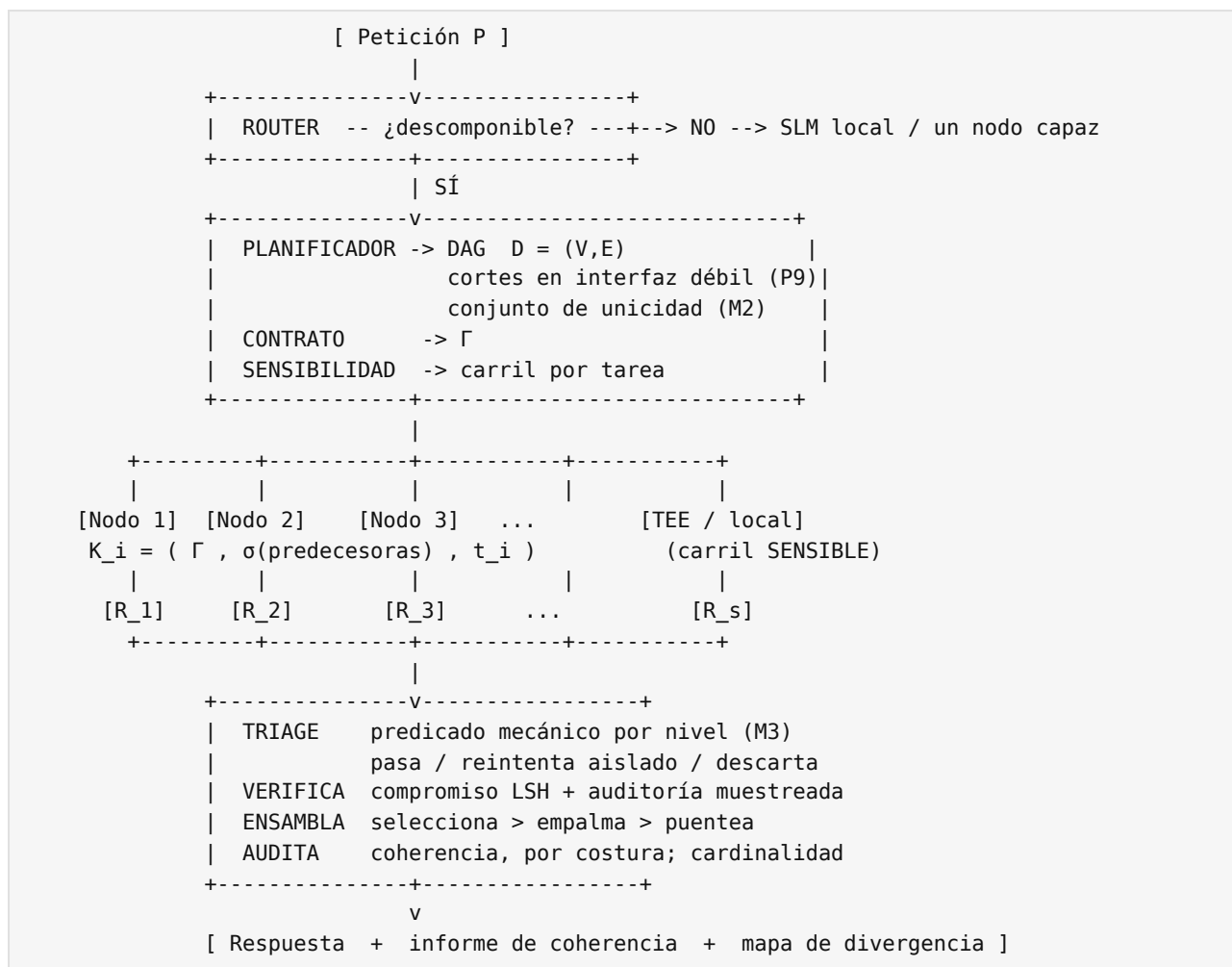
8.1 Papeles

Cliente / Orquestador — router, planificador, generador de contrato, clasificador de sensibilidad, empaquetador, despachador especulativo, verificador, ensamblador, auditor de coherencia. Requiere un SLM ($\geq 8B$ recomendado; su adecuación es H3) más un modelo de *embeddings*.

Nodo trabajador — declara un perfil, ejecuta micro-tareas, emite compromisos de verificación y telemetría. Corre un modelo pequeño completo.

Servicios de red — descubrimiento de pares (DHT), registro de reputación, contabilidad de créditos, muestreador de auditoría. Deliberadamente mínimos; sin blockchain en v0.3.

8.2 Ciclo de vida de una petición



La ejecución avanza por nivel topológico. La v1.4 decía «un nivel comienza cuando sus predecesoras han regresado y han sido verificadas». Esta versión especifica qué significa esa verificación (§10.7) y añade que **el consumo puede empezar antes de que el nivel esté completo**, si el fragmento consumido está completo y ha pasado el triage (§10.7.4).

8.3 El triage, y por qué es una especificación y no una metáfora

La biología tiene nombre para ese control, y es preciso. Gottesman, Wickner & Maurizi (1997) [116] lo llaman literalmente así:

«proponemos [...] un modelo general para lo que puede pensarse como un sistema de **triage** para manejar proteínas mal plegadas in vivo, asegurando el **replegado rápido de proteínas con potencial funcional y la degradación rápida de proteínas irreversiblemente desnaturalizadas o dañadas**.»

Esa frase es casi una especificación de la etapa de control de calidad: clasificar cada fragmento devuelto como **recuperable** → **reintentar** o **irrecuperable** → **descartar y regenerar**, y **nunca empalmar uno malo en el ensamblado**.

El mecanismo es igual de instructivo. La chaperonina GroEL/GroES no repara la pieza en su sitio: la **aísla**. Xu, Horwich & Sigler (1997) [117] describen cómo la unión de GroES «estabiliza una cámara de plegado» cuya elevación y torsión de los dominios apicales «duplica el volumen de la cavidad central y entierra los residuos hidrofóbicos de unión a péptido»,

dejando un revestimiento hidrofílico «propicio para el plegado». **Aislar y reintentar**, no parchear en contexto.

Y hay evidencia de que los niveles importan, más allá de justificar el DAG. Netzer & Hartl (1997) [118] encontraron que polipéptidos de dos dominios «pliegan eficientemente mediante plegado secuencial y cotraduccional» en traducción eucariota, mientras que las mismas proteínas plegadas post-traduccionalmente en *E. coli* sufren «mal plegamiento intramolecular de dominios que pliegan concurrentemente». Plegar los dominios de a uno, en orden, tuvo éxito donde plegarlos concurrentemente produjo interferencia. El argumento no es que el paralelismo sea malo; es que el paralelismo **irrestricto** lo es. Marsh et al. (2013) [119] añaden que el orden de ensamblaje de complejos proteicos «puede predecirse simplemente a partir de sus estructuras tridimensionales» y que hay **selección evolutiva para conservar el orden**.

Una complicación honesta. Shiber et al. (2018) [120], mediante perfilado ribosomal, encontraron que **nueve de doce** complejos hetero-oligoméricos estudiados se ensamblan cotraduccionalmente, y «el ensamblaje cotraduccional ocurre a menudo unidireccionalmente, con una subunidad completamente sintetizada acoplándose a su subunidad compañera naciente». Esto complica el cuadro «pliega todo, después ensambla» en una dirección aprovechable: un fragmento de nivel n puede consumirse mientras el nivel $n+1$ todavía se genera, siempre que el que se consume esté completo. Es un argumento a favor de ensamblado en *streaming* con una condición de completitud, no de espera de barrera.

8.4 Modelo de latencia

La afirmación $\max \ll \Sigma$ es direccionalmente correcta e incompleta. El modelo honesto es

```
T_total = T_plan + Σ_niveles E[ max_{i ∈ nivel} t_i ] + T_triage + T_verifica +  
T_ensambla
```

Dos de esos términos son locales y por tanto predecibles. La planificación escala con $|P|$ y corre en el modelo pequeño del cliente; el ensamblaje escala con $\Sigma|R_i|$. Los otros los fija el enjambre.

El primero es la cola de rezagados. $E[\max]$ sobre W extracciones concurrentes crece con W , y las colas voluntarias son pesadas: el ciclo de trabajo efectivo medido en BOINC es ≈ 0.61 y la vida media del host 91 días [47, 48]. Con probabilidad de fallo por nodo p , $P(\text{al menos un fallo}) = 1 - (1-p)^W$; a $p = 0.10$ y $W = 20$ eso es 88 %. El protocolo obliga a **peticiones especulativas** al p95 de la distribución de latencia observada por clase (§9.6), y esta versión añade el sobre-despacho con terminación temprana (§12.3), que ataca el mismo término por otra vía.

El triage añade un término que la v1.4 no tenía. Es deliberado y es barato: el predicado de §10.7 es mecánico, sin juez y sin modelo. Lo que compra es que el radio de daño de un fragmento defectuoso quede contenido en ese fragmento.

8.5 Perfiles de trabajador y determinismo

Un trabajador que corre un modelo no declarado, o una cuantización distinta, cambia silenciosamente la calidad y el registro del fragmento. El perfil es por tanto parte del protocolo y se liga al compromiso de verificación:

```
perfil = (familia_modelo, versión_modelo, cuantización,  
id_plantilla_prompt, params_muestreo, política_semilla)
```

El orquestador agrupa cada nivel del DAG en **clases de capacidad homogéneas** para controlar el desajuste de registro, mientras preserva deliberadamente **diversidad de familias entre réplicas redundantes** de la misma tarea (§7.3c). Los dos objetivos están en tensión y el protocolo hace explícito el intercambio: homogeneidad *dentro* del papel de un fragmento, diversidad *entre* candidatos para el mismo fragmento.

Y esta versión añade L al perfil. Si L es una propiedad de la clase de nodo (§5.7), el nodo debe declararla y el planificador debe poder leerla, porque N se deriva de ella. Un nodo que declara un L que no sostiene produce fragmentos que no son dominios, y eso es indistinguible desde fuera de un nodo que responde mal.

9. Especificación del protocolo (v0.3)

Esta sección está escrita para ser implementable. Los nombres de campo son normativos; las codificaciones se dan en JSON por claridad y PUEDEN ser CBOR en el cable. Los cambios respecto de v0.2 están marcados **[v0.3]**.

9.1 Identificadores

- `session_id` — 128 bits aleatorios, generado por petición, nunca reutilizado.
- `task_id` — `BLAKE2b-128(session_id || índice_nivel || índice_tarea)`, truncado a 16 bytes, en hexadecimal.
- `attempt_id` — `task_id || ':' || contador_intento`.

Un trabajador conoce `task_id` y `attempt_id`. NO DEBE conocer `session_id`; la derivación es de un solo sentido, de modo que dos trabajadores no puedan determinar que tienen fragmentos de la misma sesión comparando identificadores. Esto no derrota la correlación temporal (§13.2) y no se afirma que lo haga.

9.2 Contrato global Γ

```
{
  "v": "0.3",
  "objective": "string – qué debe lograr la respuesta completa",
  "audience": "string",
  "register": "formal|neutral|informal|technical",
  "format": "prose|markdown|json|code",
  "target_len": 0,
  "lexicon": { "prefer": [...], "forbid": [...] },
  "entities": [ { "name": "...", "canonical": "...", "role": "..." } ],
  "style_seed": "string – ancla de estilo determinista compartida",
  "budget": { "max_out_tokens": 0 }
}
```

`entities` es el mecanismo que evita nombres inconsistentes entre fragmentos — el análogo de ensamblaje de un sistema de coordenadas compartido. `style_seed` es una frase corta fija que todos los trabajadores deben igualar, que cuesta un puñado de tokens y reduce materialmente la deriva de registro.

|Γ| es el término dominante de S y es por tanto el objeto de la dirección de investigación de §6.3. **[v0.3]** Y es también el término H de la derivación de §6.5: medido en ≈ 39 tokens sobre el corpus del proyecto, es lo que hace que fragmentar demasiado fino sea caro con independencia de la coherencia.

9.3 Paquete de tarea

```
{
  "v": "0.3",
  "attempt_id": "hex",
  "contract": { /* Γ */ },
  "predecessors": [ { "task_id": "hex", "summary": "string", "tokens": 0 } ],
  "task": {
    "instruction": "string",
    "kind": "extract|classify|generate|summarize|transform|judge",
    "expects": { "format": "...", "min_tokens": 0, "max_tokens": 0 }
  },
  "unique_here": [...],
  "flank": { "lead": "string", "trail": "string", "sentences": 0 },
  "constraints": { "temperature": 0.0, "top_p": 1.0, "stop": [...] },
  "commitment_request": { "scheme": "lsh-activation-v1", "params": { "window": 32 } },
  "deadline_ms": 0,
  "lane": "PUBLIC|SANITISABLE",
  "tier": "GLOBAL|TRUSTED",
  "swarm_id": null
}
```

[v0.3] **unique_here** es la lista de elementos del conjunto de unicidad asignados a *este* fragmento y sólo a éste (§11.3). No es una instrucción de estilo: es la mitad visible de una restricción global resuelta en el plan. Un nodo no puede cumplir «exactamente una vez en toda la salida» porque no sabe qué escribieron los demás; sí puede cumplir «estos elementos van aquí».

[v0.3] **flank** transporta el solapamiento derivado en §6.5, con su longitud declarada en oraciones. Declararla explícitamente permite que el ensamblador sepa qué parte del material devuelto es flanco y no cuerpo, que es lo que hace que el solapamiento pueda descontarse del ensamblado en lugar de reaparecer como repetición.

Los paquetes del carril SENSITIVE nunca se emiten a nodos abiertos. El campo tier es ortogonal a lane: nombra la *población* que un paquete puede alcanzar en lugar de la sensibilidad de su contenido.

9.4 Resultado

```
{
  "v": "0.3",
  "attempt_id": "hex",
  "text": "string",
  "profile": { "model_family": "...", "model_version": "...", "quantization": "...",
    "prompt_template_id": "...", "sampling_params": { }, "seed": 0,
    "L_declared": 0 },
  "commitment": { "scheme": "lsh-activation-v1", "digest": "base64", "bytes": 0 },
  "telemetry": { "gen_ms": 0, "queue_ms": 0, "tokens_out": 0, "energy_j": null },
  "sig": "firma ed25519 sobre la serialización canónica de los campos anteriores"
}
```

[v0.3] **L_declared** es el tamaño de fragmento que la clase de nodo sostiene (§8.5). El orquestador lo usa para derivar N y para detectar nodos cuyo L declarado no corresponde a su comportamiento observado.

9.5 Anuncio de perfil de nodo

```
{
  "node_id": "clave pública ed25519",
  "models": [ { "family": "...", "version": "...", "quantization": "...",
    "ctx": 0, "tok_per_s_est": 0, "L_sustained": 0 } ],
  "capabilities": { "tee": false, "attestation": null },
  "swarm": { "swarm_id": null, "registry": null, "mtls_cert_fingerprint": null },
  "resources": { "vram_mb": 0, "ram_mb": 0 },
  "policy": { "max_tokens_per_task": 0, "kinds": ["extract","generate"] },
  "reputation": { "completed": 0, "audit_pass_rate": 0.0, "since": "ISO-8601" }
}
```

9.6 Despacho, especulación, sobre-despacho y reintento

1. Filtrar candidatos por tier primero —un paquete marcado TRUSTED sólo se ofrece a nodos cuya clave pública figure en la lista blanca del enjambre nombrado— y después por capacidad declarada, soporte de kind y RTT observado.
2. Para una tarea de criticidad k , despachar a k nodos seleccionados para **maximizar la diversidad de familias de modelo** sujeto a la clase de capacidad.
3. Arrancar un temporizador especulativo en el **p95** de la distribución de latencia observada *para esa clase de tarea y presupuesto de tokens*, no en una constante fija. Al expirar, despachar una réplica adicional; aceptar el primer resultado que verifique.
4. **[v0.3] Sobre-despacho con terminación temprana.** Para un nivel de N subtareas distintas, despachar $N + m$ y aceptar las primeras N que vuelvan y pasen el triage, descartando rezagadas. **Condición normativa:** esto sólo es admisible cuando las m extras son subtareas *genuinamente intercambiables* (§12.3). Si el contenido del fragmento j es necesario, recibir los otros $N-1$ no lo sustituye, y el esquema no aplica.
5. Cancelar réplicas pendientes al aceptar. Registrar las cancelaciones — un nodo cuyo trabajo se cancela habitualmente es lento, no deshonesto, y la reputación debe distinguirlos.
6. Ante fallo de verificación, re-despachar excluyendo al nodo que falló y registrar el evento para el muestreador de auditoría.

9.7 Tabla de parámetros

Parámetro	Símbolo	Por defecto	Derivación
Tamaño de fragmento	L	propiedad de la clase de nodo	§5.7; declarado por el nodo, verificado por comportamiento
Número de fragmentos	N	derivado	$N = \lceil \text{material}/L \rceil$ (§5.7). Ya no es un parámetro
Flanco	F	medido	máx(repetición más larga, dependencia más larga que cruza) (§6.5)
Costo de cabecera	H	medido	≈ 39 tokens en el corpus del proyecto (§6.5)
Presupuesto de contexto	S	derivado	$\lceil S \rceil =$
Razón de redundancia	ρ	derivada	$(L+2F)/L + H/(L \cdot s)$ (§6.5). Reportada, no configurada
Densidad de dependencias	δ	medida	relaciones necesarias que cruzan un límite, por fragmento (§6.8)
Umbral de costura	τ_{sem}	calibrado	§10.6; nunca una constante
Umbral de router	τ_{route}	calibrado, asimétrico	$\beta < 1$ en F_{β} [12]
Réplicas por criticidad	k	separado en tres	§12.4: $k_{\text{avail}}, k_{\text{verif}}, k_{\text{epist}}$

Parámetro	Símbolo	Por defecto	Derivación
Disparo especulativo	—	p95 por clase	§8.4
Extras de sobre-despacho	m	0 salvo intercambiabilidad	§9.6.4, §12.3
Tasa de muestreo de auditoría	λ	0.01-0.05	§13.3
Ancho máximo de plan	—	8	La cola de rezagados crece con el ancho
Profundidad máxima de plan	—	4	Más allá, la densidad de dependencias aconseja no fragmentar

10. Algoritmos

10.1 Router

```
función es_descomponible(P) -> (bool, score)
  f ← rasgos(P):
    · señales de tipo de tarea (extraer / enumerar / resumir vs probar / derivar / refactorizar)
    · longitud de P
    · densidad de marcadores de dependencia secuencial
    · presencia de estado mutable compartido (código, libros contables, totales)
    · petición de un artefacto único vs un conjunto de elementos
    · [v0.3] densidad de dependencias estimada  $\delta$  (§6.8)
  score ← clasificador(f)
  return (score >  $\tau_{\text{route}}$ , score)
```

τ_{route} se calibra con **F_β**, $\beta < 1$: un falso positivo —fragmentar algo que no debía fragmentarse— cuesta más que un falso negativo. Ésa es la lección de SoT-R, y es la diferencia entre un sistema que mejora sobre 2023 y uno que retrocede.

[v0.3] La inclusión de δ como rasgo es una consecuencia directa de §6.8: si la distorsión depende de la densidad de dependencias y no sólo de la tasa, entonces el router —que es quien decide si fragmentar— debe ver esa densidad. Un router que sólo ve longitud está midiendo el eje equivocado.

Y ese rasgo se midió, con un resultado que obliga a cambiar el diseño. Sobre 80 celdas operativas, un clasificador entrenado con validación dejando-una-tarea-fuera alcanza **AUC 0.57** con δ y reputación, y **0.61** sin ninguno de los dos: añadir δ no mejora nada (T0RR). Con sondas de capacidad pre-despacho en lugar de δ , **AUC 0.57** (T0RR2). **La predicción por celda no funciona**, y el fallo no es de δ en particular: ningún subconjunto de rasgos la levanta.

Lo que sí separa es la **clase de nodo**. El mismo corpus da impuestos medios por familia de +8.4 % a +26.5 %, y una política que decide por clase —fragmentar si el impuesto histórico de esa clase es negativo— obtiene **+0.0 %** frente a **+14.8 %** fragmentando siempre: con el presupuesto igualado todas las clases tienen impuesto histórico positivo, así que la política las rechaza a todas y evita el impuesto entero. La política por clase es el router que funciona: no necesita predicción por celda, y su decisión es exactamente la que P2 prescribe cuando la economía medida dice «no fragmentar».

Consecuencia de diseño. El router de §10.1 decide si *el prompt es descomponible* —y eso sigue verificado: 22 de 22 decisiones coinciden con la verdad del corpus (T0R)—. Lo que **no** debe intentar es predecir el costo de una celda concreta. Esa decisión se mueve al perfil del

nodo: la clase declara su historial, y el orquestador enruta por clase. δ se conserva como rasgo del plan y **se retira como rasgo predictivo del router**.

10.2 Planificador

```
función planificar(P, L) -> (D = (V,E), Uniq)
  fronteras ← candidatos_de_corte(P)           # [v0.3] P9
  peso      ← acoplamiento_estimado(frontera)   # dependencias que cruzarían
  cortes    ← seleccionar_mínimos(fronteras, peso, size_target = L)
  unidades  ← segmentar(P, cortes)
  para cada par ordenado (u, v):
    E ← E ∪ {(u,v)} si v requiere el *resultado* de u, no meramente su enunciado
  assert acíclico(D)
  si ancho(D) == 1: return NEGARSE             # una cadena no es paralelizable
  si profundidad(D) > profundidad_máx: return NEGARSE
  Uniq ← conjunto_de_unicidad(P,  $\Gamma$ )         # [v0.3] M2, §11.3
  asignar cada elemento de Uniq a exactamente un nodo de V
  return (D, Uniq)
```

La distinción entre requerir el *resultado* de una predecesora y requerir su *enunciado* es el punto crítico. El *least-to-most prompting* alcanza ≥ 99 % en SCAN frente a 16 % de cadena de pensamiento precisamente por respetar las dependencias de resultado secuencialmente [51]; un planificador que confunde una dependencia de resultado con una de enunciado convierte esa ganancia en pérdida.

[v0.3] Dos cambios de fondo. Primero, el corte ya no es por tamaño: L entra como `size_target` y la selección minimiza el acoplamiento cortado, que es P9 y es la minimización de δ de §6.8. Segundo, el planificador computa el conjunto de unicidad y lo asigna — la restricción global se resuelve aquí, donde hay información global, y no en el nodo, que no la tiene.

10.3 Empaquetado

```
función construir_paquete( $\Gamma$ ,  $t_i$ , preds, L, F) ->  $K_i$ 
  base ←  $\Gamma$                                      # nunca se recorta
  flanco ← tomar_flanco(vecinos( $t_i$ ), F)         # [v0.3] derivado, no porcentual
  ordenar preds por (peso de arista, recencia)
  para p en preds mientras quede presupuesto:
    s ← resumir(p.resultado, min(presupuesto, tope_por_pred))
    adjuntar(s)
  return ( $\Gamma$ , flanco, adjuntos,  $t_i$ , unique_here( $t_i$ ))
```

Γ nunca se recorta para cumplir un presupuesto. Si el presupuesto no alcanza, el planificador **aumenta L** —menos fragmentos, más grandes— en lugar de recortar contexto compartido. La v1.4 decía «reduce N », que es lo mismo bajo la formulación antigua; bajo la nueva, N es derivado y L es la palanca. Es el resultado de LongRAG, donde unidades de recuperación de 4K tokens y menos de ocho unidades superiores igualaron el estado del arte entrenado sin entrenamiento alguno [52].

10.4 Ensamblaje

```
función ensamblar(fragmentos, D,  $\Gamma$ , Uniq,  $\tau_{\text{sem}}$ ) -> (texto, informe_costuras,
informe_cardinalidad)
  ordenados  $\leftarrow$  aplanado_topológico(D)
  elegidos  $\leftarrow$  []
  para t en ordenados:
    cands  $\leftarrow$  fragmentos[t]
    elegidos.append( cands[0] si |cands| == 1
                     si no juez_selecciona(cands,  $\Gamma$ ) ) # seleccionar, no sintetizar
  discount_flanks(elegidos) # [v0.3] §9.3
  salida, costuras  $\leftarrow$  [], []
  para (a, b) en consecutivos(elegidos):
    sim  $\leftarrow$  cos( embed(ventana_final(a)), embed(ventana_inicial(b)) )
    si sim  $\geq \tau_{\text{sem}}$ :
      salida.append(a); costuras.append((a,b,"empalme",sim))
    si no:
      puente  $\leftarrow$  slm.escribir_transición(final(a), inicio(b),  $\Gamma$ )
      salida.append(a); salida.append(puente)
      costuras.append((a,b,"puente",sim))
  card  $\leftarrow$  verify_cardinality(salida, Uniq) # [v0.3] M2, §11.3
  return unir(salida), costuras, card
```

Cada costura se registra con su similitud y el camino tomado. El informe de costuras es parte de la respuesta, por P6.

[v0.3] Se añaden dos pasos. `discount_flanks` elimina el solapamiento declarado antes de empalmar, de modo que el flanco cumpla su función —permitir detectar repeticiones y continuidad— sin reaparecer como texto duplicado. `verify_cardinality` comprueba la salida contra el conjunto computado en el plan, no contra una regla enunciada en el contrato.

10.5 Consenso por alineamiento múltiple de réplicas (E16, revisado)

La sección 10.4 resuelve fragmentos *distintos* en posiciones *distintas*. Ésta resuelve k réplicas de la misma micro-tarea.

Dos niveles, deliberadamente distintos.

Nivel	Unidad	Mecanismo	Cuándo
Macro	Subtareas distintas de una tarea grande	Solapamiento y empalme con flanco (§10.4)	Trabajo generativo largo
Micro	k réplicas completas de la misma micro-tarea	Alineamiento múltiple y consenso (esta sección)	Toda micro-tarea de criticidad $k > 1$, incluida una petición atómica desde el principio

Una petición atómica —la que el router declina descomponer— se salta el nivel macro y va directo al micro con k réplicas. **Partir una pregunta atómica en preguntas parciales no es una operación soportada**, porque elimina información antes de muestrear en lugar de muestrear redundantemente; ninguna cobertura lo recupera.

10.5.1 Qué hace el estado del arte, y dónde está la apertura

La literatura reciente sobre incertidumbre en generación es abundante y buena.

Entropía semántica (Kuhn, Gal & Farquhar, ICLR 2023 [98]; Farquhar et al., *Nature* 2024 [99]) muestrea varias respuestas, las agrupa por equivalencia semántica mediante **implicación bidireccional** y calcula la entropía sobre los grupos de significado en vez de sobre las secuencias de tokens. AUROC promedio 0.790 sobre 30 combinaciones de modelo y tarea.

SelfCheckGPT (Manakul, Liusie & Gales, EMNLP 2023 [100]) produce puntuaciones de factualidad **por oración** comparando cada oración contra muestras completas.

Acuerdo semántico entre modelos distintos (Soiffer et al., 2025 [101]) usa el acuerdo entre modelos pequeños como señal de derivación.

Lo que todos comparten, y es la apertura: tratan las k respuestas como **una bolsa que se agrupa**. Ninguno las trata como **secuencias que se alinean**. Agrupar da un escalar por respuesta; alinear da una correspondencia **posición a posición**, que es lo que permite decir *dónde* divergieron y no sólo *cuánto*.

10.5.2 El instrumento existe y está maduro

El costo de sustitución se aprende, no se postula. Es la lección de BLOSUM (§5.2). En el dominio del texto ya está resuelta a nivel de cadenas: Ristad & Yianilos (1998) [96] dan «un algoritmo eficiente para aprender los costos primitivos de edición a partir de un corpus de ejemplos», mediante un transductor estocástico ajustado por EM.

Y está resuelta a nivel de frases. PPDB 2.0 (Pavlick et al., ACL 2015) [97] es una base de paráfrasis re-puntuada discriminativamente: recogieron juicios humanos sobre **26.455 pares**, cada uno evaluado por 5 personas en escala Likert de 5 puntos, y ajustaron una regresión ridge sobre **209 rasgos**.

El resultado es el dato más útil de toda esta investigación para el proyecto:

El ranking heurístico correlacionaba con el juicio humano a $\rho = 0.41$. El modelo ajustado empíricamente alcanzó $\rho = 0.71$. Y en ese modelo el coseno de embedding es un rasgo entre 209.

Eso es evidencia publicada y cuantificada de que **la distancia cruda de embedding no es el instrumento correcto** para juzgar intercambiabilidad semántica — que es exactamente lo que el mapa necesita juzgar, y exactamente lo que la v1.4 usaba.

El alineamiento por consistencia es el diseño que esto quiere. T-Coffee (Notredame, Higgins & Heringa, 2000) [94] construye una biblioteca primaria de alineamientos por pares y después la **extiende**: para cada par de residuos examina su alineamiento con residuos de las demás secuencias, y «el peso asociado a un par de residuos será la suma de todos los pesos reunidos mediante el examen de todos los tripletes que involucran a ese par». Traducido: **un acuerdo corroborado a través de una tercera respuesta independiente pesa más que un acuerdo entre dos**. Y la fase progresiva usa «puntuación específica de posición» en lugar de una matriz fija.

Y la formalización del producto también existe. Un perfil HMM (Eddy, 1998) [95] «convierte un alineamiento múltiple de secuencias en un sistema de puntuación específico de posición». Un modelo de puntuación específico de posición construido desde un conjunto de respuestas alineadas es la descripción formal de un mapa de confianza.

10.5.3 El modelo propuesto

M1 — Alineamiento múltiple de respuestas con sustitución semántica aprendida.

1. **Segmentar** cada una de las k respuestas en oraciones (la unidad de alineamiento de §5.7).
2. **Alinear** las k secuencias de oraciones con programación dinámica y penalización de hueco afín (Gotoh, 1982) [93], usando una matriz de sustitución semántica.

3. **Construir esa matriz empíricamente**, al modo BLOSUM: contar pares de oraciones que aparecen como sustituciones mutuas en casos verificados como correctos, y puntuar $\log(q_{\text{obs}} / e_{\text{esp}})$. El embedding entra como rasgo, no como veredicto.
4. **Extender por consistencia** al modo T-Coffee: ponderar cada correspondencia por su corroboración a través de las demás respuestas.
5. **Emitir** un perfil específico de posición sobre el alineamiento resultante. Ése es el mapa.

Qué es nuevo y qué no. Nada de los pasos 2, 3 y 4 es invención: son Needleman-Wunsch con Gotoh, Henikoff con Ristad, y Notredame. Lo nuevo es **aplicarlos a salidas de modelos distintos en lugar de a secuencias biológicas**, y hacerlo para producir una anotación por unidad en vez de un escalar por respuesta. La contribución es el ensamblaje del instrumento, no sus partes, y así hay que enunciarla.

10.5.4 Qué sigue retirado

La afirmación de fiabilidad del mapa de confianza sigue retirada, y M1 no la levanta.

La v1.4 midió la correlación entre acuerdo por unidad y corrección, primero contra un juez de clase par ($r = -0.030$ sobre 597 unidades, con el juez aceptando el 93.3 % — demasiado saturado para discriminar) y después contra clave de respuesta en tres corridas, que devolvieron razones de momios comunes de **3.47, 0.26 y 1.24**: por encima, por debajo y a caballo de 1 en la misma pregunta. Tres estimaciones mutuamente contradictorias no son una señal débil; son ninguna señal, medida tres veces.

M1 propone un **instrumento distinto**, no una reinterpretación de aquel resultado. Eso significa tres cosas, y conviene que estén juntas:

1. Las etiquetas de confianza se reportan como **acuerdo**, nunca como **exactitud**, y el mapa no se ofrece como garantía de fiabilidad. Esto no cambia.
2. **El mecanismo sigue siendo real.** Reportar *dónde* divergieron réplicas independientes es una capacidad, y es la que un proveedor con un solo modelo no tiene. Lo que no puede presentarse es la convergencia como evidencia de corrección.
3. M1 tiene su propia condición de muerte, enunciada en §15.6: si con el instrumento correcto y **en un régimen donde los modelos genuinamente discrepan** el acuerdo por unidad no supera al azar, entonces la localización no crea señal y M1 cae. La cláusula del régimen no es una escapatoria: es la corrección del defecto conocido de la primera medición, donde el juez aceptaba casi todo y no quedaba varianza contra la que pudiera aparecer una correlación. Declararla por adelantado es lo que impide usarla después.

Tres advertencias honestas que no cambian. El acuerdo no es verdad: modelos entrenados sobre corpus solapados comparten errores, y la convergencia sobre una falsedad común es un fallo correlacionado que el alineamiento no ve — por eso §9.6 obliga a diversidad entre familias. El mapa cuesta $k\times$, y se aplica por criticidad, no universalmente.

10.6 Calibración de umbrales

```
función calibrar_tau(pares_etiquetados, embedder,  $\beta = 0.5$ ) -> ( $\tau^*$ , curva)
  sims  $\leftarrow$  [ cos(embed(a.final), embed(b.inicio)) para (a,b,etiqueta) en pares ]
  para  $\tau$  en cuantiles(sims, 200):
    calcular precisión/exhaustividad de «es una costura rota»
     $F_\beta \leftarrow (1+\beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R)$ 
  return argmax_ $\tau$   $F_\beta$ , curva
```

$\beta < 1$ pondera la precisión: declarar rota una costura dispara una reescritura, y las reescrituras innecesarias son la forma en que un sistema degrada texto que ya estaba bien. τ debe re-derivarse cada vez que cambia el modelo de *embeddings*, por las razones de anisotropía de §6.9.

[v0.3] Mientras M1 no esté construido, este umbral es el instrumento operativo y se conserva. Cuando lo esté, la matriz de sustitución aprendida lo reemplaza en el nivel micro (§10.5) y τ_{sem} queda restringido al nivel macro, que es donde compara *final de un fragmento* con *inicio del siguiente* y no dos versiones de lo mismo.

10.7 La compuerta de triage

```
función triage(acarreo, tarea_que_lo_pidió) -> PASA | REINTENTA | DESCARTA
  si no responde_a(acarreo, tarea_que_lo_pidió): # predicado mecánico
    return REINTENTA si intentos < r si no DESCARTA
  si viola_esquema(acarreo, tarea.expects):
    return REINTENTA si intentos < r si no DESCARTA
  return PASA
```

M3 — Compuerta de triage por nivel topológico.

1. Un nivel no arranca hasta que los acarreos de sus predecesoras han pasado un **predicado mecánico**: *¿el acarreo que llegó responde a la tarea que lo pidió?* Sin juez, sin modelo, barato.
2. Un fragmento que no pasa se **aísla y se reintenta**, no se parchea en contexto ni se empalma con una advertencia.
3. Un fragmento que no pasa tras r reintentos se **descarta y se regenera** con un plan distinto. La degradación es una salida legítima del triage, no un fallo del sistema.
4. El consumo puede empezar **antes de que el nivel esté completo**, si el fragmento consumido está completo y verificado (§8.3, Shiber et al.).

Por qué importa, con un dato del proyecto. Hay un incidente registrado que esta compuerta atrapa: 42 de 60 paquetes despachados llevaban las respuestas de otro paquete. Sin compuerta, los acarreos pasan hacia adelante sin revisión y un error de un nivel se multiplica en el siguiente. Con compuerta, el radio de daño es un fragmento.

Y su condición de muerte. Si la compuerta rechaza tanto trabajo legítimo que el costo de reintento supera el daño que evita, M3 cae. Es una medición barata y su predicción es binaria: el incidente ocurre o no ocurre.

10.8 Auditoría de coherencia

Dos instrumentos, reportados por separado y nunca fundidos en una sola puntuación de «calidad» — fundirlos es exactamente como se esconde el daño [12]:

1. **Coherencia local por rejilla de entidades** [53] — menciones de entidades y papeles gramaticales entre oraciones, puntuadas desde probabilidades de transición.
2. **Taxonomía de errores de costura** — el subconjunto mecánicamente detectable de las clases de error identificadas a partir de 1.193 anotaciones humanas sobre 100 libros [54]: omisión de entidad, contenido duplicado, contradicción, cambio de registro o de tiempo verbal, referencia colgante, transición faltante, presentación repetida, nombres inconsistentes.

La cifra principal es el **impuesto de coherencia**: degradación relativa frente a generación monolítica con el mismo modelo.

[v0.3] Se añade un tercer informe, separado de los dos anteriores: el informe de cardinalidad (§11.3, punto 4), con sobre-compresión y sobre-expansión contadas por separado. Son errores distintos con causas distintas, y medirlos juntos los oculta.

Y una limitación que la v1.4 documentó y que sigue en pie. La rejilla de entidades no funciona sobre respuestas cortas: devolvió líneas base monolíticas entre 0.000 y 0.114 sobre todo el corpus, lo que convierte toda comparación relativa construida sobre ella en un cociente sobre un denominador casi nulo. O el corpus de evaluación se mueve a salidas más largas, o el instrumento se reemplaza; hasta entonces este documento tiene un instrumento de coherencia que funciona, no dos (L14).

11. Restricciones de cardinalidad

Esta sección es nueva. Trata la clase de restricción que ningún nodo puede cumplir aisladamente: «menciona este término exactamente una vez», «no repitas una formulación». Es, por las mediciones del proyecto, la clase irreducible.

11.1 Una hipótesis que no sobrevivió

La hipótesis era: *los mate pairs son el homólogo de las restricciones globales, porque son el mecanismo que la genómica inventó para información que ninguna lectura individual contiene.*

Concepto base. Un *mate pair* o lectura pareada es un par de lecturas secuenciadas desde los dos extremos de un mismo fragmento de ADN de longitud aproximadamente conocida. Weber & Myers (1997) [106] lo introdujeron porque «los pares de lecturas de ambos extremos tienen espaciado y orientación conocidos», lo que «ayuda al ensamblaje de secuencias que contienen elementos repetitivos dispersos».

La mitad de la hipótesis es correcta: las restricciones que cruzan fragmentos son reales, son la dificultad central, y la genómica sí inventó maquinaria explícita para información que ninguna pieza contiene.

La otra mitad es errónea, y lo es en cuatro ejes a la vez:

eje	mate pair	«exactamente una vez en toda la salida»
Aridad	Binaria y pre-identificada: nombra dos lecturas concretas, y el par existe antes del ensamblaje porque lo creó la preparación de la biblioteca	n-aria y cuantificada: no nombra ningún par; cuantifica sobre todos los fragmentos
Contenido métrico	Es fundamentalmente una distancia , con distribución — Myers et al. (2000) [107] reportan longitudes de inserto «normalmente distribuidas con 10 % de varianza»	No tiene distancia ni orientación
Dureza	Blanda: Opera (Gao, Sung & Nagarajan, 2011) [108] define concordancia como un predicado que el optimizador maximiza ; un mate pair discordante se tolera	Dura: una sola violación es un fallo, no un dato a ser superado en votación
Dirección de la información	Evidencia: una observación adicional muestreada de un genoma que ya existe	Especificación: no hay una respuesta verdadera que recuperar; es una condición impuesta sobre la salida

El cuarto eje es el más profundo. El ensamblaje genómico es **inferencia de máxima verosimilitud hacia una respuesta correcta que preexiste**. Swarmbly hace **satisfacción de restricciones sobre un espacio de salidas aceptables**. Los mate pairs viven del lado de la inferencia.

Qué sí es un mate pair. La transferencia no es nula: es de alcance más estrecho. Los mate pairs son el homólogo correcto de una clase real de restricción: **«el fragmento A y el fragmento B deben ser mutuamente consistentes en una posición relativa conocida»**. La conclusión debe retomar la anécdota de la introducción; la sección 4 debe recoger el hilo que dejó abierto la sección 2; una referencia a una figura en un fragmento debe corresponder a una figura definida en otro. Son binarias, tienen componente posicional, se conocen de antemano porque el plan las creó, y degradan con gracia cuando se violan parcialmente. Para éstas, el argumento de §11.3 transfiere con la homología.

11.2 El homólogo correcto está en la misma literatura

La genómica sí tiene un mecanismo cuya forma es «esto debe aparecer exactamente N veces en toda la salida». No son los mate pairs. Son la **multiplicidad** y la **unicidad**.

Multiplicidad de k-meros. Compeau, Pevzner & Tesler (2011) [104] lo enuncian como una operación de conteo que entra en la estructura del grafo: hay que determinar «cuántas veces aparece cada k-mero», y «si la multiplicidad de un k-mero es m, conectaremos su prefijo a su sufijo usando m aristas dirigidas (en lugar de una sola)». Eso es una restricción de cardinalidad global impuesta **estructuralmente** y no como comprobación posterior.

U-unitigs. Myers et al. (2000) [107], en el ensamblaje de *Drosophila*, describen que aquellas unidades «que con certeza representan ADN único fueron designadas U-unitigs» — y sobre ellas ancla todo el andamiaje. Es literalmente el mecanismo de «exactamente una vez en toda la salida»: el ensamblador **computa el conjunto de secuencias que deben aparecer una sola vez** y construye alrededor de ellas.

El modo de fallo, que confirma la transferencia. Myers (1995) [105] nombra la patología cuando se hace mal: buscar la cadena más corta que contenga todos los fragmentos hace que «en el caso de secuencias objetivo repetitivas este objetivo produzca respuestas sobre-comprimidas». **Sobre-compresión** es exactamente el fallo de un ensamblador que funde dos pasajes que debían permanecer distintos. Y su dual —emitir dos veces algo que debía aparecer una— es el fallo que `no_repeated_ngram` mide.

11.3 El modelo propuesto

La lección arquitectónica más fuerte viene de Medvedev et al. (2011) [109], que argumentan contra el tratamiento posterior: los mate pairs se han incorporado «como varios pasos heurísticos de post-procesamiento», lo que «todavía puede fallar en resolver repeticiones complejas»; su propuesta es incorporar la información **en la estructura del grafo misma**.

M2 — Restricciones de cardinalidad resueltas en la representación, no en la salida.

1. **Computar el conjunto de unicidad antes de despachar.** Qué términos, entidades y formulaciones deben aparecer exactamente una vez en la salida final. Es el análogo de identificar U-unitigs.
2. **Asignar cada elemento de ese conjunto a exactamente un fragmento**, como propiedad del plan y no como instrucción a los nodos. Un nodo no puede cumplir «exactamente una vez» porque no sabe qué escribieron los demás; el planificador sí. Es el campo `unique_here` de §9.3.

3. **Verificar la cardinalidad en el ensamblado**, contra el conjunto computado en el paso 1, no contra una regla enunciada en el contrato.
4. **Registrar los dos modos de fallo por separado**: sobre-compresión (se fundió lo que debía distinguirse) y sobre-expansión (se repitió lo que debía aparecer una vez).

El punto 2 es el cambio real, y hay un número con el que compararlo. Hoy el proyecto pide `term_once` en el contrato y lo impone mecánicamente en el ensamblador. Eso funciona, y su medición lo confirma: la imposición mecánica subió el cumplimiento de **6/24 a 18/24**, por encima del monolítico en 13/24. Pero es una corrección posterior. **Asignar la unicidad en el plan hace que la restricción sea imposible de violar en vez de detectable después**, que es precisamente el argumento de Medvedev et al.

Su condición de muerte, enunciada por adelantado: si la asignación en el plan no reduce las violaciones por debajo de lo que ya logra la imposición mecánica en el ensamblador (18/24), M2 no aporta y cae. La predicción de M2 es fuerte —la tasa de violación debe caer a cero, no mejorar— precisamente para que sea fácil de refutar.

Se midió, y la predicción fuerte es falsa tal como estaba redactada. Sobre 35 términos únicos en 10 celdas reales (2 tareas × 5 familias), los nodos que reciben `unique_here` producen **6 de 35** exactamente una vez: **0 términos omitidos** y **29 repetidos** (T03R). Un modelo pequeño no obedece «exactamente una vez» porque se lo digan; el campo le dice qué le toca, no cuántas veces escribirlo.

Y sin embargo M2 se sostiene, corregido. Pasando esas mismas salidas por la imposición mecánica del ensamblador —la de la v1.4, no una nueva— el resultado es **35 de 35** exactamente una vez. La asimetría es el hallazgo: la sobre-expansión se recorta mecánicamente, la sobre-compresión no se inventa, y hubo **cero** omisiones. Es decir: **la asignación en el plan elimina el modo de fallo irreparable, y la imposición en el ensamblador elimina el reparable.**

Corrección a M2. El mecanismo no es «asignación en el plan **en lugar de** imposición en la salida», sino **asignación en el plan más imposición mecánica en el ensamblado**. La asignación *localiza* la responsabilidad; la ejecución la *garantiza*. La frase «cero por construcción» queda **retirada**: lo que es cero por construcción es la sobre-compresión, no la violación de cardinalidad en general. La condición de muerte declarada (batir 18/24) se cumple —35/35 sobre 35 términos—, pero se cumple gracias a un componente que M2 proponía volver innecesario, y eso hay que decirlo así.

12. Redundancia: para qué sirve *k*

12.1 Una segunda hipótesis que no sobrevivió

La hipótesis era: *en lugar de enviar el mismo paquete a k nodos, enviar $N + m$ fragmentos codificados de los cuales cualesquiera N basten para reconstruir.*

La motivación era buena y está bien fundada en la literatura de almacenamiento. Weatherspoon & Kubiatowicz (2002) [126] muestran que los códigos de borrado «usan un orden de magnitud menos de ancho de banda y almacenamiento que la replicación para sistemas con MTTF similar»: con un millón de máquinas y 10 % caídas, dos réplicas completas dan «sólo dos nueves de disponibilidad», mientras que una codificación en 32 fragmentos da «más de ocho nueves» con el mismo almacenamiento. Los *fountain codes* llevan eso al límite:

Luby (2002) [124] demuestra recuperación desde «cualquiera $k + O(\sqrt{k} \cdot \ln^2(k/6))$ símbolos de codificación con probabilidad $1 - \delta$ »; Shokrollahi (2006) [125] lo mejora a «cualquier subconjunto de tamaño $(1+\epsilon)k$ » con $O(1)$ operaciones por símbolo.

Por qué no transfiere. La razón es limpia y no admite matiz.

Los fountain codes son códigos lineales sobre incógnitas compartidas. La definición de Shokrollahi lo dice: «cada símbolo de salida es la suma de algunos de los símbolos de entrada», y la decodificación es eliminación gaussiana o *peeling* sobre ese sistema lineal. Todo el mecanismo por el que «cualquiera $k(1+\epsilon)$ bastan» **es** que los símbolos recibidos son **ecuaciones lineales en las mismas incógnitas**.

Los fragmentos de Swarmby no son ecuaciones en incógnitas compartidas. Cada nodo **genera texto nuevo**. No hay XOR, no hay cuerpo algebraico, no hay inversa. No hay nada que despejar.

Y no es una inferencia propia: es la limitación explícita de la literatura de *coded computing*. Kosaian, Rashmi & Venkataraman [128] constatan que muchos trabajos emplean códigos de borrado para cómputo de **funciones lineales**, y que «hasta donde sabemos, ninguno de los trabajos existentes es aplicable a clases más amplias de cómputos no lineales». Lo que sí existe confirma el diagnóstico por el lado positivo: Mallick et al. (2019) [127] obtienen hasta **3× de aceleración** aplicando códigos rateless a multiplicación matriz-vector distribuida — **porque la multiplicación matriz-vector es lineal**. Y el único intento serio de codificar inferencia no lineal, ApproxIFER [129], logra recuperación **aproximada** sobre salidas de clasificación de baja dimensión, con pérdidas de exactitud de hasta ~6-9 % en modo degradado.

Se buscó específicamente trabajo previo aplicando códigos fountain a redundancia de agentes LLM o a inferencia descentralizada de LLM. **No existe**. La transferencia no está intentada, y la linealidad explica por qué.

12.2 Aplicando el método de §3

Esta caída es el ejemplo más limpio de la prueba del modo de fallo. Los fountain codes se propusieron por su propiedad de recuperación —«cualquiera $k(1+\epsilon)$ bastan»— sin ninguna condición sobre cuándo esa propiedad deja de existir. La condición existe, está publicada, y es la linealidad. Buscarla habría ahorrado la hipótesis entera.

12.3 Qué sí transfiere: la economía, no el mecanismo

La lección real de la literatura de almacenamiento no es sobre XOR. Es sobre **umbrales y quóruns**: para una fiabilidad objetivo, $(N, N+m)$ bate a la duplicación k veces.

M5a — Sobre-despacho con terminación temprana. Despachar N sub-tareas distintas más m extras, aceptar las primeras N que vuelvan y pasen el triage, descartar rezagadas. Es un esquema de umbral, entrega el ahorro de ρ que se buscaba, **y no es un fountain code**. Llamarlo así sería el error que la sección 3 existe para evitar.

Con una condición que hay que enunciar, porque es el núcleo del compromiso: las m extras sólo son gratis si las sub-tareas son **genuinamente intercambiables**. El punto entero de la codificación de borrado es que *cualquiera* k símbolos sirven. Los fragmentos de Swarmby no son intercambiables: si el contenido del fragmento j es necesario, recibir los otros $N-1$ no lo sustituye. Hacerlos intercambiables exige redundancia de contenido, que cuesta exactamente el ρ que se quería ahorrar. Por eso §9.6.4 lo declara normativamente condicional.

12.4 Separar los tres propósitos de k

Éste es el hallazgo que reordena el papel de k , y es probablemente más valioso que la hipótesis original.

k sirve hoy a **tres propósitos distintos** que el diseño no separa:

- **Disponibilidad** (k_{avail}) — que la tarea se complete aunque un nodo se caiga.
- **Verificación** (k_{verif}) — que un nodo deshonesto no imponga un resultado falso.
- **Redundancia epistémica** (k_{epist}) — que el mapa de divergencia tenga algo que alinear.

Sarmenta (2002) [133] mide el costo de cada vía en computación voluntaria: la votación «reduce las tasas de error exponencialmente con la redundancia, pero requiere que todo el trabajo se haga varias veces, y no funciona bien cuando hay muchos sabotadores»; el *spot-checking* «reduce la tasa de error linealmente con la cantidad de trabajo a realizar, mientras sólo cuesta una fracción extra del tiempo original»; y la combinación acreditada da «niveles de corrección garantizables matemáticamente con mucho menor ralentización». BOINC [130] implementa la versión desplegada: replicación adaptativa que logra «una cota baja en la tasa de error [...] incluso en presencia de voluntarios maliciosos, imponiendo sólo una pequeña sobrecarga de rendimiento».

M5b — La consecuencia de diseño. Si k existe por **verificación**, los códigos de borrado nunca fueron la herramienta; el *spot-checking* con credibilidad lo es. Si k existe por **disponibilidad**, el sobre-despacho con umbral lo resuelve. Y si k existe por el **mapa de divergencia** —redundancia epistémica— entonces no es reemplazable por ninguna de las dos, porque su producto no es tolerancia a fallos sino señal.

Tres propósitos, tres mecanismos, y hoy un solo parámetro. **Separarlos es una mejora inmediata del protocolo, disponible sin medir nada** — es por eso que §15.6 lo pone primero en el orden de ataque.

Y conecta con algo que la v1.4 ya había descubierto por otra vía. La sección sobre enjambres de confianza (§13.5) observaba que una lista blanca criptográfica elimina la necesidad de redundancia adversaria pero no la de redundancia epistémica, y exigía que bajar k a 1 se declarara explícitamente. Eso era esta separación, vista desde un caso particular. Ahora es general.

Su condición de muerte: si los tres resultan estar tan acoplados en la práctica que separarlos no ahorra nada, M5 cae.

13. Privacidad, verificación y nodos adversarios

13.1 La fragmentación no es cifrado

El desarrollo temprano de este concepto describía la fragmentación descontextualizada como una forma de «cuasi-cifrado», con el razonamiento de que un nodo que tiene un fragmento sin contexto global no tiene nada de valor. Ese razonamiento no sobrevive al contacto con la literatura de reidentificación, y la afirmación está retirada.

Cuatro hallazgos convergen. El primero es que los cuasi-identificadores bastan: la combinación de código postal, fecha de nacimiento y sexo identifica unívocamente a cerca del 87 % de la población estadounidense aunque ninguno de los tres sea un identificador por sí solo [55], y aunque una revisión posterior sitúe la cifra cerca del 63 %, eso no tranquiliza.

Toda defensa sintáctica de fragmentación o generalización propuesta en esa literatura ha sido rota por un ataque posterior [56], y los datos dispersos de alta dimensión resultan inherentemente reidentificables a partir de un puñado de atributos gruesos y ruidosos [57].

El segundo es que el estilo es en sí mismo un identificador. La atribución de autoría opera a escala de internet [58], sobrevive al acortamiento y al cambio de dominio entre plataformas [59], y funciona por debajo de 280 caracteres [60]. El tercero es que las representaciones intermedias se invierten: los *embeddings* de texto revelan casi tanto como el texto [61], y los prompts pueden recuperarse sólo desde las salidas del modelo [62].

El cuarto es decisivo porque aborda esta arquitectura directamente. En inferencia partida, el ataque ActInv alcanza precisión y exhaustividad por encima del 98 % en casi todos los casos evaluados, con ROUGE-L consistentemente por encima de 0.96. Cortar tras dos bloques de cliente de Qwen3-0.6B da 99.76 % de precisión, y aun a siete bloques retiene 77.74 %. Las defensas rinden poco: al 70 % de dispersión de activaciones la precisión decrece «sólo modestamente» [63].

Swarmbly no transmite activaciones, lo que la sitúa en mejor posición que la inferencia partida. Pero la dirección de la evidencia es inequívoca, y hay un argumento interno adicional:

§6.2 establece que la coherencia requiere enviar el contrato global Γ a todo trabajador. Un nodo que tiene Γ tiene el objetivo, la audiencia, el formato y las restricciones de la sesión. Descontextualización y coherencia son antagonistas por construcción.

13.2 Lo que sí puede afirmarse

Swarmbly reduce la superficie de exposición frente a un proveedor centralizado que lee y retiene el prompt completo, y frente a esquemas de paralelismo de tubería en que los nodos observan activaciones intermedias y texto en generación. **No provee garantía criptográfica de confidencialidad.** Un adversario que controle una fracción significativa de nodos, o que correlacione por tiempos e identificador de sesión, puede reconstruir una porción sustancial de una sesión.

Dos canales residuales merecen nombrarse porque son baratos de pasar por alto: **correlación temporal** (los fragmentos de una sesión llegan en ráfaga) y **huella del contrato** (un Γ distintivo es en sí mismo un identificador de sesión entre los nodos que lo reciben). Las mitigaciones —despacho con fluctuación, paráfrasis del contrato por nodo— cuestan latencia y coherencia respectivamente, que es otra vez §6.2.

[v0.3] Y esta versión añade un tercer canal, que es consecuencia de M2. El campo `unique_here` (§9.3) le dice a un nodo qué elementos son únicos en toda la salida. Eso es información global sobre la sesión que la v1.4 no enviaba. El intercambio es explícito: M2 compra corrección de cardinalidad al precio de filtrar la estructura de unicidad del documento. En el carril SANITISABLE esto es aceptable; en cualquier cosa por encima, `unique_here` debe omitirse y la verificación de cardinalidad recaer enteramente en el ensamblador — que es la conducta de la v1.4 y es peor, pero es la que corresponde.

13.3 Verificación

La confidencialidad criptográfica fuerte es inasequible aquí, y conviene enunciar los números para que la conclusión no se confunda con derrotismo. El MPC de propósito general sobre un transformador corre a una ralentización del orden de 10^4 – $10^6\times$, con 280.99 GB de comunicación para una sola inferencia de BERT-Base [64, 65]; los mejores sistemas de dos partes reportan unos 8 minutos por token para LLaMA-7B [66]. Las pruebas de conocimiento

cero de inferencia necesitan menos de 15 minutos para probar un paso hacia adelante de un modelo de 13B [67]. Nada de esto cabe en una economía voluntaria.

Lo que sí cabe es un esquema por capas:

Capa 1 — integridad computacional por compromiso sensible a la localidad. Un esquema de compromiso sobre activaciones detecta sustitución no autorizada de modelo, prompt o precisión con 100 % de exactitud, cero falsos positivos y cero falsos negativos en las pruebas reportadas, a 258 bytes por 32 tokens —unas 1000× de compresión frente a *embeddings* crudos— validando más rápido que la inferencia original [68]. Esto es lo que hace posible un mercado de nodos: cuesta casi nada y cierra el fraude obvio, que es un nodo anunciando un modelo de 8B y sirviendo uno de 1B.

Capa 2 — auditoría pública muestreada. Verificación a aproximadamente 1 % del costo de inferencia, segura bajo un supuesto de *un verificador honesto* en lugar de mayoría honesta, con probabilidad de fallo $P_{\text{fail}} \sim \rho^k$ [69]. La propiedad esencial: **los trabajadores no pueden distinguir una tarea de auditoría de una real.**

Capa 3 — la selección como defensa. Con $k > 1$ réplicas, la selección por juez de \$10.4 ya descarta fragmentos anómalos como efecto secundario de mejorar la calidad [39]. Es la defensa más barata del sistema porque la paga otra cosa.

[v0.3] Capa 0 — el triage. La compuerta de \$10.7 no es una defensa criptográfica y no se presenta como tal, pero es la primera que actúa y es gratuita: un fragmento que no responde a la tarea que lo pidió no entra al ensamblado, venga de un nodo lento, roto o deshonesto. Su valor es de contención, no de detección.

Lo que ninguna capa hace es verificar la *fidelidad semántica*. La capa 1 prueba que un modelo declarado se corrió sobre una entrada declarada; no prueba que la prosa resultante sea verdadera ni no maliciosa. Esa brecha importa porque todo fragmento devuelto es entrada no confiable que fluye al modelo del cliente — territorio de inyección de prompts, cadena de suministro y manejo impropio de salidas [70]. Y no se puede confiar en que el cliente lo note: modelos de razonamiento de mercado atribuyen fallos en sistemas agénticos con menos del 10 % de exactitud [71]. Un sistema que no puede atribuir fallos *honestos* no detectará los adversarios. La respuesta del protocolo es acotar el radio de daño: los fragmentos son datos, nunca instrucciones; el ensamblador corre con defensas de manejo de salida; y se imponen esquemas de salida por kind antes de que un fragmento entre al contexto de ensamblaje.

13.4 Carriles de sensibilidad

Carril	Criterio	Destino	Costo
PUBLIC	Sin PII, sin secreto comercial	Nodos voluntarios abiertos	Ninguno
SANITISABLE	PII detectable y seudonimizable	Nodos abiertos; rehidratado localmente	Riesgo residual real (abajo)
SENSITIVE	Salud, legal, financiero, identificable	Ejecución local, o TEE atestiguado	<7 % de sobrecarga media en computación confidencial sobre H100 [72]; mediciones independientes bajo Intel TDX reportan 8.9-21.8 % [73]

El carril de TEE es lo que hace el protocolo adoptable por una organización, y es asequible: una sobrecarga de un dígito porcentual es la única primitiva de confidencialidad de este espacio con esa propiedad.

El carril SANITISABLE debe describirse honestamente. Contra un modelo sin defensas sobre un corpus de texto legal, la extracción de PII alcanza ~23 % de exhaustividad y ~30 % de

precisión, y la *inferencia* de PII desde 100 candidatos alcanza 70 %, 50 % y 28 % en tres corpus. La privacidad diferencial a $\epsilon=8$ reduce la exhaustividad de extracción a cerca del 3 % — pero no a cero [74], y la generación diferencialmente privada degrada medibles la calidad del lenguaje [75]. La sanitización reduce el riesgo; no lo elimina, y la interfaz debe decirlo en lugar de enterrarlo.

13.5 Niveles dinámicos de privacidad y enjambres de confianza

Los carriles de §13.4 clasifican el *trabajo*. No dicen nada de la *población de máquinas* que ese trabajo puede alcanzar. Un segundo eje está disponible y cuesta casi nada añadirlo: la topología puede escalonarse, de modo que la decisión de enrutamiento sea un par — qué carril y qué malla.

Clasificación, y dónde corre. Toda petición pasa un clasificador de privacidad antes de la planificación, en dos modos. El primero es una **bandera dura manual** ---privacy=trusted, --privacy=local— determinista, declarada por el usuario, y nunca anulada por la vía automática. El segundo es **triage automático**: un modelo local pequeño hace reconocimiento de entidades sobre el prompt y eleva el nivel cuando detecta entidades de clases reguladas.

La propiedad esencial es que ese clasificador corre **enteramente en el cliente**. Un clasificador de privacidad que consulta a la red para decidir si el prompt es privado ya divulgó el prompt. Es además deliberadamente orientado a exhaustividad: debe sobre-clasificar, porque el costo de enrutar un prompt público a un enjambre de confianza es algo de rendimiento, mientras que el costo del caso contrario es el fallo que el nivel existe para evitar.

Los tres niveles. Nivel 1, la malla global no confiable, es el valor por defecto y es la red que este documento describe: nodos voluntarios abiertos, carriles PUBLIC y SANITISABLE, la pila de verificación completa, y redundancia al k derivado en §7.4.

Nivel 2, el enjambre de confianza, es una submalla con permisos. La membresía es una lista blanca criptográfica de claves públicas bajo un registro controlado por el operador; cada enlace lleva TLS mutuo; el despliegue típico es una LAN corporativa, una red de campus o una superposición VPN. El protocolo no cambia —un enjambre de confianza es el mismo protocolo sobre una población restringida, no un segundo protocolo— y esa restricción es deliberada. Los tiempos de ida y vuelta dentro de un enjambre así colapsan de decenas o centenas de milisegundos a muy por debajo de uno, lo que significa que la asimetría de ancho de banda de §1.2 queda localmente suspendida. Sería técnicamente posible correr una partición más fina dentro del cortafuegos. Swarmbly no lo hace, porque un despliegue que se comporta de una manera dentro del perímetro y de otra fuera son dos sistemas que implementar, verificar y razonar. Lo que la baja latencia compra en su lugar es holgura en el presupuesto de contexto: **L mayor, F más generoso, p más alto**, y por tanto mejor coherencia al mismo tiempo de reloj — una mejora obtenida gastando el mismo parámetro de diseño y no introduciendo un mecanismo nuevo.

Nivel 3, ejecución puramente local, se selecciona con --privacy=local y significa lo que dice: ningún paquete sale de la máquina. Es el único nivel en que Swarmbly hace una afirmación incondicional de confidencialidad, y puede hacerla precisamente porque no hay red sobre la cual hacerla.

Redundancia en un enjambre de confianza. La v1.4 ya observaba aquí que k servía a dos propósitos a la vez y que una lista blanca elimina uno de ellos pero no el otro. La sección 12.4 generaliza esa observación a tres propósitos, y esta sección es ahora su caso particular. Un enjambre de confianza **PUEDE** fijar $k_{\text{verif}} = 1$, porque la defensa adversaria ya está resuelta en la capa de identidad. **No puede** fijar $k_{\text{epist}} = 1$ sin perder el mapa de divergencia, y cuando lo hace, los metadatos de respuesta registran que no se produjo mapa y el cliente lo

expone explícitamente en lugar de presentar un mapa vacío como acuerdo. k_{avail} queda acotado por la ecuación de cobertura: con $c = 1$ no hay margen alguno contra pérdida, así que el diseño exige $k_{\text{avail}} \geq 2$ siempre que la tasa de pérdida intra-enjambre medida exceda la tolerancia ε , por fiable que sea la LAN.

Por qué el nivel importa más allá de la ingeniería. Los regímenes de protección de datos están escritos alrededor de procesadores identificables y contractualmente vinculados: las relaciones de encargado del RGPD, los acuerdos de socio comercial bajo HIPAA y sus equivalentes presuponen una entidad que pueda ser nombrada, auditada y responsabilizada. Un voluntario anónimo no puede ser encargado bajo ninguno. Un nodo en lista blanca, mutuamente autenticado, dentro del registro del propio operador, sí. El nivel 2 no es una prestación de rendimiento; es la construcción bajo la cual esta arquitectura se vuelve lícita donde el nivel 1 no lo es.

Y lo que no hace. Un enjambre de confianza reubica la confianza, no la elimina. Quien controla la lista blanca controla el enjambre, lo que hace de la gobernanza del registro una función crítica de seguridad; y un miembro comprometido dentro del perímetro es *más* peligroso que un nodo no confiable fuera, precisamente porque la redundancia que lo habría atrapado pudo haberse reducido. El TLS mutuo autentica el canal y la identidad; no dice nada sobre si el modelo detrás de esa identidad es el declarado. Por eso el compromiso sensible a la localidad de §13.3 sigue siendo **OBLIGATORIO** dentro de un enjambre de confianza aun donde se relajen el muestreo de auditoría y el voto por mayoría.

13.6 Resistencia a Sybil: una limitación, declarada

Sin una autoridad de identidad confiable, un solo adversario puede presentar arbitrariamente muchas identidades distintas, derrotando **cualquier** esquema basado en redundancia o voto por mayoría [76]. Los sistemas de reputación no escapan: el algoritmo P2P canónico requiere un conjunto de pares *pre-confiables* para ser resistente a Sybil, lo que reintroduce el ancla que pretendía eliminar [77].

Que esto no sea meramente teórico se ve en el despliegue de computación voluntaria de referencia, donde el 41.4 % de los hosts pertenecía a usuarios de un solo host, el 44.2 % a usuarios con 2-10 hosts, y el mayor usuario individual operaba 2.987 hosts [47] — concentración extrema, por un participante benigno sin incentivo alguno para ocultarlo.

Swarmbly no es por tanto resistente a Sybil en sentido fuerte, y el protocolo lo dice. Adopta confianza por capas: reputación acumulada, un costo de entrada al registro, auditoría muestreada con penalización económica, y un conjunto de nodos ancla operados por la fundación para el arranque en frío.

14. Economía, gobernanza y sostenibilidad

14.1 Créditos, no tokens

El desarrollo previo proponía una preminación del 15 % para el fundador y una comisión de protocolo del 0.5 % sobre cada micropago. Ambas están retiradas por razones regulatorias y narrativas. El marco suizo clasifica los tokens como de pago, de utilidad o de activo, con una prueba de dos condiciones para escapar a la clasificación como valor [78]; un instrumento preminado y transferible con expectativa de apreciación es el arquetipo que la dispara. Las exenciones europeas son estrechas — 1.000.000 € en doce meses, o 150 personas por estado miembro, con reglas de proveedor de servicios en vigor desde el 30 de diciembre de 2024 [79].

El diseño que queda fuera de ambos regímenes es deliberadamente poco emocionante: créditos no transferibles entre cuentas, ganados procesando y gastados solicitando; sin preventa y sin preminación; utilidad inmediata desde el primer día; saldos que caducan para desincentivar el acaparamiento; y conversión fiat en un solo sentido — las empresas compran capacidad a través del brazo comercial, los voluntarios no venden créditos.

14.2 Licencia y gobernanza

La implementación del protocolo es **AGPL-3.0-or-later**. Su cláusula 13 cierra la brecha de uso en red que la GPL deja abierta [80]. Tres salvedades: la obligación se adhiere al Programa y sus modificaciones y no a una pila propietaria circundante; la licencia cubre software y no el protocolo, de modo que una reimplementación de sala limpia es lícita; y su fuerza práctica es disuasión y no litigio.

La guía empírica más fuerte disponible viene de la reciente oleada de cambios de licencia: de los cuatro proyectos más conocidos que endurecieron sus licencias, tres produjeron un fork independiente exitoso (OpenSearch, Valkey, OpenTofu), y dos de los cuatro revirtieron después a AGPL [81, 82]. Empezar en AGPL y quedarse ahí es la posición que la historia respalda.

Dos decisiones estructurales se siguen. La licencia dual se rechaza — requiere una entidad que pueda vender excepciones propietarias, lo que es incompatible con una fundación cuyo mandato es la apertura; los ingresos vienen del servicio gestionado. Y **la marca, no el copyright, es la palanca de control operativa**. La contribución es por firma DCO, no por CLA, porque la cesión de copyright crea fricción precisamente con la comunidad que este proyecto necesita.

Sobre estructura: una *Verein* suiza puede constituirse rápido y sin capital mínimo, lo que basta para sostener derechos y recibir subvenciones; una *Stiftung* es el instrumento correcto más adelante. La afirmación de que una fundación es «inadquirible e insilenciable» es exagerada en cualquier caso — las fundaciones se capturan por juntas, dependencia de donantes y control de repositorios y marcas. La independencia es una práctica, no una forma jurídica.

14.3 Sostenibilidad, no reivindicada

Los centros de datos consumieron 415 TWh en 2024, cerca del 1.5 % de la electricidad mundial, con proyecciones a 945 TWh en 2030 [83]. Las instalaciones a hiperescala de EE. UU. se abastecen de redes medidas en 545 gCO₂/kWh frente a una media nacional de 370 g [84]. Esas cifras respaldan la *motivación*.

No respaldan una afirmación de beneficio neto, y no la hago. La energía por token varía en casi tres órdenes de magnitud entre configuraciones, y los aceleradores de centro de datos logran la menor energía por token en la gran mayoría de los escenarios; el consumo en reposo de 12–59 W lo paga íntegro un nodo disponible y sin usar [85]. El PUE global es 1.54, pero los hiperescaladores operan a 1.09–1.15 frente a un ~1.0 efectivo de un hogar — un margen de 9–15 %, no un orden de magnitud [86].

El compromiso que asumo es procedimental: adoptar el estándar Software Carbon Intensity — $SCI = ((E \times I) + M) / R$, ISO/IEC 21031:2024, que excluye explícitamente las compensaciones [87]— instrumentar nodos y cliente, y **publicar el resultado muestre lo que muestre**. El argumento motivador defendible es el del carbono incorporado: extender la vida útil de hardware que ya existe evita nueva fabricación.

Sobre la demanda inducida. Abaratar un recurso suele aumentar su consumo total en lugar de desplazar el uso existente —la paradoja de Jevons— y un revisor en un fondo climático lo

planteará. La afirmación de Swarmbly no es que esa demanda desaparezca, sino que **la absorbe hardware ya fabricado**. Dos condiciones acotan el argumento y las dos se enuncian: vale mientras **exista capacidad ociosa**, y vale sólo para la fracción de tráfico servida por hardware voluntario genuinamente preexistente, lo que **excluye explícitamente los nodos ancla** del período de arranque.

14.4 Arranque: nodos ancla, declarados

Una red cuya oferta y demanda deben llegar simultáneamente no arranca sola. El arranque de Swarmbly subsidia la oferta: la fundación opera capacidad alquilada para que el servicio sea rápido y estable desde el primer día, y la retira a medida que crece la oferta comunitaria.

Esto crea una exposición de integridad que el protocolo maneja por divulgación. Durante ese período una porción del «enjambre voluntario» es hardware de centro de datos alquilado, y para esa porción **el argumento de carbono incorporado no aplica y la afirmación de descentralización sólo es parcialmente cierta**. Los compromisos son por tanto explícitos: tales nodos se llaman **nodos ancla operados por la Fundación** y se etiquetan como tales en el registro; el panel público reporta en tiempo real **la cuota de tráfico servida por nodos ancla frente a nodos comunitarios**; y la Fundación publica una trayectoria objetivo para esa cuota y reporta contra ella.

15. Evaluación

15.1 Categorías de tarea

El encaje requiere cinco atributos simultáneos: descomponible en subtareas genuinamente independientes; tolerante a latencia; intensivo en tokens; dependencias débiles entre fragmentos; contenido verificable o no sensible.

Categoría	Encaje	Razonamiento
Procesamiento masivo de documentos	Alto	Vergonzosamente paralelo, sin dependencias, tolerante a latencia
Generación de datos sintéticos, etiquetado	Alto	Independiente por muestra; verificable por filtro; gran volumen
Barridos de migración de código	Alto	Independiente por archivo; verificable compilando y corriendo pruebas
Evaluación y juicio a escala	Alto	Independiente; la agregación es un voto
RAG sobre corpus grandes	Moderado	Etapas de mapeo paralela; pero pocas unidades grandes baten a muchas pequeñas [52]
Informes largos estructurados	Moderado	Descomponible por sección — y precisamente donde SoT se degrada [12]
Razonamiento matemático multi-salto	Pobre	Dependencias de resultado [51]
Código con estado mutable compartido	Pobre	El fallo canónico por decisiones implícitas en conflicto [33]
Chat interactivo de baja latencia	Muy pobre	Los métodos sin pérdida de un solo nodo ya dominan [13, 14, 15]

[v0.3] Bajo §6.8, esta tabla admite una lectura que la v1.4 no tenía: las filas de encaje pobre son las de **δ alta**, y las de encaje alto son las de **δ baja**. Eso no es una coincidencia de redacción — es lo que la superficie $D(\rho, \delta)$ predice, y convierte la tabla en algo comprobable en lugar de una lista de intuiciones.

15.2 El criterio de abandono

Adelante / alto. Debe existir un ρ al que la degradación de coherencia esté **por debajo del 5 % relativo a la generación monolítica, en al menos una categoría de tarea**. Si no existe tal ρ , la arquitectura no es viable para ensamblaje generativo, y el proyecto debe detenerse o restringirse a cargas sin costura que romper — clasificación, extracción, etiquetado.

Cómo se aplica el criterio, y por qué la forma importa. «Por debajo del 5 %» es una afirmación sobre una cantidad estimada a partir de un corpus finito, así que el criterio se salda contra un **intervalo, no contra una estimación puntual**: la cota superior de un intervalo bootstrap del 95 %, agrupado por prompt, debe caer por debajo del 5 % en la celda nombrada. Esa cláusula es parte del preregistro original y no una adición posterior — que es la única razón por la que vale algo, dado que un criterio apretado después de que llegan los datos no prueba nada y uno aflojado prueba menos.

[v0.3] Y esta versión añade una segunda línea base, por la crítica de Zhang et al. (§2.4): el cómputo adicional debe justificarse también contra **un solo agente con self-consistency**, no sólo contra el monolítico ingenuo. Una arquitectura multi-agente que sólo bate al monolítico de una pasada está comparándose contra el rival equivocado.

El criterio no se ha medido — y la medición más limpia es un costo. El impuesto agregado está confundido con la longitud de salida **en las dos direcciones** (presupuesto completo: el fragmentado escribe **1.39×** el monolítico y puntúa más; presupuesto repartido: **0.43×** y puntúa menos), y truncar a un presupuesto de lectura fijo **T** cambia el signo del impuesto con **T** (de **-15.66 %** en **T=40** a **+7.69 %** en **T=120**, **T13**). El instrumento por posición de primera mención (**T13b**, **SWIP-0001**), corregido para que una clave omitida se impute al final del texto (peor caso), tampoco decide: desplazamiento **+0.0036** de la propia longitud, **IC 95 % [-0.0948, +0.0994]** — un nulo con un intervalo $\sim 5\times$ más ancho que el umbral — y las dos mitades del corpus disienten en signo, así que el instrumento **se niega** (la imputación del peor caso además re-acopla la métrica a la longitud: $\rho = -0.452$, sobre 423 claves de las que el fragmentado omite **217** frente a **82** del monolítico). La medición que sí sobrevive es económica: con presupuesto igualado, fragmentar **cuesta en las cinco familias** (sobre las 95 celdas del criterio, **T09R**: medias **+6.2 %** a **+22.1 %**, medianas **+4.3 %** a **+29.4 %**; agregado **+11.17 %**, **IC 95 % [+4.80, +17.34]**, **T09R**), y la política por clase medida es **no fragmentar** (**+0.0 %** frente a **+14.8 %** fragmentando siempre). Es el resultado que esta sección reporta primero, porque es el que los datos sostienen.

15.3 Lo que se midió, y lo que se retiró

El impuesto de coherencia: criterio NO CUMPLIDO. `table_summary`, $\rho = 3.5$, $N = 2$, $k = 1$, sobre 16 prompts reservados:

Impuesto de coherencia	+2.30 %
IC 95 % (agrupado por prompt)	[-2.05 %, +7.49 %]
Criterio	cota superior por debajo del 5 %
Veredicto	NO CUMPLIDO — corto por 2.49 puntos en la cota superior
Efecto del prompt mediano	exactamente 0.00 %
Prompts en cero o por debajo	11 de 16 (6 negativos, 5 exactamente cero)
Control, $N = 8$, $k = 1$ (obligado a fallar)	+16.23 % , IC [+11.33 %, +20.28 %] — se comportó

La estimación puntual pasa el umbral y el intervalo no. El criterio se escribió contra la cota superior exactamente para este caso, y no se reescribe ahora que el caso se ha presentado.

La distribución es el resultado más informativo, y la media es el estadístico equivocado para ella. El prompt mediano no pierde nada, once de dieciséis quedan en cero o por debajo, y la media no sólo está tirada por los prompts positivos: está *fabricada* por dos de ellos, que suman el **140 %** del total; quitándolos, la media de los catorce restantes es **-1.06 %**. La descripción honesta no es «fragmentar cuesta 2.3 %» sino **en 11 de 16 prompts de resumen de tablas, partir el trabajo en dos fue gratis, y en dos de ellos fue caro.**

Dos retiradas de la v1.4 que siguen en pie. La curva de impuesto decreciente en p está retirada: el eje p nunca se movió, porque 13 de 96 celdas quedaban por encima de su piso de empaquetado, y la métrica de coherencia no era neutral respecto del brazo (+46.7 % de impuesto aparente sobre texto que no cambió). Y la afirmación de fiabilidad del mapa de confianza está retirada: $r = -0.030$ sobre 597 unidades contra un juez saturado, y después razones de momios comunes de 3.47, 0.26 y 1.24 contra clave de respuesta. §10.5.4 explica por qué M1 no levanta esa retirada.

15.4 La curva L: el instrumento funciona, el corpus no

Se construyó y se corrió un experimento para medir la curva de calidad frente a L — el parámetro que §5 vuelve central. **No pudo responder la pregunta, y la razón es informativa.**

El corpus se generó con 72 documentos derivados del producto de $L \in \{5, 10, 20, 40\}$ y $N \in \{2, 4, 8\}$, con 12 documentos por tamaño. El brazo monolítico responde **1 de 72** preguntas globales, incluso con el material más pequeño. Eso no es un resultado sobre fragmentación: es un piso. Con la línea base en el suelo no hay diferencia que medir.

La verificación del instrumento fue lo que salvó la corrida. Antes de concluir nada se comprobó que la calificación fuera correcta: una respuesta perfecta construida a mano puntúa 100 % en los 72 documentos. El calificador no es el problema. Después se sondearon cinco familias de modelo sobre el mismo material: **4 de 5 hacen las búsquedas locales a ~0.84**, y las preguntas globales salen **4 de 60** en todas ellas. Una hipótesis de que el envoltorio de contrato estaba estrangulando la respuesta global se probó con un A/B y **quedó refutada por su propia medición**: 4/60 en ambos sentidos, delta 0.000.

Conclusión, y es de diseño de corpus y no de arquitectura: el corpus necesita una pregunta *global* que este parque de modelos pueda responder sobre material pequeño. Hasta tenerla, la curva L no es medible, y la mitad final del corpus —que sigue sin usarse— **no debe correrse**, porque gastar la partición reservada contra un instrumento que no discrimina destruye la única reserva que queda.

Actualización de §15.4 (corpus reconstruido y admitido; la primera corrida de la curva fue inválida y se retiró). El corpus se reconstruyó con preguntas globales de aridez ≤ 3 — prompts/lcurve_v2.json — y pasó su compuerta de admisión: el brazo monolítico despeja el piso de 0.50 con llama3.2:3b al **61.1 %** (azar **0.182**; las otras cuatro familias no lo despejan). La primera corrida fragmentada sobre las celdas declaradas (N, L) **se retiró**: cada fragmento copiaba sus filas verbatim, la concatenación reconstruía el documento original y el modelo respondía el mismo texto que el monolítico — las 48 celdas coincidieron con su contraparte monolítica una a una (48/48), así que la corrida no medía nada de la fragmentación, y L quedaba confundido con el tamaño del documento. La corrida rediseñada (cada fragmento *responde* los valores por fila y su propia suma parcial; el ensamblador combina de forma determinista, run_lcurve_v2.py) sí mide algo de la fragmentación, pero **su ensamblador computa sólo 3 de las 6 formas de pregunta global**: pair_sum, triple_argmax y pair_warehouse —32 de las 72 globales de dev— se califican como fallo por construcción.

Sobre las celdas de la corrida el ensamblador cubre 79 de 144 globales (54.9 %), y ése es el techo del brazo fragmentado: en L=5 y L=10 lo alcanza exactamente. Comparado contra el monolítico del mismo documento, pregunta por pregunta (T08R2):

L	computables	fragmentado (computables)	monolítico (computables)	fragmentado (todas)	monolítico (todas)
5	23/36	100.0 %	73.9 %	63.9 %	77.8 %
10	20/36	100.0 %	75.0 %	55.6 %	69.4 %
20	19/36	73.7 %	63.2 %	38.9 %	58.3 %
40	17/36	52.9 %	47.1 %	25.0 %	44.4 %

Sobre las preguntas que el ensamblador sabe computar, el fragmentado **supera** al monolítico en todos los L: diferencia pareada **+18.9** puntos, IC 95 % **[+1.4, +37.5]**. Sobre todas las globales la diferencia es **-16.7** puntos **[-27.8, -4.6]**, pero esa cifra mide la cobertura del ensamblador y no la fragmentación, así que el veredicto de arquitectura **se niega**. Dos advertencias sobre la ventaja: parte de ella es de diseño —el fragmentado agrega con código exacto y el monolítico hace su propia aritmética, que es justo lo que el protocolo propone—, y la muestra es pequeña (79 preguntas, una familia). Tres límites más: la pendiente en L no está medida (cada L usa su propia banda de tamaños; en los 15 documentos con dos o más L hay 11 empates, 3 a favor del L menor y 1 del mayor); la fidelidad de valor tampoco (el registro cuenta filas respondidas, no correctas); y las sumas parciales no son verificables sin las respuestas crudas. `run_lcurve_v3.py` implementa las seis formas, corre todos los L sobre los mismos documentos y guarda la fidelidad por fila y las respuestas crudas. Con la v2 sola, esta sección **no afirma paridad, costo ni victoria** en la curva-L.

La corrida v3 existe (`data/lcurve_v3_runs.jsonl`): ensamblador con las seis formas, L = 5, 10, 20 y 40 sobre los mismos 8 documentos de 80 y 160 filas, fidelidad por fila y respuestas crudas; el monolítico se re-corrió en las mismas condiciones y salió idéntico, byte a byte, al de la admisión. Resultado (T08R3):

L	fragmentado	monolítico	fidelidad de extracción
5	24/24 = 100.0 %	12/24 = 50.0 %	960/960
10	24/24 = 100.0 %	12/24 = 50.0 %	958/960
20	24/24 = 100.0 %	12/24 = 50.0 %	960/960
40	21/24 = 87.5 %	12/24 = 50.0 %	923/960

Diferencia pareada **+46.9** puntos, IC 95 % **[+33.3, +59.4]**. Dentro de documento la pendiente en L es plana hasta L=20 y cae en L=40: 2 documentos empeoran, ninguno mejora y 6 empatan. Las respuestas crudas son auditables: los fragmentos extraen casi sin error, pero suman bien su propio bloque sólo 1 vez de 360.

Y el control que separa los dos efectos ya corrió (T08R4): la misma extracción sobre el documento entero (N=1), con el mismo lector y el mismo ensamblador, sobre los mismos 8 documentos. En 3 de ellos el modelo contestó sin numerar y se leyó por posición, re-calificando desde las respuestas crudas.

filas	monolítico	N=1 + código	fidelidad N=1	fragmentos + código	efecto de agregar con código	efecto de partir
80	58.3 %	100.0 %	314/320	100.0 %	+41.7	+0.0
160	41.7 %	16.7 %	158/640	93.8 %	-25.0	+77.1

Sobre los 8 documentos, el efecto de partir es **+38.5** puntos, IC 95 % **[+12.5, +68.8]**, y el de agregar con código **+8.3** **[-16.7, +33.3]**. Pero el promedio esconde lo importante: **son dos mecanismos distintos, y cada uno domina en un régimen**. Con 80 filas, extraer el

documento entero en una sola llamada ya es casi perfecto, y toda la ganancia viene de sacarle la aritmética al modelo; partir no añade nada. Con 160 filas, la extracción en una sola llamada colapsa —el modelo pierde el formato y entra en bucle: 1.140 líneas con apenas un centenar de valores distintos— y sólo partir la mantiene cerca del 100 %. Es la primera medición directa de la tesis de §7.5 según la cual el tamaño de trabajador que basta depende del presupuesto que se le pide: un nodo de 3B extrae bien hasta cierto tamaño de tarea y se desploma por encima, y fragmentar es lo que mantiene cada tarea por debajo de ese umbral.

Límites, declarados. Son 8 documentos, una familia y dos tamaños, así que el umbral está entre 80 y 160 filas y no se ha localizado. Lo que colapsa puede ser la longitud de la entrada, la de la salida (321 respuestas por llamada) o ambas; este diseño no las separa. El calificador del monolítico es tolerante, lo que juega a su favor. Y la conclusión no depende de la lectura por posición: sin ella, el efecto de partir es mayor (+42.7).

15.5 Composición: el tamaño de muestra que hace falta

La celda declarada del experimento de composición dio media **-0.35**, con desviación estándar entre grupos de **20.36** y error estándar de **4.16**, para un IC de **[-8.49, +7.80]** contra un umbral de 5.0. El intervalo contiene el umbral y el cero: no decide nada.

El cálculo de tamaño de muestra, contrastado contra el error estándar medido, dice **60 prompts como mínimo y 72 con margen**. Y hay un resultado negativo útil en el camino: **repetir no compra nada**, porque el *pipeline* es determinista a temperatura 0. Más corridas del mismo prompt no reducen la varianza entre prompts, que es la que domina.

Dos mediciones de composición sí son concluyentes y sostienen decisiones de esta versión. La imposición mecánica de `term_once` en el ensamblador subió el cumplimiento de **6/24 a 18/24**, por encima del monolítico en 13/24 — es la evidencia de que resolver la restricción en el sistema y no en el prompt funciona, y es la línea base que M2 debe batir (§11.3). Y `no_repeated_ngram` se quedó en **4/12 frente a 11/12** del monolítico bajo toda política de asignación de contexto probada — es la clase irreducible, y §6.4 explica por qué ninguna asignación puede alcanzarla.

15.6 Qué predice cada modelo, y cómo se lo mata

Los cinco modelos de esta versión son falsables. Enunciarlo aquí es lo que separa una versión mayor de un manifiesto, y es la condición bajo la cual este documento puede publicarse.

M1 — Alineamiento con sustitución semántica aprendida (§10.5). *Predice:* un mapa construido por alineamiento con costo aprendido separa correcto de incorrecto mejor que uno construido por conteo de coincidencias, y mejor que un escalar por respuesta, **en régimen no saturado**. *Se mata si:* con el instrumento correcto y en un régimen donde los modelos genuinamente discrepan, el acuerdo por unidad no supera al azar. Entonces la localización no crea señal y el mapa pierde su afirmación de fiabilidad, aunque conserve su capacidad de reportar divergencia.

M2 — Cardinalidad resuelta en la representación (§11.3). *Predice:* asignar la unicidad en el plan hace `no_repeated_ngram` y `term_once` inviolables por construcción, no sólo detectables. La tasa de violación debe caer a cero, no mejorar. *Se mata si:* la asignación en el plan no reduce las violaciones por debajo de 18/24, que es lo que ya logra la imposición mecánica.

M3 — Compuerta de triage por nivel (§10.7). *Predice:* el radio de daño de un fragmento defectuoso queda contenido en ese fragmento. Un incidente del tipo «42 de 60 paquetes contaminados» se vuelve imposible. *Se mata si:* la compuerta rechaza tanto trabajo legítimo que el costo de reintento supera el daño que evita.

M4 — ρ como tasa-distorsión indirecta con segundo eje δ (§6.7-6.8). *Predice:* a ρ constante, variar la densidad de dependencias del corte mueve la distorsión. Y existe una región donde ninguna ρ alcanza una distorsión aceptable. *Se mata si:* la distorsión resulta ser función sólo de ρ , con δ sin efecto medible. Entonces el segundo eje es decoración y el encuadre no aporta sobre la derivación empírica de §6.5.

M5 — Sobre-despacho con umbral, y k separado en tres (§12.3-12.4). *Predice:* separar disponibilidad, verificación y redundancia epistémica permite bajar el costo total manteniendo las tres garantías, porque hoy un solo parámetro paga por las tres al precio de la más cara. *Se mata si:* las tres resultan estar tan acopladas en la práctica que separarlas no ahorra nada.

Estado tras la campaña de referencia (§15.9).

Modelo	Veredicto	Sobre qué
M1	sin instrumento	el alineador semántico está construido y verificado en banco (T10), pero el corpus real no produjo el régimen no saturado que M1 necesita (T10R, bloqueado)
M2	sobrevive, corregido	23/35 por obediencia del nodo; 35/35 con la imposición mecánica. La frase «cero por construcción» se retira (§11.3)
M3	sobrevive	los paquetes contaminados no pasan la compuerta; el incidente 42/60 no se reproduce (T02R)
M4	falsado	en dos instrumentos independientes y con un experimento controlado a ρ constante (T04R, T07R)
M5	sin medir	sigue siendo una separación conceptual; nada en esta campaña la toca

El orden en que conviene atacarlos, por costo creciente y por lo que cada uno desbloquea:

1. **M5** no necesita medición: es una separación conceptual de un parámetro que ya existe. Se implementa y se observa.
2. **M3** es barato y su predicción es binaria: el incidente ocurre o no ocurre.
3. **M2** se mide sobre el corpus de composición que ya existe, contra una línea base ya medida (18/24).
4. **M4** necesita el corpus con dificultad calibrada del que depende toda la agenda experimental actual — el mismo que §15.4 dice que falta.
5. **M1** necesita construir el alineador, y es el más caro. También es el que produce la propiedad más distintiva.

15.7 Una nota sobre el instrumento, que es el hallazgo metodológico

Este proyecto tiene un defecto recurrente, y nombrarlo vale más que cualquiera de las mediciones individuales:

Una comprobación que afirma más de lo que midió.

Un agregado se reporta sin preguntar de dónde viene. Apareció cuatro veces en un solo día de trabajo, y **cada vez dentro de código escrito para atrapar la aparición anterior**. Los casos: un veredicto que leía una clave que la función de bootstrap no devolvía, y que por tanto habría dicho «no cumplido» sobre cualquier dato; una sonda que se degradó silenciosamente a una familia de modelo y concluyó sobre cinco; un diagnóstico de dificultad derivado de un solo modelo y presentado como propiedad del corpus; y un umbral que se disparaba por el efecto de una sola celda.

La defensa que funcionó no fue más cuidado. Fue **hacer que la comprobación se niegue**: derivar la lista de familias del propio lanzador y rehusar por debajo de dos; retirar una conclusión agrupada si quitar el mayor contribuyente la deshace; cruzar el plan de tamaño de

muestra contra el error estándar medido y rehusar si difieren más de un 25 %. Un instrumento que puede negarse es más valioso que uno que acierta más a menudo.

Cuatro umbrales de este documento existen por esa razón y se declaran: un mínimo de **20 grupos para emitir veredicto**; un **piso de línea base de 0.20** —añadido *después* de ver los datos y etiquetado como tal en el preregistro, porque una enmienda declarada vale y una silenciosa no—; y las dos tolerancias de control del párrafo anterior.

15.9 La campaña de referencia: 473 corridas sobre cinco familias

Todo lo anterior de §15 se midió con el arnés del proyecto sobre el corpus del proyecto. Esta sección reporta algo distinto: una **implementación de referencia levantada por separado** (swarmbly_ref/), corrida contra **cinco familias de modelos locales** —llama3.2:3b, qwen2.5:3b, gemma2:2b, phi3.5:3.8b, granite3.1-dense:2b— sobre un corpus de 22 tareas, y un **arnés de falsación** (swarmbly_validation/) que juzga los cinco modelos contra el registro resultante. El registro es un JSONL de **473 corridas** con digest anclado; los tres artefactos se publican con este documento y cada cifra de abajo se recomputa ejecutándolos.

Lo que hay que decir primero es lo que murió.

Prueba	Veredicto	Qué mide
T0R	✓	el router decide descomponible/no contra la verdad del corpus: 22/22
T02R	✓	M3: los paquetes contaminados no pasan la compuerta
T03R	✓	M2: 23/35 por obediencia del nodo, 35/35 tras la imposición mecánica
T04R	✗	M4: Spearman(δ , impuesto) = -0.15 sobre 127 celdas intra-categoría
T05R	✓	la contabilidad de ρ predice el tamaño de paquete con 7.5 % de error
T06R	✓	el calificador puntúa 21/21 la respuesta perfecta; 100 de 105 monolíticos despejan el piso
T07R	✗	M4, experimento controlado: δ sube en 32/32 pares y el impuesto empeora sólo en 15/32
T08R	✗	curva-L: la banda predicha (factor 3-6×) no aparece
T08R2	⊙	curva-L del corpus admitido: el ensamblador cubre el 54.9 % de las globales, así que el veredicto se niega; sobre las computables, +18.9 puntos a favor del fragmentado
T08R3	✓	curva-L v3: fragmentos + código +46.9 puntos sobre el monolítico; mezcla partir y agregar con código
T08R4	✓	control N=1: partir +38.5 puntos; con 80 filas la ganancia es de agregar con código, con 160 filas es de partir
T09R	⊙	criterio de abandono: rehusado , confundido con la longitud de salida
T10R	■	M1: sin régimen no saturado en este corpus
T0RR / T0RR2	✗	enrutabilidad por celda: AUC 0.57 y 0.57
T0LR	✓	L* varía por familia — §5.7 medido por primera vez
T13	⊙	el impuesto truncado cambia de signo con el presupuesto de lectura T — el acoplamiento con la longitud es estructural y el instrumento de truncado se niega
T13b	⊙	impuesto por posición, corregido por omisiones (SWIP-0001): desplazamiento +0.0036 [-0.0948, +0.0994] , mitades discordantes — el criterio no se ha medido

M4 queda falsado, y el experimento que lo mata es el que la estrategia declaró decisivo. T07R corta el **mismo prompt** dos veces con el **mismo L**: una vez en interfaces de acoplamiento débil (P9) y otra maximizando el acoplamiento que cruza el corte. δ sube en los 32 pares y ρ queda igual dentro de un 5 %, de modo que el único eje que se movió es δ . M4 predice que el corte fuerte empeora la distorsión **en todos** los pares; empeora en **19 de 32**, que es indistinguible de una moneda. La medición observacional converge: Spearman(δ ,

impuesto) = -0.15 sobre 127 celdas intra-categoría, con el signo contrario al predicho. **El segundo eje no es decoración conceptual: es una predicción que se comprobó y no se cumplió.**

El router por celda queda refutado, y esto cambia el diseño, no sólo el marcador. Un clasificador con validación dejando-una-tarea-fuera sobre 80 celdas operativas alcanza AUC 0.57 con δ y reputación y 0.61 sin ninguno de los dos. Añadir sondas de capacidad pre-despacho lo lleva a 0.57. Lo que sí separa es la **clase de nodo**: el impuesto medio por familia va de $+8.4\%$ a $+26.5\%$, y una política que enruta por clase obtiene $+0.0\%$ frente a $+14.8\%$ fragmentando siempre. La consecuencia está escrita en §10.1: el router decide *si* fragmentar, el perfil del nodo decide *quién*, y nadie predice el costo de una celda concreta.

M2 sobrevive con una corrección que vale más que la confirmación. Los nodos reales no obedecen `unique_here`: 6 de 35 términos aparecen exactamente una vez, con 29 repeticiones. Pero **0 omisiones** — y la imposición mecánica del ensamblador lleva el resultado a 35/35. El modo de fallo que la asignación en el plan elimina es el irreparable; el reparable lo sigue eliminando el ensamblador. §11.3 reescribe M2 en esos términos y retira «cero por construcción».

El criterio de abandono no se ha cumplido ni incumplido: no se ha medido. El agregado da $+11.17\%$ con IC 95 % [$+4.80$, $+17.34$] sobre 95 celdas, lo que parecería resolver el criterio a favor. No se puede afirmar, porque el brazo fragmentado recibió más presupuesto de salida que el monolítico: cada fragmento heredaba el `max_tokens` completo del prompt entero, de modo que un plan de N fragmentos disponía de N veces la salida. Tres estadísticos independientes confirman que eso basta para explicar la ventaja: el estadístico sin denominador da **$+0.595$** , el suelo del artefacto —el mismo cálculo con el impuesto permutado— da sólo **$+0.125$** , y la prueba directa dentro de cada tarea da mediana **$+0.571$** , positiva en 16 de 18 tareas.

Estratificando por longitud, el agregado se deshace donde debe:

Estrato	n	impuesto	IC 95 %	¿cumple el criterio?
todas las celdas	123	-32.5%	[-54.7 , -12.0]	sí
longitud comparable (0.8 – $1.25\times$)	31	-21.3%	[-42.0 , -1.0]	sí
el fragmentado NO escribe más ($\leq 1.0\times$)	41	-6.3%	[-22.3, $+14.1$]	no
el fragmentado escribe más ($> 1.25\times$)	67	-54.4%	[-77.1 , -24.6]	sí

La fila que importa es la tercera: cuando el brazo fragmentado **no** escribe más, la ventaja cae a -6.3% y el intervalo cruza el cero. Es una estratificación posterior y no un experimento, así que no resuelve nada por sí sola — pero es exactamente el patrón que el confundido predice, y por eso el veredicto es **rehusar** y no «cumplido con matices».

La causa se localizó en el código, no se conjeturó: `benchmark.py` daba a cada fragmento el `max_out_tokens` de la tarea completa. La corrección está aplicada y expuesta como una sola orden —`run_benchmark.py --matched`—, que reparte ese presupuesto entre los fragmentos y registra las celdas con sufijo propio para que las dos mediciones convivan. Los campos `output_words` y `length_ratio` son ahora campos de primera clase del registro, de modo que un análisis futuro no pueda repetir el error en silencio.

Lo que esta campaña no toca. T07R, T04R, T05R, T06R, T02R, T0R, T0LR y T10R comparan brazos con presupuesto equivalente o no dependen del impuesto agregado, así que el confundido no los alcanza. Los veredictos sobre M2, M3 y M4 se sostienen enteros.

Y una quinta aparición del defecto de §15.7, esta vez en el análisis de esta misma sección. Explorando los datos apareció una interacción atractiva: los nodos débiles parecían

ganar con la fragmentación y los fuertes perder. Antes de confirmarla se escribió un preregistro que declaraba el defecto sospechado —impuesto = 1 - frag/mono lleva mono en el denominador, así que correlacionarlo con el mono de la misma celda produce asociación *por construcción*— y el diseño que lo rompe: estimar la fuerza del nodo con sus resultados en las **demás** tareas. Con el predictor independiente, $\rho = -0.078$ ($p = 0.192$); y una simulación donde los dos scores se sortean independientes devuelve **+0.684**, frente al **+0.675** observado con el estadístico acoplado. La asociación entera la fabrica la fórmula. **El hallazgo se retiró**, y el test que lo mató se publica con el arnés (T12) para que la retirada sea reproducible en lugar de una nota al pie.

15.8 Métricas

Métrica	Definición	Objetivo	Estado
Impuesto de coherencia	Δ fracción de oraciones sin costura vs monolítico	cota superior del IC 95 % por debajo del 5 %	NO CUMPLIDO en el corpus de tablas (§15.3); REHUSADO en la campaña de referencia por confundido de longitud (§15.9)
ρ operativo	Tokens de entrada por token de prompt	distancia a (L+2F)/L	Re-derivado en §6.5; el objetivo absoluto «<2.0» de la v1.4 queda retirado por no ser alcanzable ni significativo
Curva de calidad en L	Calidad vs tamaño de fragmento	existencia de L_min y de una banda	medida por familia : L* varía entre familias, pero la banda ancha predicha no aparece (§15.9, L20)
Efecto de δ a ρ constante	Distorsión vs densidad de dependencias	efecto medible	medido y nulo : δ sube en 32/32 pares a ρ igual y el impuesto empeora en 15/32 (§15.9)
Violaciones de cardinalidad	sobre-compresión y sobre-expansión, por separado	sobre-compresión cero por construcción; sobre-expansión cero tras imposición	sobre-compresión 0/35; sobre-expansión 29/35 en crudo y 0/35 tras imposición (§15.9)
Contención del triage	radio de daño de un fragmento defectuoso	un fragmento	contenido : los 3 paquetes contaminados se rechazan (§15.9)
Aceleración efectiva	vs monolítico, mismo modelo	$>1.5\times$	no medido
Aceleración vs línea base honesta	vs decodificación especulativa y vs self-consistency	reportada aunque sea <1	no medido
p95 de latencia bajo rotación	$p=0.10$, $N=8$	$<2\times$ del caso sin fallos	no medido
Detección de nodos deshonestos	adversarios inyectados atrapados	$>95\%$	no medido
Sobrecarga de verificación	costo extra por fragmento	$<5\%$	no medido
SCI	gCO ₂ e por unidad funcional	publicado y comparado	no medido

16. Limitaciones y resultados negativos

Enunciadas llanamente y con extensión. Una especificación cuyos modos de fallo están documentados puede ser mejorada por gente que no la escribió; una que los esconde sólo puede ser descubierta como equivocada.

L1 — La pérdida de calidad por generación independiente es teórica, no incidental. La generación paralela supone independencia condicional, y la calidad se degrada en proporción a la fuerza de las dependencias reales [20]. A igual presupuesto de cómputo, la descomposición es un canal con pérdida [21]. Esto no se resuelve con ingeniería de prompts; sólo se rodea enrutando, que es por lo que existe §10.1. **[v0.3]** Y §6.8 le da nombre y eje: es δ .

L2 — La coherencia es el eje que se rompe, y los agregados lo esconden. SoT mejora relevancia y diversidad mientras degrada coherencia e inmersión [12]. El texto fusionado jerárquicamente exhibe ocho clases recurrentes de error de coherencia [54].

L3 — El ensamblador opera en un régimen conocido por ser poco fiable. La fiabilidad del modelo se degrada con la longitud de entrada en todos los modelos probados; un distractor daña y cuatro se componen; y los modelos rinden *mejor* sobre contextos barajados que sobre contextos lógicamente coherentes [88]. El sesgo posicional añade una curva en U en que los fragmentos del medio quedan sistemáticamente infra-ponderados [89].

L4 — El límite de contexto se reubica, no se elimina. Se traslada al cliente, que es el nodo más débil del sistema.

L5 — Un orquestador de 8B puede ser inadecuado. Es H3, y un resultado negativo exigiría un requisito de cliente mayor, lo que estrecha la base de usuarios.

L6 — Sin resistencia fuerte a Sybil. Véase §13.6.

L7 — La fragmentación no es cifrado. Véase §13.1.

L8 — El beneficio ambiental no está probado. Véase §14.3.

L9 — La analogía genómica es vocabulario más instrumentos, no una herencia. [v0.3] Esta limitación cambia de forma respecto de la v1.4. Ya no es «es sólo una convención de nombres»: §3 da un criterio, y cinco homologías lo pasan trayendo desigualdades y números concretos. Pero sigue siendo cierto que **ningún algoritmo de ensamblaje genómico corre dentro de Swarmby**, y un revisor de bioinformática debe leer §5, §6.4 y §11 como transferencias de *instrumento*, no de implementación.

L10 — El riesgo dominante no es técnico. La computación voluntaria lleva dos décadas en declive: los primeros proyectos atrajeron del orden de un millón de voluntarios, y la base ha encogido a unos doscientos mil [47]. Swarmby debe explicar qué hace distinto su bucle de incentivos, y «créditos de red» no es por sí solo una respuesta. Es, en mi evaluación, más probable que esto termine el proyecto que cualquier limitación algorítmica.

L11 — Los sistemas multi-agente fallan de maneras caracterizadas. La taxonomía MAST atribuye 47.9 % de los fallos al diseño del sistema, 32.2 % a desalineación entre agentes y 20.0 % a verificación de tareas, con repetición de pasos (15.7 %) y desobediencia de especificación (11.8 %) como los modos individuales más comunes [90]. Swarmby es un sistema multi-agente y debe esperar esa distribución. **[v0.3]** §2.4 mapea las tres categorías contra las tres áreas que esta versión aborda; esa coincidencia es una convergencia, no una inmunidad.

L12 — Un enjambre de confianza reubica la confianza; no la elimina. Véase §13.5.

L13 — La afirmación de fiabilidad del mapa de confianza sigue retirada. Medida contra un juez de clase par, el acuerdo por unidad no predijo la aceptabilidad juzgada ($r = -0.030$ sobre 597 unidades), y $k > 1$ costó 17 a 20 puntos de coherencia en la misma corrida. El experimento con clave de respuesta se corrió tres veces y devolvió razones de momios de 3.47, 0.26 y 1.24. El mecanismo sigue divulgado y especificado, y reportar *dónde* divergieron las réplicas sigue siendo una capacidad real; afirmar que la convergencia indica corrección,

no. **[v0.3] M1** propone un instrumento distinto con su propia condición de muerte (§15.6); proponerlo no levanta esta retirada, y nada en esta versión debe leerse como si lo hiciera.

L14 — El segundo instrumento de coherencia no funciona sobre respuestas cortas. La rejilla de entidades devolvió líneas base monolíticas entre 0.000 y 0.114. Este documento tiene un instrumento de coherencia que funciona, no dos.

[v0.3] L15 — La curva L no es medible con el corpus actual. El brazo monolítico responde 1 de 72 preguntas globales. El calificador está verificado y las cinco familias hacen búsquedas locales a ~ 0.84 , así que el defecto está en el diseño del corpus y no en el instrumento ni en los modelos — pero el efecto práctico es que L , el parámetro que §5 vuelve central, **no tiene aún curva medida**. La partición reservada no debe gastarse hasta que exista una pregunta global responsable (§15.4).

[v0.3] L16 — $L_{\min}$ y la banda óptima están predichos, no medidos. §5.6 predice un piso duro y una banda ancha por analogía con el tamaño de dominio proteico, donde Pfam y SCOP difieren en casi el doble sobre el mismo material. Es una predicción con fundamento, y es una predicción. Cualquier L que este proyecto use hoy es una conjetura informada.

[v0.3] L17 — M2 filtra estructura. El campo `unique_here` entrega a un nodo información global sobre la sesión que la v1.4 no entregaba (§13.2). La corrección de cardinalidad y la descontextualización son antagonistas, igual que lo son la coherencia y la descontextualización, y por la misma razón.

[v0.3] L18 — El sobre-despacho sólo aplica a subtareas intercambiables. El ahorro de §12.3 es real y es estrecho. Si el contenido del fragmento j es necesario, no hay esquema de umbral que lo sustituya, y creer lo contrario es exactamente el error que hundió la hipótesis de los fountain codes.

[v0.3] L19 — La cota de tasa-distorsión existe y no es computable. §6.7 hereda la existencia de un piso y la monotonía cualitativa de la curva. No hereda un número. Un lector que se lleve de §6 la impresión de que existe un ρ óptimo calculable se habrá llevado más de lo que el documento dice.

L20 — La banda ancha de L está predicha y no aparece. §5.6 predice un piso $L_{\min}$ nítido y una banda óptima ancha, por analogía con el tamaño de dominio proteico. Con curvas de más de un punto por tarea, `longform` da una banda de factor **2.7** y `table_outturn` una de factor **1.0** — es decir, un solo L óptimo. La analogía acertó en que L es propiedad de la clase de nodo (TOLR mide L^* distinto por familia) y erró en la anchura de la banda. La ganancia media de usar el L^* de cada familia en vez de un L común es **+0.360** de calidad: real, medible y pequeña.

L21 — El impuesto medido depende del presupuesto de salida, y hasta igualarlo no hay veredicto de factibilidad. Es la limitación más importante de esta versión. Todo el material de §15.9 sobre M2, M3 y M4 se sostiene; la afirmación «fragmentar sale a cuenta» **no se ha medido**. La corrida con presupuesto igualado ya se hizo y da un costo, no una ventaja (T09R); pero ese agregado sigue confundido con la longitud de salida en las dos direcciones, así que el veredicto de factibilidad **se niega** (§15.9).

L22 — El defecto recurrente de §15.7 apareció por quinta vez, dentro del análisis que lo documenta. La defensa que funcionó no fue más cuidado: fue preregistrar el defecto sospechado y el diseño que lo rompe **antes** de correr el análisis. Con el estadístico acoplado y un umbral elegido a ojo, el hallazgo falso habría salido confirmado con números grandes. Un lector debe asumir que este documento contiene una sexta aparición todavía sin detectar, y el arnés publicado existe para que sea él quien la encuentre.

17. Declaración de arte previo

Nota sobre el estado. Los elementos E1-E18 fueron divulgados públicamente en la versión 1.4 de este documento, con fecha del 14 de agosto de 2026, y constituyen arte previo desde entonces. **Los elementos E19-E24 son nuevos en esta versión y constituyen arte previo desde la publicación de este documento**, no antes. El periodo en que se mantuvieron inéditos fue una decisión deliberada para no afirmar antes de medir, y su costo fue exactamente la prioridad que no protegía mientras tanto.

Los elementos se divulgan con la intención de que entren al dominio público a efectos de patentabilidad; el autor reserva el copyright del texto bajo CC BY 4.0 y licencia la implementación bajo AGPL-3.0-or-later.

E1. Un método de inferencia distribuida de modelos de lenguaje en que la unidad de distribución es una **subtarea semántica derivada de la petición**, despachada una vez por fragmento por sesión a nodos que ejecutan cada uno un modelo independiente completo. (§1.2, §8.2)

E2. Un **presupuesto de contexto** S como parámetro explícito del protocolo que gobierna conjuntamente la coherencia de ensamblaje, la verificabilidad del fragmento, la fuga de privacidad y la capacidad requerida del trabajador, con la razón de redundancia ρ como medida de costo reportada. (§6)

E3. Un **contrato global** Γ transmitido con cada fragmento como mecanismo de consistencia entre fragmentos, con la tabla de entidades como sistema de coordenadas de nombres compartido. (§9.2)

E4. Un **router con costo de decisión asimétrico** que puede negarse a fragmentar, calibrado con F_β con $\beta < 1$. (§10.1)

E5. Un **planificador de DAG de dependencias** que distingue tareas que requieren el *resultado* de una predecesora de las que requieren sólo su *enunciado*. (§10.2)

E6. Un **procedimiento de empaquetado** en que el contrato global nunca se elide para cumplir un presupuesto de contexto. (§10.3)

E7. Un **ensamblador de selección y empalme** en que múltiples fragmentos candidatos se resuelven por selección y no por síntesis, con puenteo generativo invocado sólo en una costura cuya similitud de frontera cae bajo un umbral calibrado. (§10.4)

E8. **Calibración empírica del umbral de costura** a partir de pares etiquetados bajo un objetivo asimétrico, re-derivada por modelo de *embedding*. (§10.6)

E9. Una **auditoría de coherencia** devuelta como parte de la respuesta del protocolo. (§10.8)

E10. **Enrutamiento por carriles de sensibilidad** — PUBLIC / SANITISABLE / SENSITIVE — como mecanismo de confidencialidad. (§13.4)

E11. Un **esquema de verificación de dos capas** que combina un compromiso sensible a la localidad sobre activaciones ligado a un perfil de nodo declarado, con auditoría pública muestreada indistinguible del trabajo real. (§13.3)

E12. **Despacho redundante preservador de diversidad:** las réplicas de un fragmento crítico se asignan a nodos de familias de modelo deliberadamente *distintas*. (§7.3c, §9.6)

E13. **Despacho especulativo con disparo al p95 por clase** con contabilidad de cancelaciones que distingue nodos lentos de nodos deshonestos. (§8.4, §9.6)

E14. Una **unidad de crédito no transferible, no preminada y caducable** ganada por procesamiento verificado de fragmentos. (§14.1)

E15. La **descomposición cobertura/conversión** $Q \approx V \cdot \prod_i C_i$ como instrumento de diseño para inferencia de enjambre. (§7.2, §7.3)

E16. **Consenso por alineamiento múltiple de réplicas generadas independientemente**, con puntuación de acuerdo por unidad y unidades por debajo del umbral expuestas como regiones de baja confianza. Divulgado como **mecanismo**: su correlación con la corrección se midió y no sobrevivió, así que no se reclama beneficio de fiabilidad. (§10.5, §15.3)

E17. Un **modelo de cobertura para ensamblaje semántico** en que el plan y el contrato previos a la generación sirven de secuencia de referencia, la pérdida de paquetes y no la colocación de muestras es el elemento estocástico, y el requisito de redundancia se deriva como $c \geq \ln(1/\epsilon)/(1-p)$. (§7.4)

E18. **Niveles dinámicos de privacidad con enjambres de confianza**: un clasificador de privacidad en el cliente que nunca sale de la máquina, guiando una decisión de enrutamiento en un eje ortogonal a la sensibilidad del contenido. (§13.5)

Elementos nuevos de esta versión — aún no divulgados públicamente.

E19. Un **criterio de admisibilidad de homologías** para transferir mecanismos entre dominios científicos, consistente en dos pruebas conjuntas: que la homología aporte un procedimiento, una desigualdad o un número aplicables (prueba del instrumento), y que aporte el enunciado de la patología que ocurre al violarse su condición (prueba del modo de fallo); con el descarte explícito de las transferencias que sólo pasan la primera. (§3)

E20. Una **regla de fragmentación por interfaz de acoplamiento débil**, en que el límite de fragmento se elige minimizando las dependencias que lo cruzan en lugar de dividir en partes de tamaño uniforme; con el tamaño L como propiedad declarada de la clase de nodo, el número de fragmentos derivado como $N = \lceil \text{material}/L \rceil$, y el flanco F definido como el máximo entre la unidad repetible más larga detectable y la dependencia más larga que cruza un límite — del que se deriva el presupuesto de contexto como $\rho \approx (L+2F)/L + H/(L \cdot s)$. (§5.4, §5.7, §6.5)

E21. Un **mapa de divergencia construido por alineamiento múltiple de respuestas con matriz de sustitución semántica aprendida empíricamente** a partir de pares de oraciones verificados como intercambiables, con extensión por consistencia mediante corroboración a través de terceras respuestas independientes, y emisión de un perfil específico de posición — en sustitución de la comparación por similitud de *embedding* contra umbral. (§10.5)

E22. Un **método de satisfacción de restricciones de cardinalidad global en el plan**: computar antes del despacho el conjunto de elementos que deben aparecer exactamente una vez en la salida final, asignar cada elemento a exactamente un fragmento como propiedad del plan, verificar la cardinalidad contra ese conjunto en el ensamblado, y registrar por separado la sobre-compresión y la sobre-expansión. (§11.3)

E23. Una **compuerta de triage por nivel topológico** en que un acarreo debe pasar un predicado mecánico antes de habilitar el nivel siguiente, un fragmento que no pasa se aísla y se reintenta en lugar de parchearse en contexto, uno que no pasa tras r reintentos se descarta y se regenera, y el consumo puede comenzar antes de que el nivel esté completo si el fragmento consumido está completo y verificado. (§10.7)

E24. La **separación del parámetro de redundancia en tres parámetros de propósito único** —disponibilidad, verificación y redundancia epistémica— con un mecanismo distinto asignado a cada uno (sobre-despacho con umbral y terminación temprana; *spot-checking* con credibilidad; réplicas de familias diversas para el mapa de divergencia), y con la condición normativa de que el sobre-despacho sólo es admisible entre subtareas genuinamente intercambiables. (§12.3, §12.4)

18. Conclusión

El conocimiento necesario para construir modelos de lenguaje es público. El capital necesario para operarlos no lo es, y esa asimetría —y no ningún secreto— es lo que concentra el control sobre una tecnología de propósito general. Mientras tanto, el hardware capaz de servir inferencia está ocioso en cientos de millones de hogares y oficinas, ya fabricado, ya consumiendo energía.

El obstáculo entre esos dos hechos es físico y específico: cuatro a cinco órdenes de magnitud entre la interconexión de centro de datos y los enlaces de consumo, que todo diseño existente de inferencia entre pares cruza en cada token generado. La afirmación estructural de este trabajo es que cruzarla **una vez por unidad de trabajo en lugar de una vez por token** es un régimen distinto y no una optimización del mismo.

Lo que esta versión añade a esa afirmación es una teoría de la unidad de trabajo. La v1.4 sabía que había que fragmentar y no sabía en qué. Esta versión lo deriva: el fragmento es un dominio de tarea, se corta donde el acoplamiento es débil, su tamaño es una propiedad medida de la clase de nodo, su solapamiento es el máximo entre dos cantidades observables, y de ahí sale el presupuesto de contexto en lugar de barrerse. El ahorro medido de esa sola derivación es del 29 % del cómputo de la red, y no relaja ninguna garantía porque la teoría dice exactamente qué medir.

Cinco modelos falsables acompañan esa teoría, y los cinco vienen con su condición de muerte escrita antes de la medición. Dos hipótesis entraron a la revisión y no sobrevivieron — los *mate pairs* y los *fountain codes* — y el diagnóstico de por qué cayeron resultó más útil que las homologías habrían sido: **una transferencia que trae un mecanismo sin su patología está importando la conclusión sin el trabajo.** Ése es el hallazgo metodológico de esta versión, y es lo que la sección 3 convierte en criterio.

El caso en contra es igual de específico y está en la sección 16. La generación independiente pierde calidad por razones teóricas y no incidentales. El modelo del lado del cliente del que depende el diseño está en el extremo débil de la capacidad de planificación medida. La computación voluntaria lleva veinte años contrayéndose. Y varias mediciones han ido en contra: el criterio de coherencia no se cumplió, la curva del impuesto en ρ y la afirmación de fiabilidad del mapa de confianza siguen retiradas, M4 queda falsado, y con presupuesto de salida igualado fragmentar **cuesta** en las cinco familias medidas.

Lo que sobrevive es específico y es suficiente para seguir. La curva $\mathcal{L}$ —el eje que esta versión vuelve central— ya se midió con un brazo fragmentado genuino, y mide la tesis del diseño de forma directa: **extraer con el modelo y agregar con código supera al modelo solo** (T08R3), y el control con el documento entero separa los dos mecanismos (T08R4). Con 80 filas la ganancia viene de sacarle la aritmética al modelo y partir no añade nada; con 160 filas la extracción en una sola llamada colapsa y sólo partir la sostiene. Un nodo pequeño extrae bien hasta cierto tamaño de tarea, y fragmentar es lo que mantiene cada tarea por debajo de ese umbral. El alcance es estrecho —pocos documentos, una familia, dos tamaños— y §15.4 lo

declara como tal. A eso se suman un control que falló como se le exigía, un instrumento de calificación verificado, un arnés que se niega cuando su instrumento no puede decidir, y un conjunto de modelos que dicen por adelantado qué los mataría. Lo que falta por resolver es dónde está ese umbral y cómo se desplaza con la clase de nodo, y si existe un presupuesto de contexto que satisfaga coherencia, privacidad, verificabilidad y capacidad de trabajador a la vez, a un costo por debajo del valor de la capacidad agregada.

Este documento se publica ahora por la misma razón que antes lo retenía: **los modelos que propone eran falsables, y ya se han sometido a su prueba**. Los veredictos van dentro — uno falsado, uno corregido, uno en pie, dos sin medir— y la sección 15.6 dice cómo atacar los que faltan y cuánto cuesta cada uno. Con esta publicación, los elementos E19–E24 pasan a ser arte previo.

Si funciona, el resultado no es una manera más barata de comprar lo que ya se vende. Es capacidad de inferencia que crece con el número de personas que participan en lugar de con la cantidad de capital disponible para construir, sostenida bajo una licencia y una estructura de gobernanza diseñadas para que ninguna parte pueda cercarla. Vale la pena intentarlo aun con una probabilidad sustancial de fracaso, y vale la pena intentarlo en abierto, donde pueda comprobarse.

19. Referencias

Numeración. Los marcadores [1]–[90] conservan la numeración de la versión 1.4, de modo que una cita de aquel documento sigue apuntando a la misma fuente. Los marcadores [92]–[135] son nuevos en esta versión y corresponden a la revisión bibliográfica de los fundamentos de fragmentación; todos ellos fueron verificados contra la fuente en línea indicada, y cuando sólo se pudo confirmar el resumen se indica. Las entradas que arrastraban de la v1.4 un identificador sin confirmar se completaron contra la fuente para esta publicación. Las que siguen marcadas $\triangle$ tienen confirmación parcial y lo dicen, en lugar de rellenarse de memoria.

Inferencia y entrenamiento descentralizados

- [1] Borzunov, A., et al. (2023). Petals: Collaborative inference and fine-tuning of large models. *ACL 2023: System Demonstrations*. <https://arxiv.org/abs/2209.01188>
- [2] Borzunov, A., et al. (2023). Distributed inference and fine-tuning of large language models over the internet. *arXiv*. <https://arxiv.org/abs/2312.08361>
- [3] Petals project. (2023). *Petals project repository, release v2.2.0* [software].
- [4] Ryabinin, M., & Gusev, A. (2020). Towards crowdsourced training of large neural networks using decentralized mixture-of-experts. *NeurIPS*, 33, 3659–3672.
- [5] Ryabinin, M., Dettmers, T., Diskin, M., & Borzunov, A. (2023). SWARM parallelism. *ICML*, 29633–29654.
- [6] Bittensor. (s. f.). *Incentivizing intelligence*. <https://bittensor.com/academia>
- [7] Lui, E., & Sun, J. (2025). Bittensor protocol: The Bitcoin in decentralized artificial intelligence? A critical and empirical analysis. *arXiv*. <https://arxiv.org/abs/2507.02951>
- [8] Douillard, A., et al. (2023). DiLoCo. *arXiv*. <https://arxiv.org/abs/2311.08105>
- [9] Jaghouar, S., et al. (2024). OpenDiLoCo. *arXiv*. <https://arxiv.org/abs/2407.07852>

- [10] Jaghouar, S., et al. (2024). *INTELLECT-1 technical report*. <https://arxiv.org/abs/2412.01152>
- [11] Ramasinghe, S., et al. (2025). Subspace Networks: Scaling decentralized training with communication-efficient model parallelism. *NeurIPS*. <https://arxiv.org/abs/2506.01260>

Decodificación paralela y descomposición

- [12] Ning, X., Lin, Z., Zhou, Z., Wang, Z., Yang, H., & Wang, Y. (2024). Skeleton-of-thought. *ICLR*. <https://arxiv.org/abs/2307.15337>
- [13] Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. *ICML*. <https://arxiv.org/abs/2211.17192>
- [14] Cai, T., et al. (2024). Medusa. *arXiv*. <https://arxiv.org/abs/2401.10774>
- [15] Fu, Y., et al. (2024). Break the sequential dependency of LLM inference using lookahead decoding. *ICML*. <https://arxiv.org/abs/2402.02057>
- [16] Liu, M., et al. (2024). APAR. *arXiv*. <https://arxiv.org/abs/2401.06761>
- [17] Jin, T., et al. (2025). Learning to keep a promise (PASTA). *ICML*. <https://arxiv.org/abs/2502.11517>
- [18] Jin, S., Wu, Y., Zheng, H., Zhang, Q., & Lentz, M. (2024). Adaptive skeleton graph decoding. *arXiv*. <https://arxiv.org/abs/2402.12280>
- [19] Rodionov, G., et al. (2025). Hogwild! Inference. *NeurIPS*. <https://arxiv.org/abs/2504.06261>
- [20] Kang, W., Galim, K., Oh, S., et al. (2026). ParallelBench. *ICLR*. <https://arxiv.org/abs/2510.04767>
- [21] Tran, H., & Kiela, D. (2026). Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets. *arXiv*. <https://arxiv.org/abs/2604.02460>
- [51] Zhou, D., et al. (2023). Least-to-most prompting. *ICLR*. <https://arxiv.org/abs/2205.10625>
- [52] Jiang, Z., et al. (2024). LongRAG. *arXiv*. <https://arxiv.org/abs/2406.15319>

Red y hardware

- [22] NVIDIA. (s. f.). *NVIDIA H100 product documentation*.
- [23] NVIDIA. (s. f.). *NVIDIA Quantum-2 InfiniBand documentation*.
- [24] Sevilla, J. (2025). *How far can decentralized training over the internet scale?* Epoch AI.
- [25] Prime Intellect. (2025). *Planetary-scale inference: Building a distributed inference engine for the public internet* [entrada de blog]. <https://www.primeintellect.ai/blog/inference>

Ensamblaje de genomas

- [26] Lander, E. S., & Waterman, M. S. (1988). Genomic mapping by fingerprinting random clones. *Genomics*, 2(2), 231-239. [https://doi.org/10.1016/0888-7543\(88\)90007-9](https://doi.org/10.1016/0888-7543(88)90007-9)
- [27] Khadiev, K., & Safina, L. (2024). Quantum algorithms for the shortest common superstring and text assembling problems. *QIC*, 24(3-4), 267-294.
- [28] Ahmed, M., et al. (2015). A survey of genome sequence assembly techniques and algorithms using high-performance computing. *The Journal of Supercomputing*, 71(1), 293-339. <https://doi.org/10.1007/s11227-014-1297-4>
- [29] Pevzner, P. A., Tang, H., & Waterman, M. S. (2001). An Eulerian path approach to DNA fragment assembly. *PNAS*, 98(17), 9748-9753.

- [30] Nagarajan, N., & Pop, M. (2013). Sequence assembly demystified. *Nature Reviews Genetics*, 14, 157–167.
- [31] Kingsford, C., Schatz, M. C., & Pop, M. (2010). Assembly complexity of prokaryotic genomes using short reads. *BMC Bioinformatics*.
- [32] Chaisson, M. J. P., Wilson, R. K., & Eichler, E. E. (2015). Genetic variation and the de novo assembly of human genomes. *Nature Reviews Genetics*.

Sistemas multi-agente, selección y agregación

- [33] Yan, W. (2025). *Don't build multi-agents*. Cognition engineering blog.
- [38] Brown, B., et al. (2024). Large language monkeys. *arXiv*. <https://arxiv.org/abs/2407.21787>
- [39] Maryanskyy, A., Budnikov, D., & Kaliyev, A. T. (2026). When agents disagree. *arXiv*. <https://arxiv.org/abs/2603.20324>
- [40] Żywot, A., Chen, Y., Yuan, S., Søgaard, A., & de Rijke, M. (2026). Can small agents collaborate to beat a single large language model? *arXiv*. <https://arxiv.org/abs/2601.11327>
- [41] Wang, J., et al. (2025). Mixture-of-agents. *ICLR*. <https://arxiv.org/abs/2406.04692>
- [42] Chen, Y., Niu, G., Cheng, J., Han, B., & Sugiyama, M. (2025). When and why does multi-agent debate fail? *arXiv*. <https://arxiv.org/abs/2510.20963>
- [71] Zhang, G., et al. (2025). AgenTracer: Who is inducing failure in the LLM agentic systems? *arXiv*. <https://arxiv.org/abs/2509.03312>
- [90] Cemri, M., et al. (2025). Why do multi-agent LLM systems fail? *arXiv*. <https://arxiv.org/abs/2503.13657>

Modelos pequeños, planificación y enrutamiento

- [43] Belcak, P., et al. (2025). Small language models are the future of agentic AI. *arXiv*. <https://arxiv.org/abs/2506.02153>
- [44] Schepanowski, C., & Ling, C. (2025). On the limits of innate planning in large language models. *arXiv*. <https://arxiv.org/abs/2511.21591>
- [45] Valmeekam, K., et al. (2022). PlanBench. *arXiv*. <https://arxiv.org/abs/2206.10498>
- [46] Ong, I., et al. (2024). RouteLLM. *arXiv*. <https://arxiv.org/abs/2406.18665>

Embeddings y coherencia

- [34] Ethayarajh, K. (2019). How contextual are contextualized word representations? *EMNLP*. <https://arxiv.org/abs/1909.00512>
- [35] Steck, H., et al. (2024). Is cosine-similarity of embeddings really about similarity? *WWW '24 Companion*. <https://arxiv.org/abs/2403.05440>
- [36] Muennighoff, N., et al. (2023). MTEB. *EACL*. <https://arxiv.org/abs/2210.07316>
- [37] Sentence-Transformers. (s. f.). *Semantic similarity and paraphrase mining* [documentación].
- [53] Barzilay, R., & Lapata, M. (2008). Modeling local coherence: An entity-based approach. *Computational Linguistics*, 34(1).
- [54] Chang, Y., et al. (2024). BoookScore. *ICLR*. <https://arxiv.org/abs/2310.00785>
- [88] Chroma. (2025). *Context rot* [informe técnico].

[89] Liu, N. F., et al. (2023). Lost in the middle. *TACL*. <https://arxiv.org/abs/2307.03172>

Verificación, privacidad y seguridad

[50] Zhang, Y., Wang, S., Liu, X., Tan, S., Popa, R. A., & Moallemi, C. C. (2024). Proof of sampling. *arXiv*. <https://arxiv.org/abs/2405.00295>

[55] Sweeney, L. (2002). k-Anonymity. *IJUFKS*, 10(5), 557–570.

[56] Machanavajjhala, A., et al. (2007). ℓ -Diversity. *ACM TKDD*, 1(1).

[57] Narayanan, A., & Shmatikov, V. (2008). Robust de-anonymization of large sparse datasets. *IEEE S&P*.

[58] Narayanan, A., et al. (2012). On the feasibility of internet-scale author identification. *IEEE S&P*.

[59] Overdorf, R., & Greenstadt, R. (2016). Blogs, Twitter feeds, and Reddit comments: Cross-domain authorship attribution. *Proceedings on Privacy Enhancing Technologies*, 2016(3), 155–171. <https://doi.org/10.1515/popets-2016-0021>

[60] Belvisi, N. M. S., et al. (2020). Forensic authorship analysis of microblogging texts using n-grams and stylometric features. *IEEE 8th International Workshop on Biometrics and Forensics (IWBIF 2020)*. <https://arxiv.org/abs/2003.11545>

[61] Morris, J. X., et al. (2023). Text embeddings reveal (almost) as much as text. *EMNLP*. <https://arxiv.org/abs/2310.06816>

[62] Zhang, C., et al. (2024). Extracting prompts by inverting LLM outputs. *EMNLP*.

[63] Fan, M., Liu, Y., Wang, F., & Chen, C. (2026). What does the server see? *arXiv*. <https://arxiv.org/abs/2605.23158>

[64] Keller, M. (2020). MP-SPDZ. *ACM CCS*.

[65] Hao, M., et al. (2022). Iron: Private inference on transformers. *NeurIPS*.

[66] Lu, W., et al. (2025). BumbleBee. *NDSS*.

[67] Sun, H., Li, J., & Zhang, H. (2024). zkLLM. *ACM CCS*. <https://arxiv.org/abs/2404.16109>

[68] Ong, J. M., et al. (2025). TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference. *ICML*. <https://arxiv.org/abs/2501.16007>

[69] Wang, K., et al. (2025). VeriLLM: A lightweight framework for publicly verifiable decentralized inference. *arXiv*. <https://arxiv.org/abs/2509.24257>

[70] OWASP Foundation. (2025). *OWASP top 10 for LLM applications*.

[72] Zhu, J., et al. (2024). Confidential computing on NVIDIA Hopper GPUs: A performance benchmark study. *arXiv*. <https://arxiv.org/abs/2409.03992>

[73] Wang, W., et al. (2026). Benchmarking confidential GPU inference on NVIDIA H100 under Intel TDX. *arXiv*. <https://arxiv.org/abs/2607.19353>

[74] Lukas, N., et al. (2023). Analyzing leakage of personally identifiable information in language models. *IEEE S&P*.

[75] Çano, E., & Habernal, I. (2025). Differentially-private text generation degrades output language quality. *arXiv*. <https://arxiv.org/abs/2509.11176>

[76] Douceur, J. R. (2002). The Sybil attack. *IPTPS*. https://doi.org/10.1007/3-540-45748-8_24

[77] Kamvar, S. D., Schlosser, M. T., & Garcia-Molina, H. (2003). The EigenTrust algorithm. *WWW*.

Computación voluntaria

- [47] Anderson, D. P. (2019). BOINC: A platform for volunteer computing. *Journal of Grid Computing*. <https://arxiv.org/abs/1903.01699>
- [48] Anderson, D. P., & Fedak, G. (2006). The computational and storage potential of volunteer computing. *CCGrid*.
- [49] de Almeida, A. R. (2025). Idle consumer GPUs as a complement to enterprise hardware for LLM inference: Performance, cost and carbon analysis. *Proceedings of the 6th International Artificial Intelligence and Blockchain Conference (AIBC 2025)*. ACM. <https://doi.org/10.1145/3775043.3775047>  autoría sin verificar.

Licencias, gobernanza, energía

- [78] FINMA. (s. f.). *Guidelines for enquiries regarding the regulatory framework for ICOs*.
- [79] Parlamento Europeo y Consejo de la UE. (2023). *Reglamento (UE) 2023/1114 (MiCA)*.
- [80] Free Software Foundation. (2007). *GNU Affero General Public License, versión 3* (cláusula 13).
- [81] Trollope, R. (2025). *Redis is now available under the AGPLv3 open source license*. Redis blog. <https://redis.io/blog/agplv3/>; Banon, S. (2024). *Elasticsearch is open source. Again!* Elastic blog. <https://www.elastic.co/blog/elasticsearch-is-open-source-again>
- [82] Wondrasek, J. A. (2026). *The open source license change pattern – MongoDB to Redis timeline 2018 to 2026 and what comes next*. SoftwareSeni blog. <https://www.softwareseni.com/the-open-source-license-change-pattern-mongodb-to-redis-timeline-2018-to-2026-and-what-comes-next/>  fuente secundaria; respaldo parcial de la afirmación.
- [83] Agencia Internacional de la Energía. (2025). *Energy and AI*.
- [84] Guidi, G., et al. (2026). Assessing the carbon emissions and energy consumption of U.S. hyperscale data centers. *arXiv*. <https://arxiv.org/abs/2606.05420>
- [85] Fadel Argerich, M., et al. (2026). Watt Counts: Energy-aware benchmark for sustainable LLM inference on heterogeneous GPU architectures. *arXiv*. <https://arxiv.org/abs/2604.09048>
- [86] Uptime Institute. (2025). *Global data center survey 2025*.
- [87] Green Software Foundation. (2024). *Software carbon intensity (SCI) specification* (ISO/IEC 21031:2024).

Nuevas en la versión 2 — alineamiento y matrices de sustitución

- [92] Henikoff, S., & Henikoff, J. G. (1992). Amino acid substitution matrices from protein blocks. *PNAS*, 89(22), 10915–10919. <https://www.pnas.org/doi/10.1073/pnas.89.22.10915>
- [93] Gotoh, O. (1982). An improved algorithm for matching biological sequences. *J. Mol. Biol.*, 162(3), 705–708. [https://doi.org/10.1016/0022-2836\(82\)90398-9](https://doi.org/10.1016/0022-2836(82)90398-9)
- [94] Notredame, C., Higgins, D. G., & Heringa, J. (2000). T-Coffee: A novel method for fast and accurate multiple sequence alignment. *J. Mol. Biol.*, 302(2), 205–217. https://tcoffee.org/Publications/Ps_pdf/tcoffee.pdf
- [95] Eddy, S. R. (1998). Profile hidden Markov models. *Bioinformatics*, 14(9), 755–763. <https://doi.org/10.1093/bioinformatics/14.9.755>

Costos aprendidos y alineamiento de texto

- [96] Ristad, E. S., & Yianilos, P. N. (1998). Learning string-edit distance. *IEEE TPAMI*, 20(5), 522–531.
- [97] Pavlick, E., Rastogi, P., Ganitkevitch, J., Van Durme, B., & Callison-Burch, C. (2015). PPDB 2.0. *ACL-IJCNLP 2015*, 425–430. <https://doi.org/10.3115/v1/P15-2070>

Incertidumbre y acuerdo entre generaciones

- [98] Kuhn, L., Gal, Y., & Farquhar, S. (2023). Semantic uncertainty. *ICLR*. <https://arxiv.org/abs/2302.09664>
- [99] Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature*, 630, 625–630. <https://doi.org/10.1038/s41586-024-07421-0>
- [100] Manakul, P., Liusie, A., & Gales, M. (2023). SelfCheckGPT. *EMNLP*, 9004–9017. <https://doi.org/10.18653/v1/2023.emnlp-main.557>
- [101] Soiffer, D., Kolawole, S., & Smith, V. (2025). Semantic agreement enables efficient open-ended LLM cascades. *arXiv*. <https://arxiv.org/abs/2509.21837> (preprint, no revisado por pares)

Ensamblaje, cobertura, repeticiones y unicidad

- [102] Bresler, G., Bresler, M., & Tse, D. (2013). Optimal assembly for high throughput shotgun sequencing. *BMC Bioinformatics*, 14(Suppl 5), S18. <https://arxiv.org/abs/1301.0068>
- [103] Motahari, A. S., Bresler, G., & Tse, D. N. C. (2013). Information theory of DNA shotgun sequencing. *IEEE Trans. Inf. Theory*, 59(10), 6273–6288. <https://web.stanford.edu/~dntse/papers/mbt.pdf>
- [104] Compeau, P. E. C., Pevzner, P. A., & Tesler, G. (2011). How to apply de Bruijn graphs to genome assembly. *Nature Biotechnology*, 29(11), 987–991. <https://doi.org/10.1038/nbt.2023>
- [105] Myers, E. W. (1995). Toward simplifying and accurately formulating fragment assembly. *J. Comput. Biol.*, 2(2), 275–290. <https://doi.org/10.1089/cmb.1995.2.275>
- [106] Weber, J. L., & Myers, E. W. (1997). Human whole-genome shotgun sequencing. *Genome Research*, 7(5), 401–409.
- [107] Myers, E. W., et al. (2000). A whole-genome assembly of Drosophila. *Science*, 287(5461), 2196–2204.
- [108] Gao, S., Sung, W.-K., & Nagarajan, N. (2011). Opera: Reconstructing optimal genomic scaffolds. *J. Comput. Biol.*, 18(11), 1681–1691. <https://doi.org/10.1089/cmb.2011.0170>
- [109] Medvedev, P., Pham, S., Chaisson, M., Tesler, G., & Pevzner, P. (2011). Paired de Bruijn graphs. *RECOMB 2011*, LNCS 6577, 238–251. https://doi.org/10.1007/978-3-642-20036-6_22

Plegado, dominios y control de calidad

- [110] Porter, L. L., & Rose, G. D. (2012). A thermodynamic definition of protein domains. *PNAS*, 109(24), 9420–9425. <https://doi.org/10.1073/pnas.1202604109>
- [111] Han, J.-H., Batey, S., Nickson, A. A., Teichmann, S. A., & Clarke, J. (2007). The folding and evolution of multidomain proteins. *Nature Rev. Mol. Cell Biol.*, 8(4), 319–330. <https://doi.org/10.1038/nrm2144>
- [112] Bashton, M., & Chothia, C. (2007). The generation of new protein functions by the combination of domains. *Structure*, 15(1), 85–99. <https://doi.org/10.1016/j.str.2006.11.009>

- [113] Zhang, Y., Chandonia, J.-M., Ding, C., & Holbrook, S. R. (2005). Comparative mapping of sequence-based and structure-based protein domains. *BMC Bioinformatics*, 6, 77. <https://doi.org/10.1186/1471-2105-6-77>
- [114] Schaeffer, R. D., et al. (2023). ECOD domain classification of 48 whole proteomes from AlphaFold Structure Database using DPAM2. *PLoS Comput. Biol.*
- [115] Zhu, K., Su, H., Peng, Z., & Yang, J. (2023). A unified approach to protein domain parsing with inter-residue distance matrix. *Bioinformatics*, 39(2), btad070. <https://doi.org/10.1093/bioinformatics/btad070>
- [116] Gottesman, S., Wickner, S., & Maurizi, M. R. (1997). Protein quality control: Triage by chaperones and proteases. *Genes & Development*, 11, 815–823.
- [117] Xu, Z., Horwich, A. L., & Sigler, P. B. (1997). The crystal structure of the asymmetric GroEL–GroES–(ADP)₇ chaperonin complex. *Nature*, 388(6644), 741–750. <https://doi.org/10.1038/41944>
- [118] Netzer, W. J., & Hartl, F. U. (1997). Recombination of protein domains facilitated by co-translational folding in eukaryotes. *Nature*, 388(6640), 343–349. <https://doi.org/10.1038/41024>
- [119] Marsh, J. A., et al. (2013). Protein complexes are under evolutionary selection to assemble via ordered pathways. *Cell*, 153(2), 461–470. <https://doi.org/10.1016/j.cell.2013.02.044>
- [120] Shiber, A., et al. (2018). Cotranslational assembly of protein complexes in eukaryotes revealed by ribosome profiling. *Nature*, 561(7722), 268–272. <https://doi.org/10.1038/s41586-018-0462-y>

Teoría de la información y codificación

- [121] Shannon, C. E. (1959). Coding theorems for a discrete source with a fidelity criterion. *IRE Int. Convention Record*, 7, 325–350.
- [122] Cover, T. M., & Thomas, J. A. (1991). *Elements of information theory*, cap. 13. Wiley.
- [123] Nagle, A., Girish, A., Bondaschi, M., Gastpar, M., Makkuva, A. V., & Kim, H. (2024). Fundamental limits of prompt compression: A rate–distortion framework for black-box language models. *NeurIPS*. <https://arxiv.org/abs/2407.15504>
- [124] Luby, M. (2002). LT codes. *FOCS 2002*, 271–282. <https://doi.org/10.1109/SFCS.2002.1181950>
- [125] Shokrollahi, A. (2006). Raptor codes. *IEEE Trans. Inf. Theory*, 52(6), 2551–2567.
- [126] Weatherspoon, H., & Kubiawicz, J. D. (2002). Erasure coding vs. replication: A quantitative comparison. *IPTPS 2002*, LNCS 2429, 328–337.
- [127] Mallick, A., Chaudhari, M., Palanikumar, G., Sheth, U., & Joshi, G. (2019). Rateless codes for near-perfect load balancing in distributed matrix-vector multiplication. *Proc. ACM Meas. Anal. Comput. Syst.*, 3(3), art. 58.
- [128] Kosaian, J., Rashmi, K. V., & Venkataraman, S. Learning a code: Machine learning for approximate non-linear coded computation. *arXiv*. <https://arxiv.org/abs/1806.01259>
- [129] Soleymani, M., Ali, R. E., MahdaviFar, H., & Avestimehr, A. S. (2022). ApproxIFER. *AAAI-22*, 8342–8350.

Computación voluntaria (nuevas)

- [130] Anderson, D. P. (2004). BOINC: A system for public-resource computing and storage. *5th IEEE/ACM Int. Workshop on Grid Computing*. <https://doi.org/10.1109/GRID.2004.14>
- [131] Anderson, D. P., & Fedak, G. (2006). The computational and storage potential of volunteer computing. *arXiv*. <https://arxiv.org/abs/cs/0602061>
- [132] Anderson, D. P. (2018). BOINC: A platform for volunteer computing. *arXiv*. <https://arxiv.org/abs/1903.01699>
- [133] Sarmenta, L. F. G. (2002). Sabotage-tolerance mechanisms for volunteer computing systems. *Future Generation Computer Systems*, 18(4), 561–572.

Inferencia descentralizada y descompuesta (nuevas)

- [134] Dahshan, M., Mamun, Q., & Debnath, T. (2026). SWARM-LLM: Collaborative inference for edge-based small language models. *IEEE VTC2026-Spring*.
- [135] Zhang, H., et al. (2025). If multi-agent debate is the answer, what is the question? *arXiv*. <https://arxiv.org/abs/2502.08788>

Swarmby AI — Sebastián A. Espinoza-Ulloa · Versión 2.0. Compañero en inglés: *WHITEPAPER_V2_EN.md*. Extensión focalizada: *WHITEPAPER_EXT_ES.md*. Versión anterior, superada: *WHITEPAPER_ES.md* (v1.4).